

university of groningen

Computer Architecture (WBCS010-05)

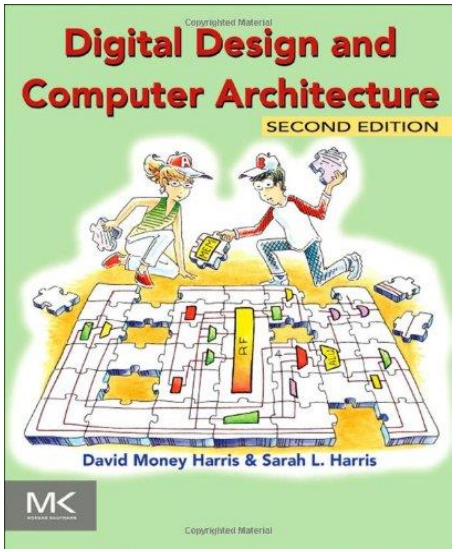
Lecture 2: Combinational Logic

Farhad Merchant
f.a.merchant@rug.nl

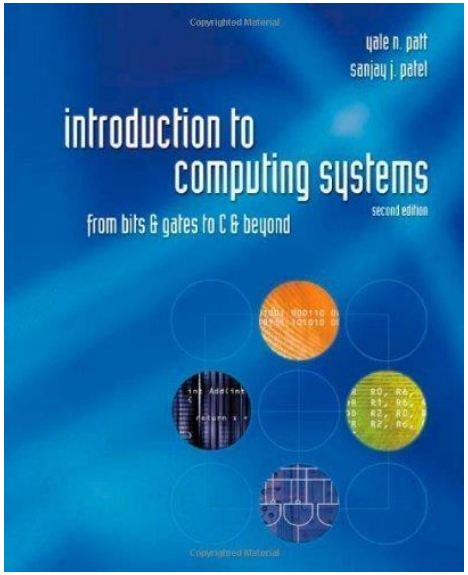
university of groningen

Reading Assignments for This/Next Week

- Chapters 1-2 in Harris & Harris



- Chapters 1,2,3 in Patt and Patel



- Supplementary Lecture Slides on Binary Numbers

university of groningen

What Will We Learn Today?

- Basic building blocks of modern computers
  - Transistors
  - Logic gates
- Boolean algebra
- Combinational logic circuits
- How to use Boolean algebra to represent combinational circuits
- Minimizing logic circuits

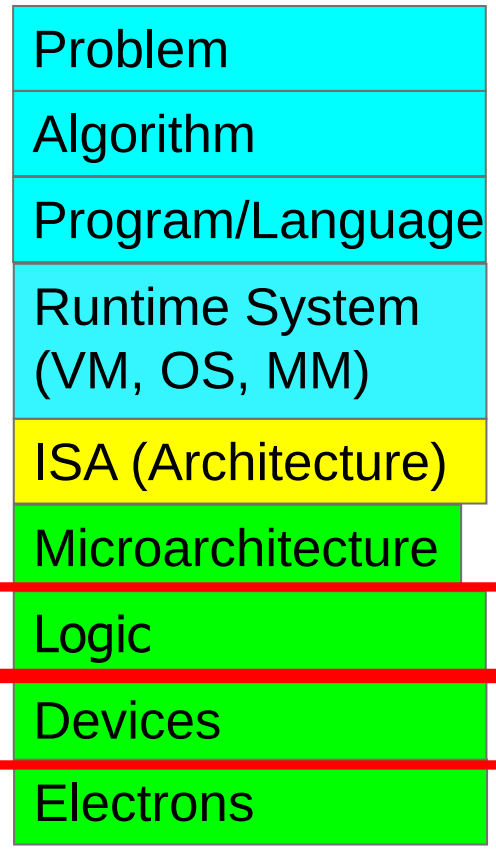
All Computers are Built Upon the Same Building Blocks

Transistors

university of groningen

Transistors

- **Computers are built from very large numbers of very small (and relatively simple) structures: transistors**
  - Intel 4004, in 1971, had 2300 MOS transistors
  - Intel's Pentium IV microprocessor, 2000, was made up of more than 42 **M**illion MOS transistors
  - Apple's M2 Max, offered for sale in 2022, is made up of more than 67 **B**illion MOS transistors
  - Micron's V-NAND flash memory (2022), is made up of more than 5 **T**rillion MOS transistors



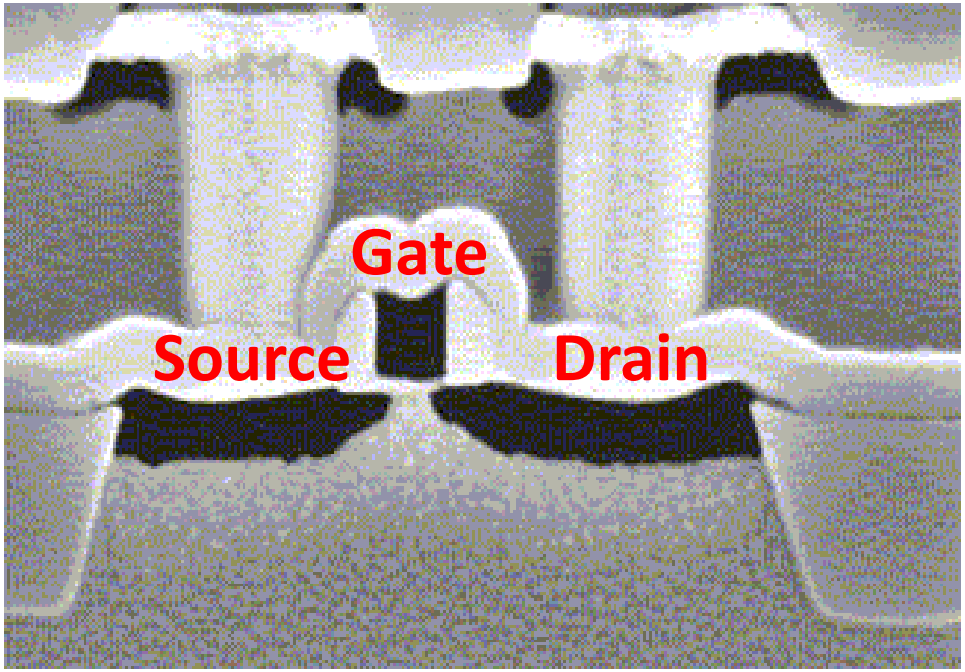
› **This lecture**

- How the MOS transistor works (as logic element)
- How transistors are connected to form logic gates
- How logic gates are interconnected to form larger units that are needed to construct a computer

university of groningen

MOS Transistor

- By combining
  - Conductors (**M**etal)
  - Insulators (**O**xide)
  - **S**emiconductors



- We get a Transistor (MOS)
- Why is this useful?
  - We can combine many of these to realize simple logic gates
- The electrical properties of metal-oxide semiconductors are well beyond the scope of what we want to understand in this course
  - They are below our lowest level of abstraction

Sections 1.6 and 1.7 in Harris and Harris provide some lower-level explanations

university of groningen

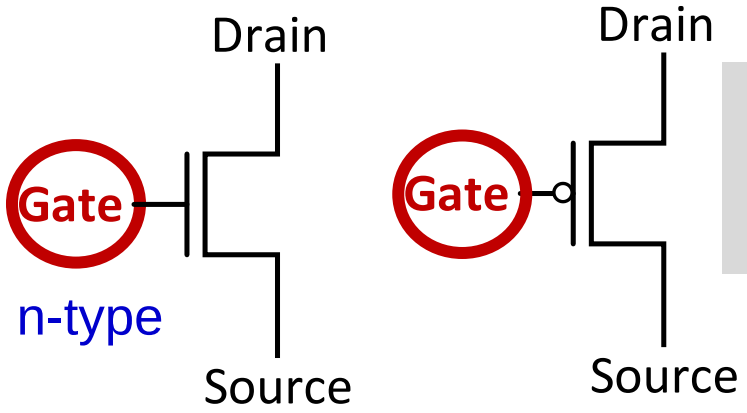
Different Types of MOS Transistors

› There are t[illegible]es of MO[illegible]nsistors: n-type and p-type



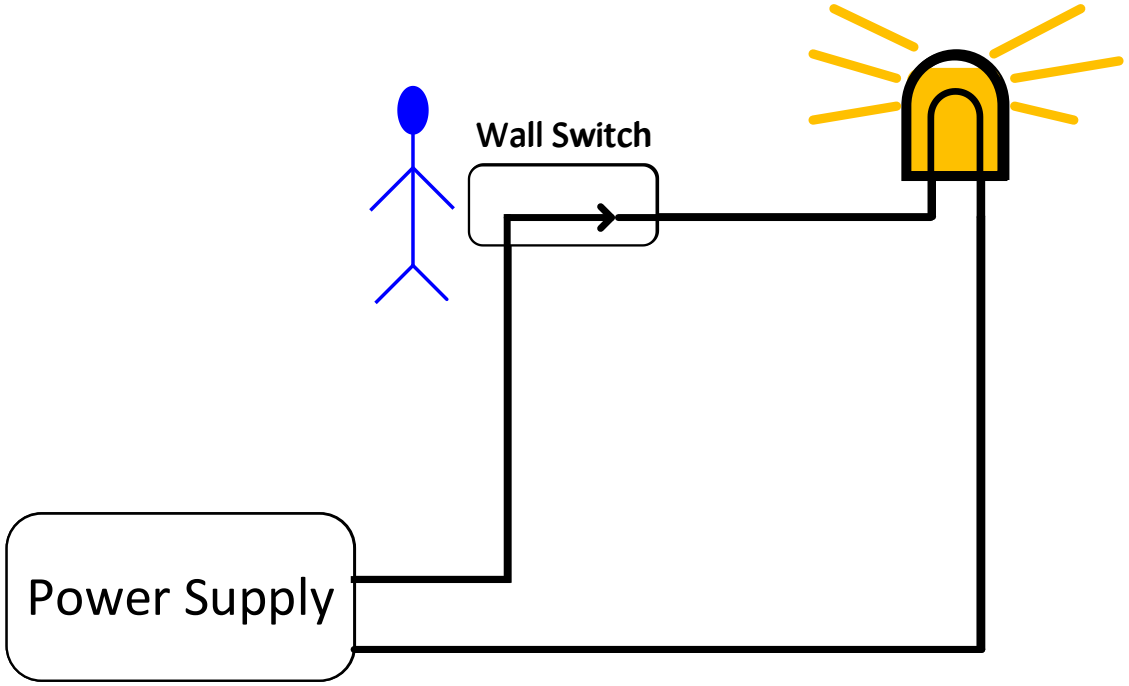
› They both operate "logically," very similar to the way wall switches work

The circuit is closed when the gate is supplied with 3V



The circuit is closed when the gate is supplied with 0V

How Does a Wall Switch Work?

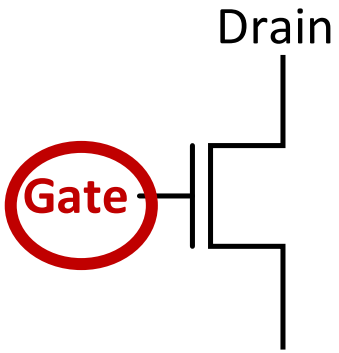


- In order for the lamp to glow, electrons must flow
- In order for electrons to flow, there must be a closed circuit from the power supply to the lamp and back to the power supply
- The lamp can be turned on and off by simply manipulating the wall switch to make or break the closed circuit

university of groningen

Transistor Works As a Switch

› Instead of the wall switch, we could use an n-type or a p-type MOS transistor to make or break the closed circuit



Schematic of an n-type MOS transistor

If the gate of an n-type transistor is supplied with a **high** voltage, the connection from source to drain acts like a piece of wire (i.e., the circuit is closed)

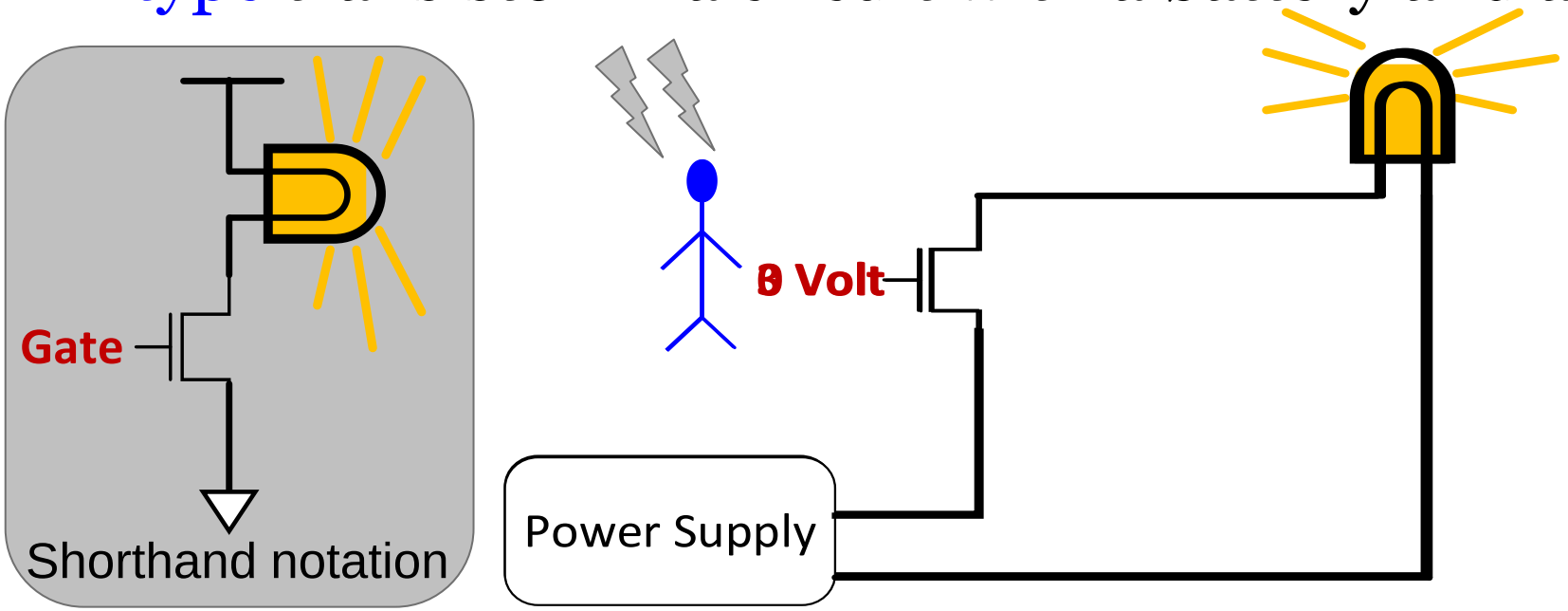
Depending on the technology, high voltage can range from 0.3V to 3V

If the gate of the n-type transistor is supplied with **zero** voltage, the connection between the source and drain is broken (i.e., the circuit is open)

university of groningen

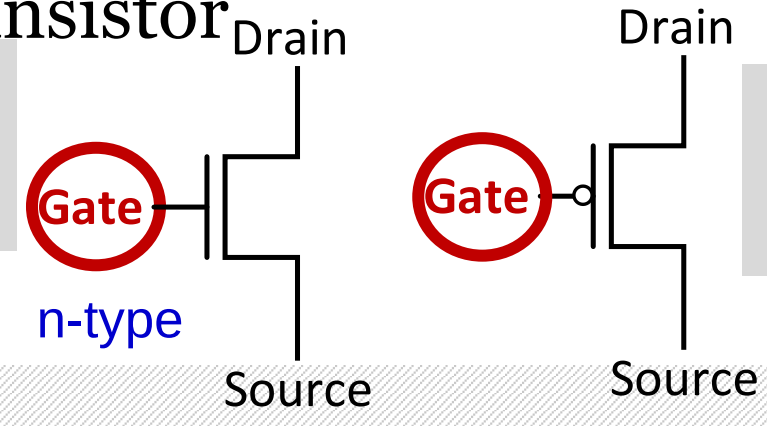
Abstraction: Transistor As a Switch

› n-type transistor in a circuit with a battery and a bulb



› p-type transistor works in exactly the opposite fashion from n-type transistor

The circuit is closed when the gate is supplied with 3V



The circuit is closed when the gate is supplied with 0V

p-type

university of groningen

Logic Gates

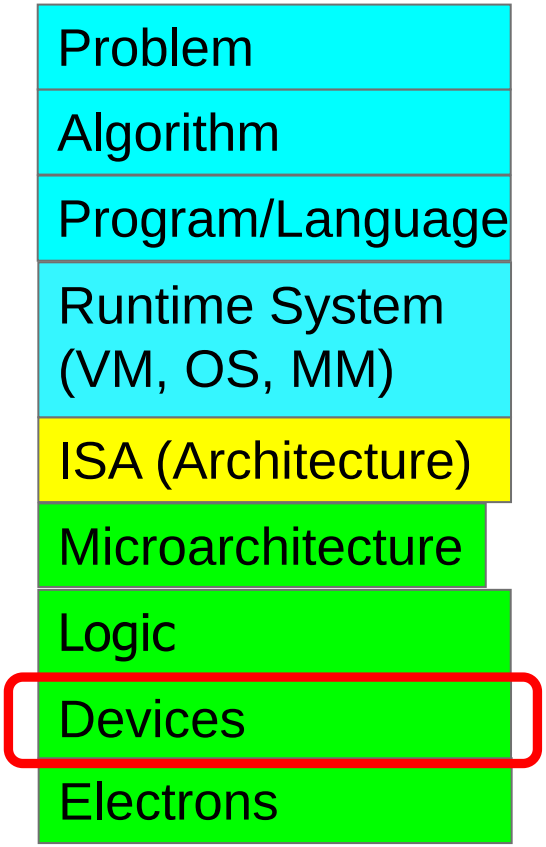
university of groningen

One Level Higher in the Abstraction

- **Now, we know how a MOS transistor works**
- How do we build logic structures out of MOS transistors?

  - We construct basic logical units out of individual MOS transistors

  - These logical units are called logic gates
    - They implement simple Boolean functions



George Boole, "The Mathematical Analysis of Logic," 1847.

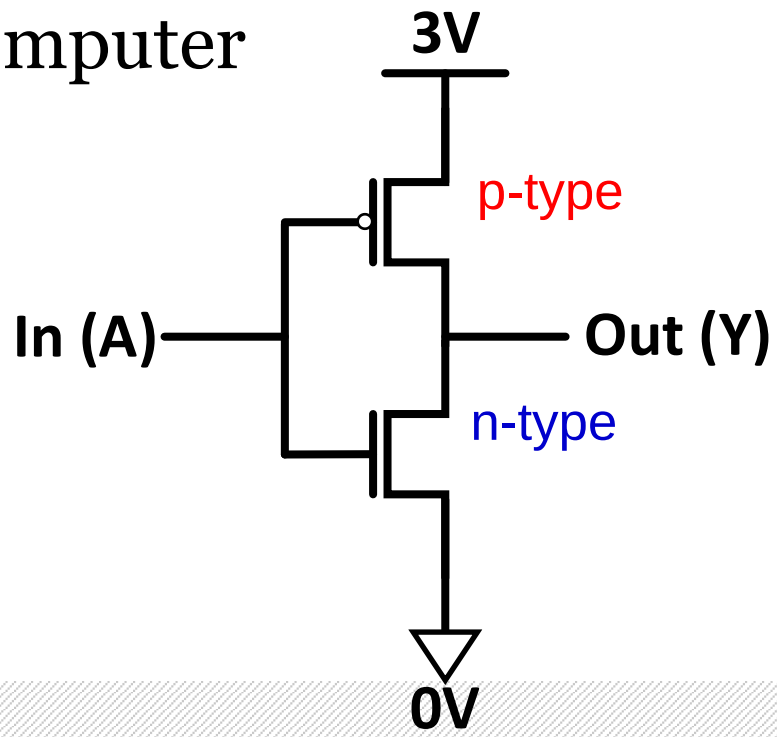
university of groningen

Making Logic Blocks Using CMOS Technology

- Modern computers use both n-type and p-type transistors, i.e. Complementary MOS (CMOS) technology

nMOS + pMOS = CMOS

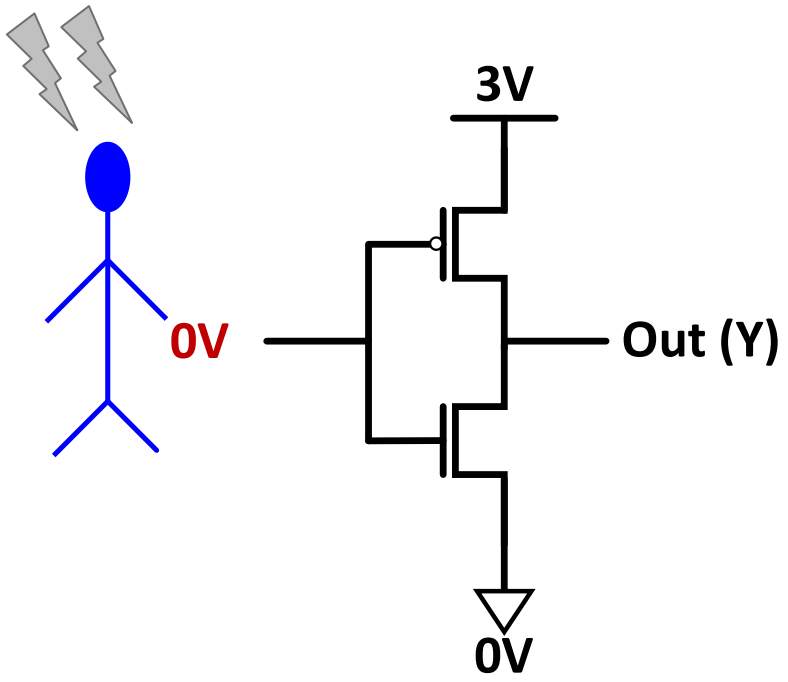
- The simplest logic structure that exists in a modern computer

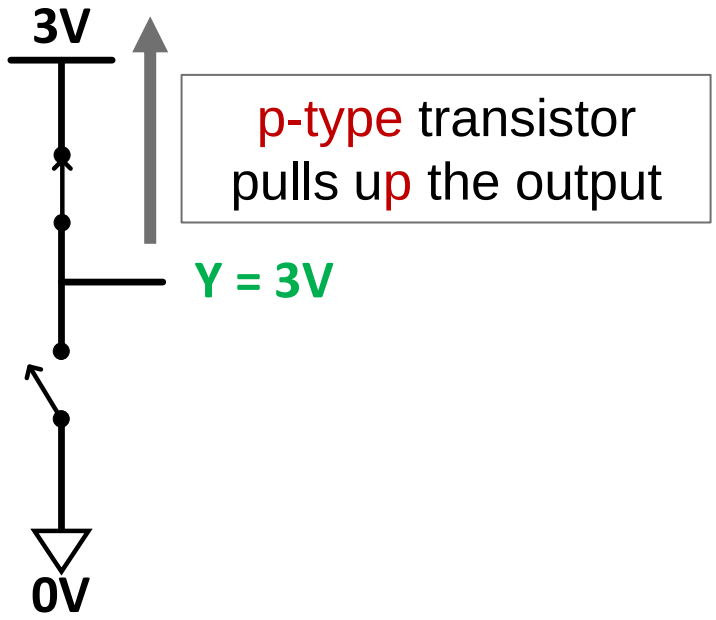


What does this circuit do?

Functionality of Our CMOS Circuit

What happens when the input is connected to 0V?

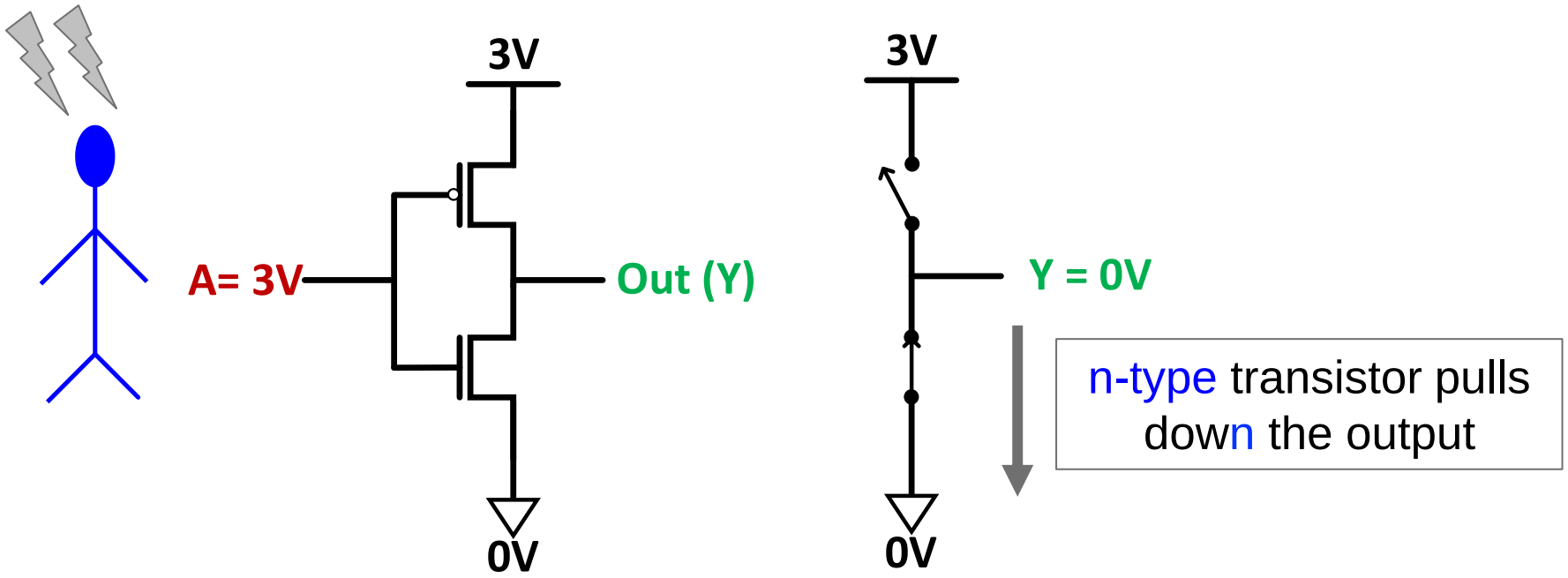




p-type transistors are good at pulling u**p** the voltage

Functionality of Our CMOS Circuit

What happens when the input is connected to 3V?

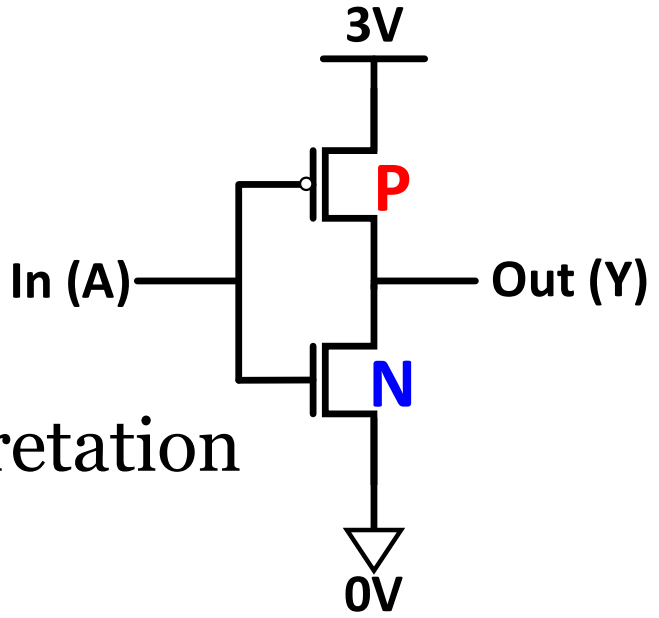


n-type transistors are good at pulling down the voltage

university of groningen

CMOS NOT Gate (Inverter)

- This is the CMOS NOT Gate
- Why do we call it NOT?
  - If A = 0V then Y = 3V
  - If A = 3V then Y = 0V
- **Digital circuit:** one possible interpretation
  - Interpret 0V as logical (binary) 0 value
  - Interpret 3V as logical (binary) 1 value



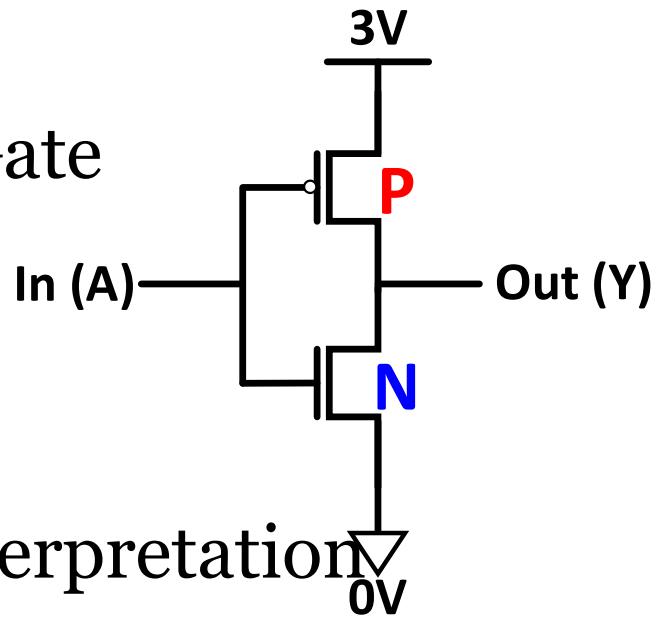
A	P	N	Y
0	**ON**	OFF	1
1	OFF	**ON**	0

$$Y = \overline{\overline{A}}$$

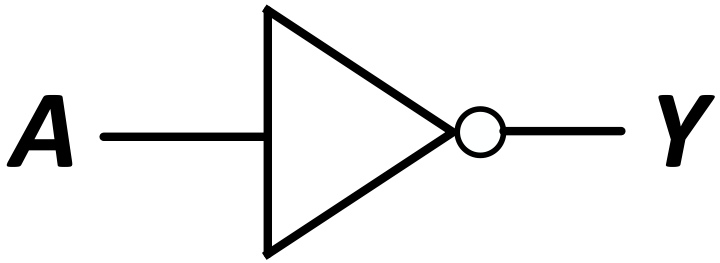
university of groningen

CMOS NOT Gate (Inverter)

- This is actually the CMOS NOT Gate
- Why do we call it NOT?
  - If A = 0V then Y = 3V
  - If A = 3V then Y = 0V
- **Digital circuit:** one possible interpretation
  - Interpret 0V as logical (binary) 0 value
  - Interpret 3V as logical (binary) 1 value



$$Y = \bar{A}$$



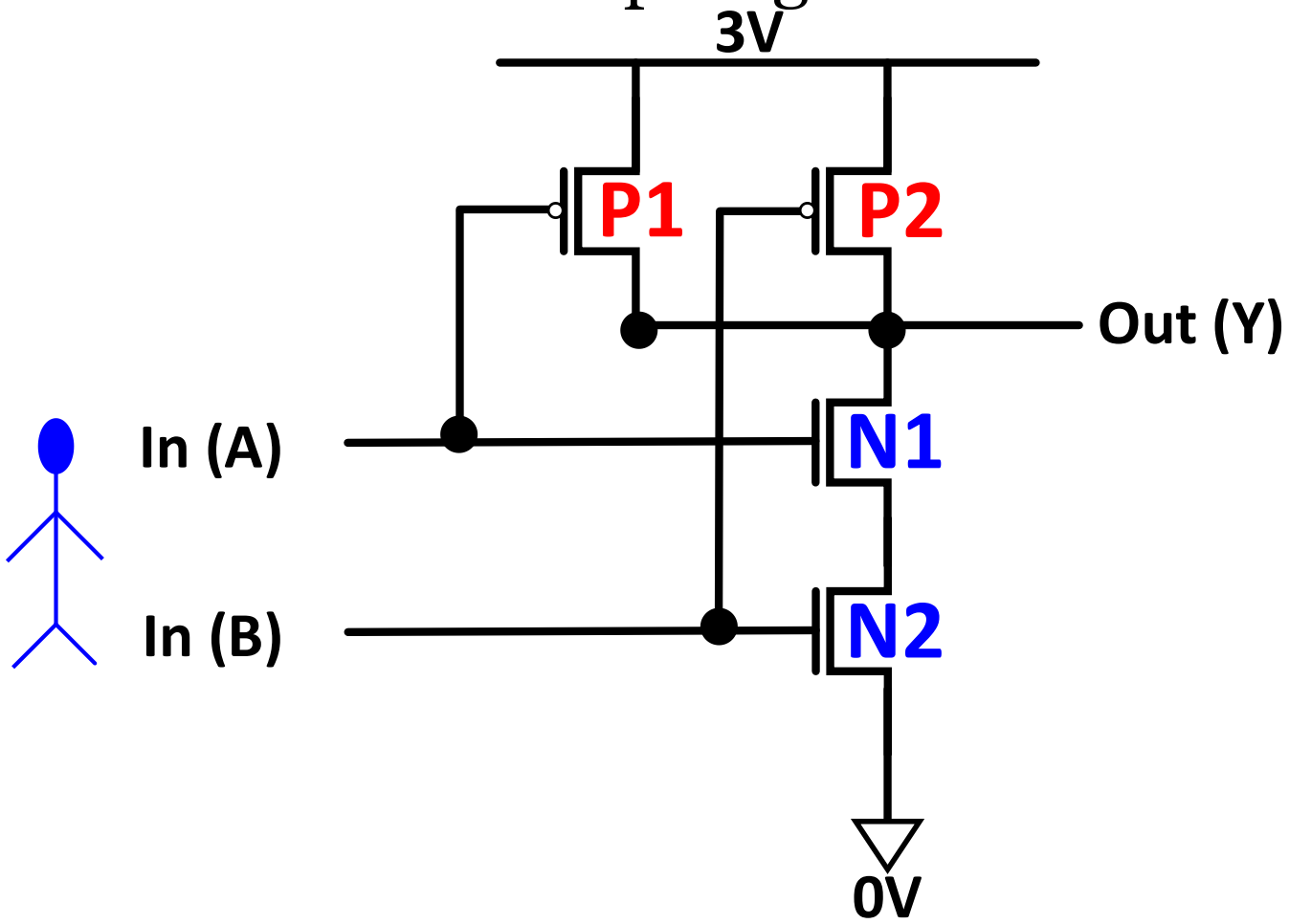
We call this a NOT gate
or an inverter
(bubble indicates inversion)

Truth table: shows the logical output of the circuit for each possible input

A	Y
0	1
1	0

Another CMOS Gate: What Is This?

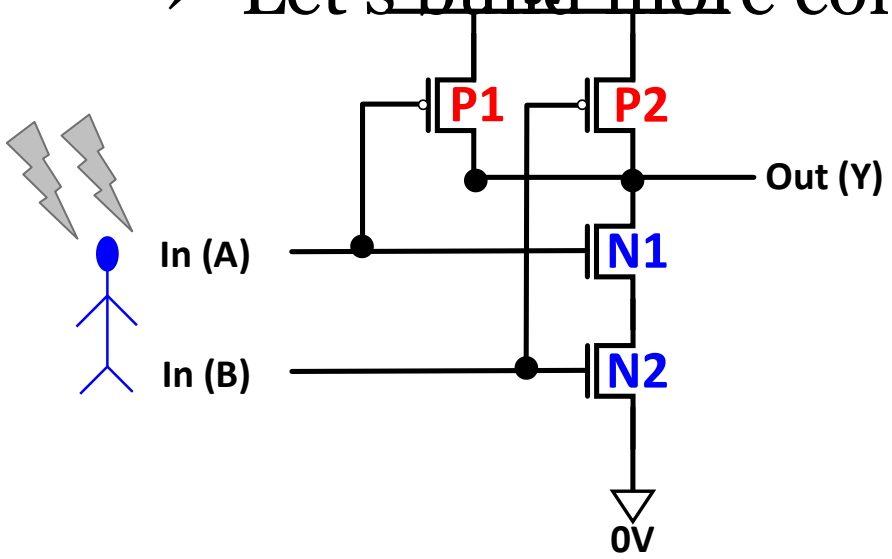
› Let's build more complex gates!



university of groningen

CMOS NAND Gate

› Let's build more complex gates!



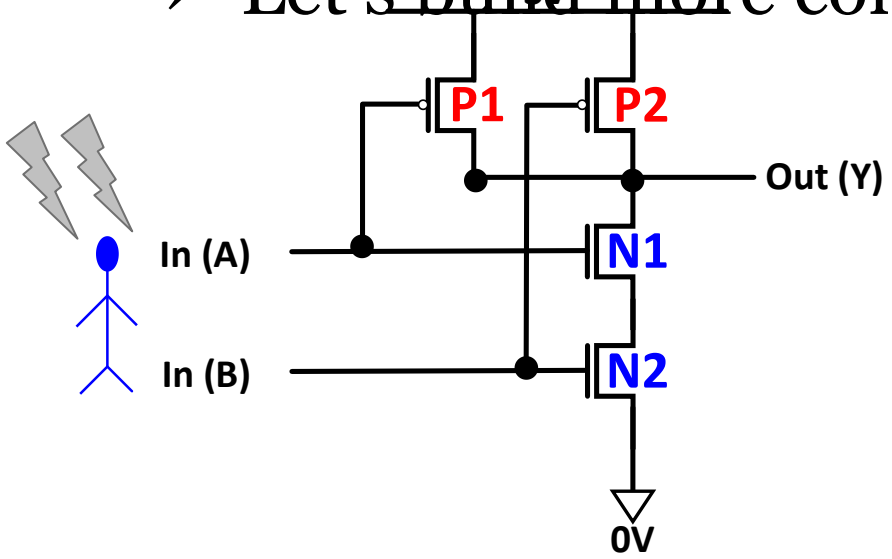
$$Y = \overline{A \cdot B} = \overline{AB}$$

A	B	P1	P2	N1	N2	Y
0	0	**ON**	**ON**	OFF	OFF	1
0	1	**ON**	OFF	OFF	**ON**	1
1	0	OFF	**ON**	**ON**	OFF	1
1	1	OFF	OFF	**ON**	**ON**	0

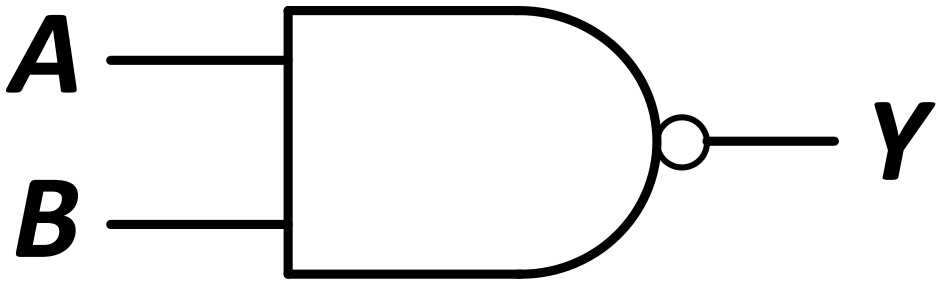
- P1 and P2 are in parallel; **only one must be ON to pull up the output to 3V**
- N1 and N2 are connected in series; **both must be ON to pull down the output to 0V**

CMOS NAND Gate

› Let's build more complex gates!



$$Y = \overline{A \cdot B} = \overline{AB}$$



A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

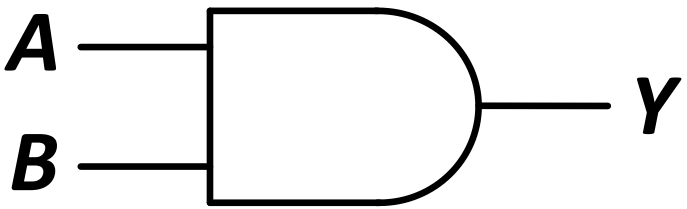
We call this a NAND gate
(bubble indicates inversion)

CMOS AND Gate

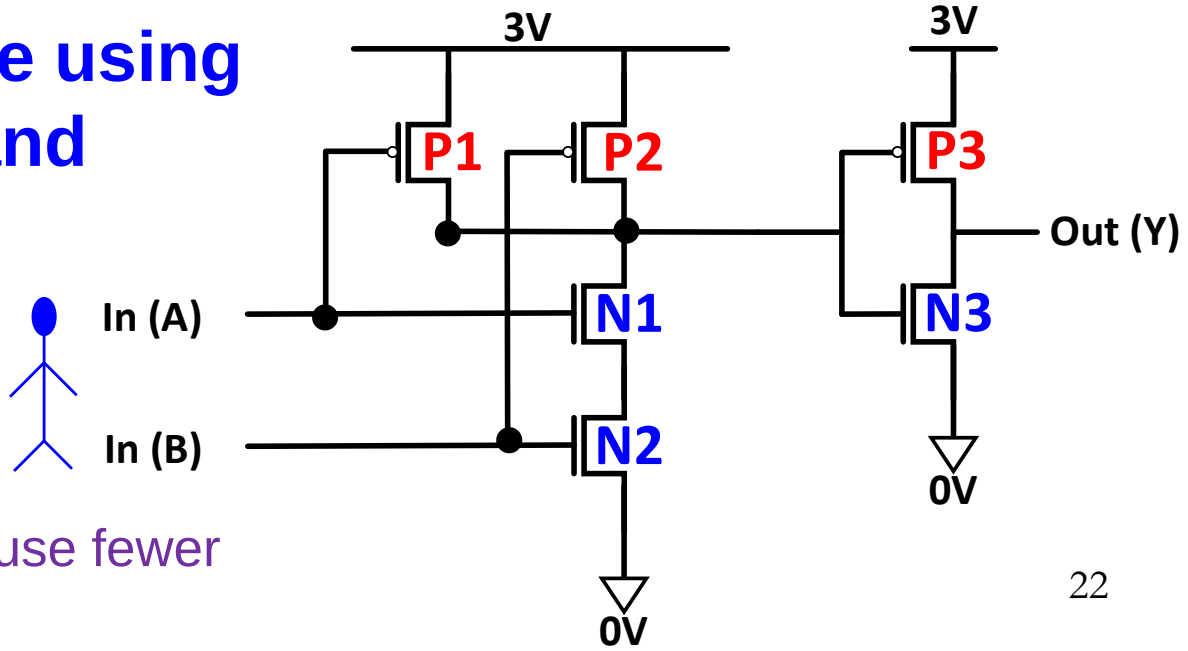
$$Y = A \cdot B = AB$$

› How can we make an AND gate?

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

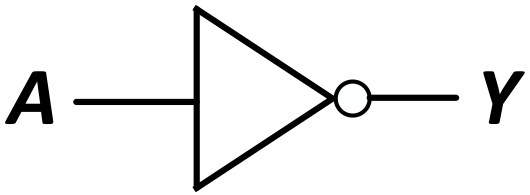


We make an AND gate using one NAND gate and one NOT gate

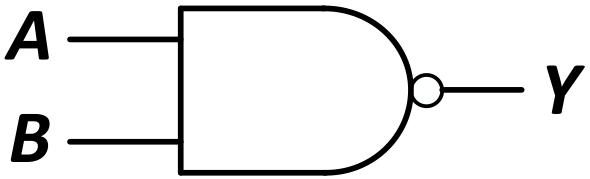


Food for thought: Can we not use fewer transistors for the AND gate?

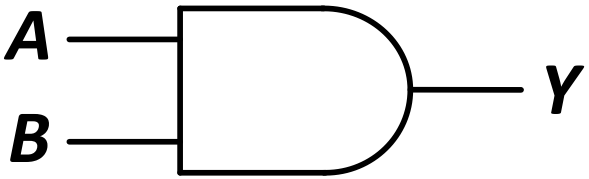
CMOS NOT, NAND, AND Gates



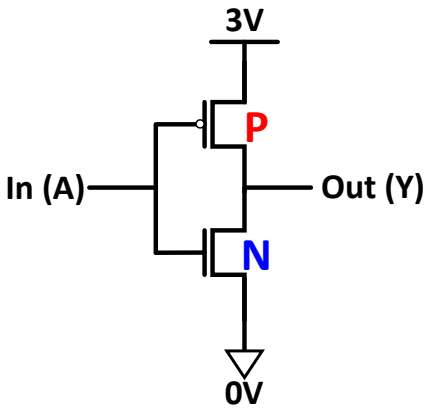
A	Y
0	1
1	0

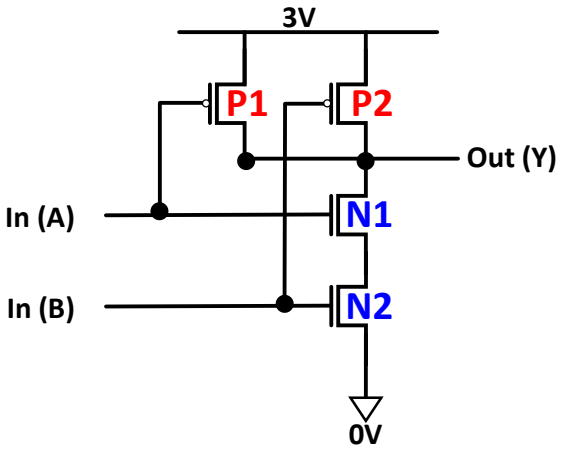


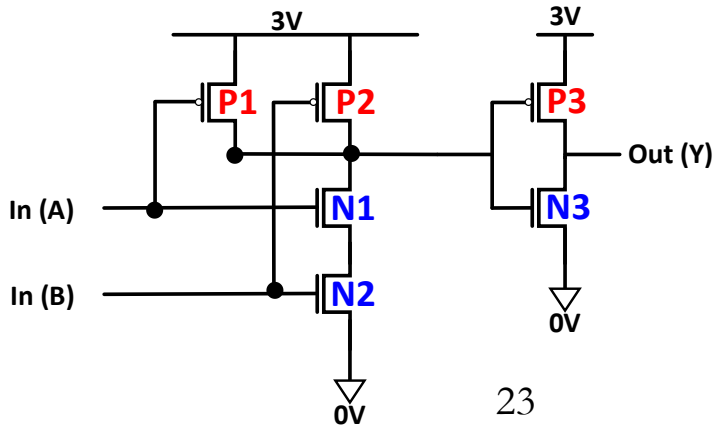
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1



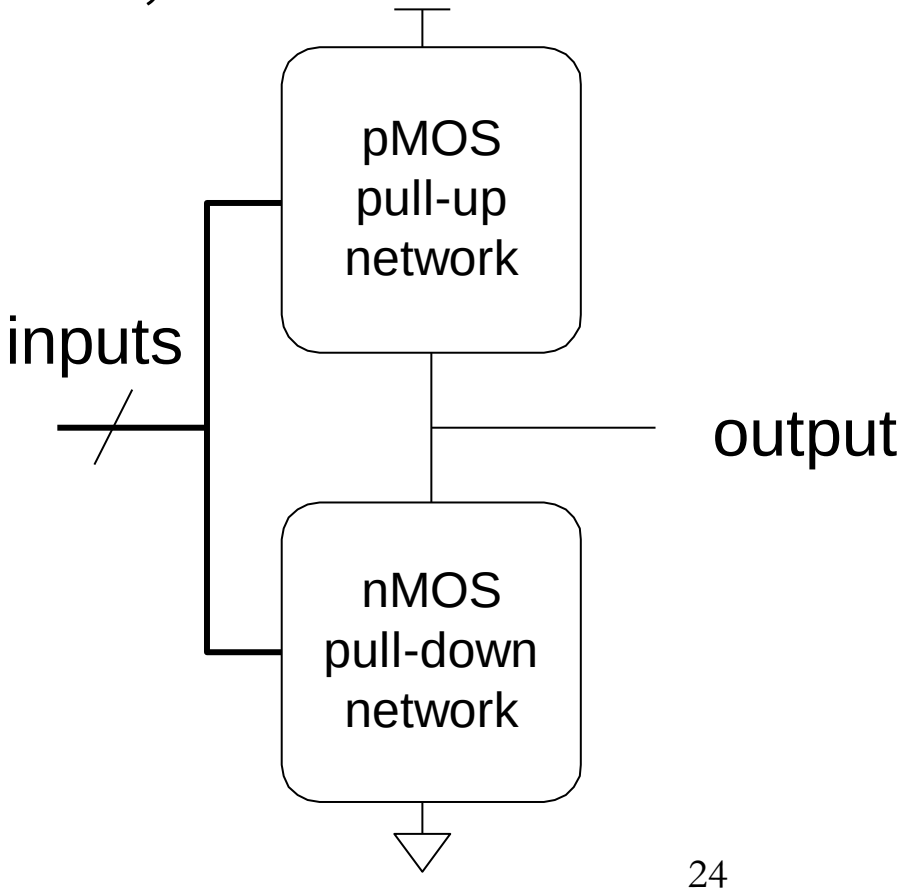




General CMOS Gate Structure

› The general form used to construct any inverting logic gate, such as: NOT, NAND, or NOR

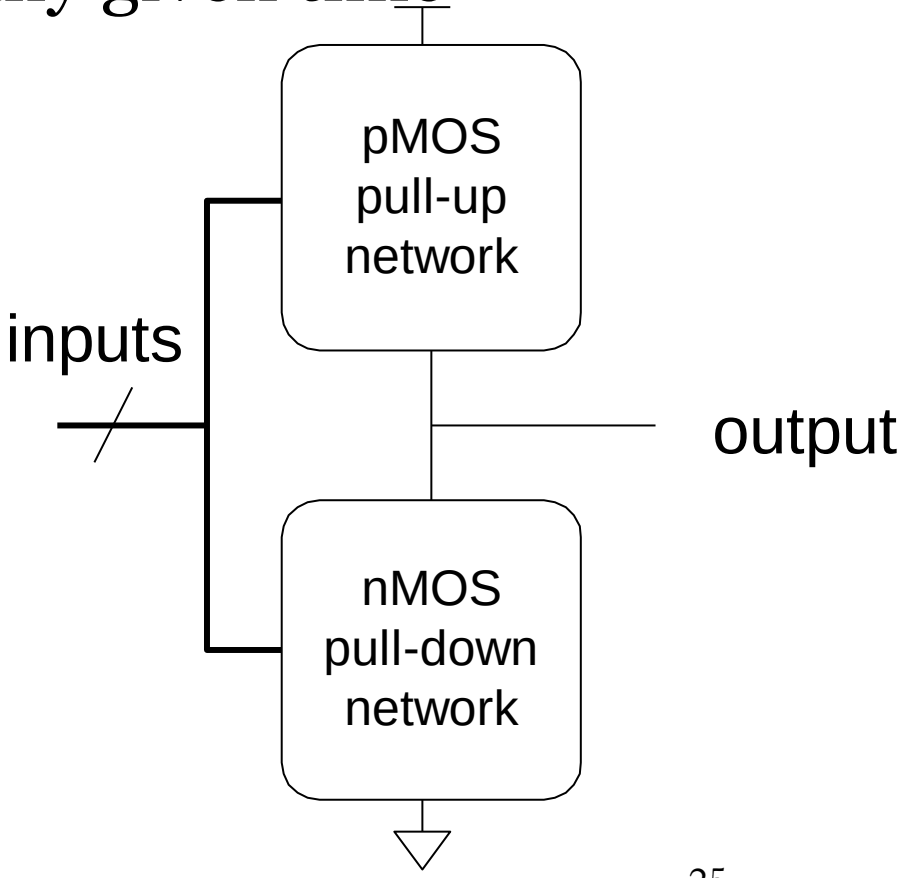
- The networks may consist of transistors in series or in parallel
- When transistors are in parallel, the network is **ON** if one of the transistors is **ON**
- When transistors are in series, the network is **ON** only if all transistors are **ON**



pMOS transistors are used for pull-u**p**
nMOS transistors are used for pull-dow**n**

General CMOS Gate Structure (II)

- Exactly one network should be ON, and the other network should be OFF at any given time
  - If both networks are ON at the same time, there is a short circuit → likely incorrect operation
  - If both networks are OFF at the same time, the output is floating → undefined



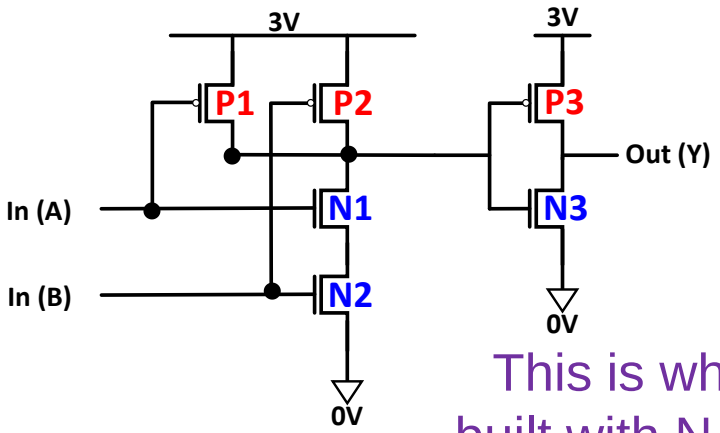
pMOS transistors are used for pull-u**p**
nMOS transistors are used for pull-dow**n**

university of groningen

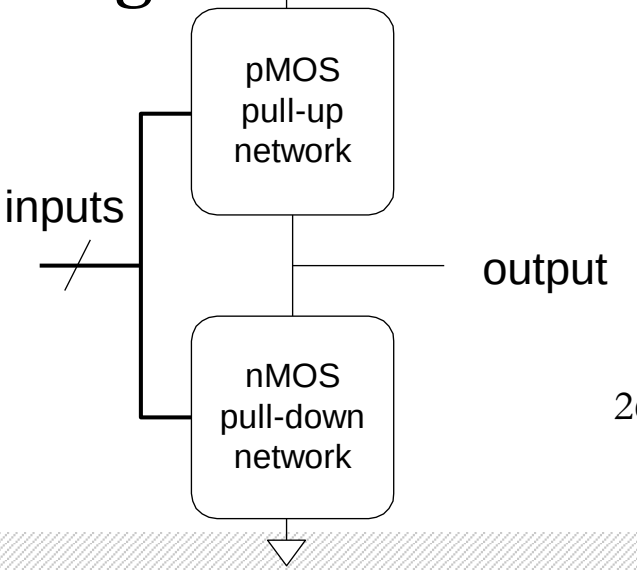
Digging Deeper: Why This Structure?

- MOS transistors are **imperfect** switches
- pMOS transistors pass 1's well but 0's poorly (holes carry charge)
- nMOS transistors pass 0's well but 1's poorly (electrons carry charge)

- **p**MOS transistors are good at "pulling u**p**" the output
- **n**MOS transistors are good at "pulling dow**n**" the output



This is why AND is built with NAND + NOT



See Section 1.7 in H&H

university of groningen

Digging Deeper: Latency

- Which one is slower?
  - Transistors in series
  - Transistors in parallel

- Series connections are slower than parallel connections
  - More resistance on the wire

- How do you alleviate this latency?
  - See H&H Section 1.7.8 for an example: pseudo-nMOS Logic

Used in the past when pMOS transistors could not be fabricated well

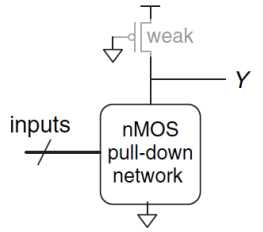


Figure 1.39 Generic pseudo-nMOS gate

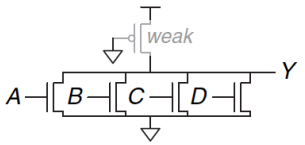


Figure 1.40 Pseudo-nMOS four-input NOR gate

university of groningen

Digging Deeper: Power Consumption

› Dynamic Power Consumption

- Power used to charge capacitance as signals change (0 ←→ 1)
- $C * V^2 * f$
  - C = capacitance of the circuit (wires and gates)
  - V = supply voltage
  - f = charging frequency of the capacitor

› Static Power Consumption

- Power used when signals do not change
- $V * I_{leakage}$
  - supply voltage * leakage current

› Energy Consumption

- Power * Time

See more in H&H Chapter 1.8

university of groningen

Common Logic Gates

Buffer



A	Z
0	0
1	1

AND



A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR



A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

XOR



A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

Inverter



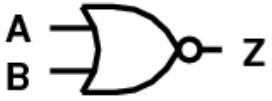
A	Z
0	1
1	0

NAND



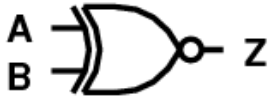
A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

NOR



A	B	Z
0	0	1
0	1	0
1	0	0
1	1	0

XNOR



A	B	Z
0	0	1
0	1	0
1	0	0
1	1	1

Larger Gates

- We can extend the gates to more than 2 inputs
- Example: 3-input AND gate, 10-input NOR gate
- See your readings

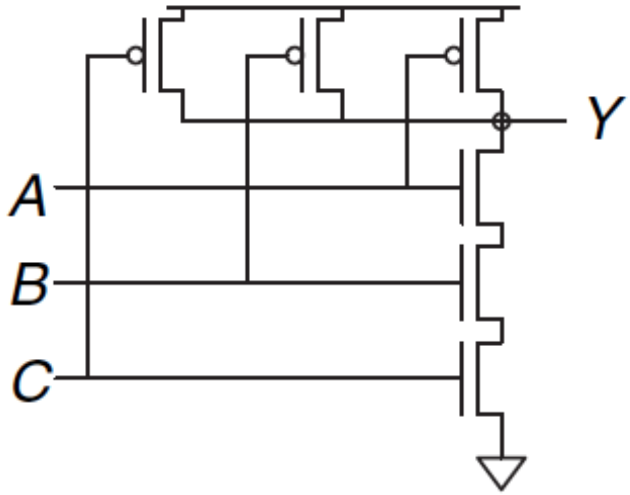
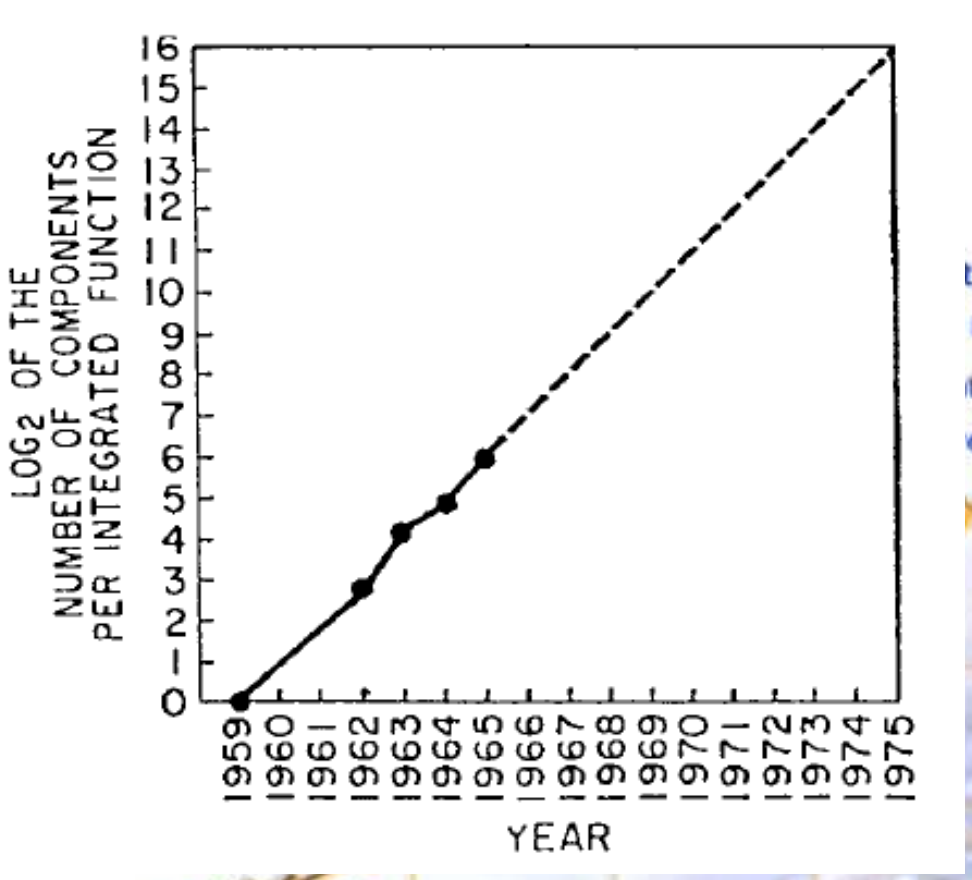


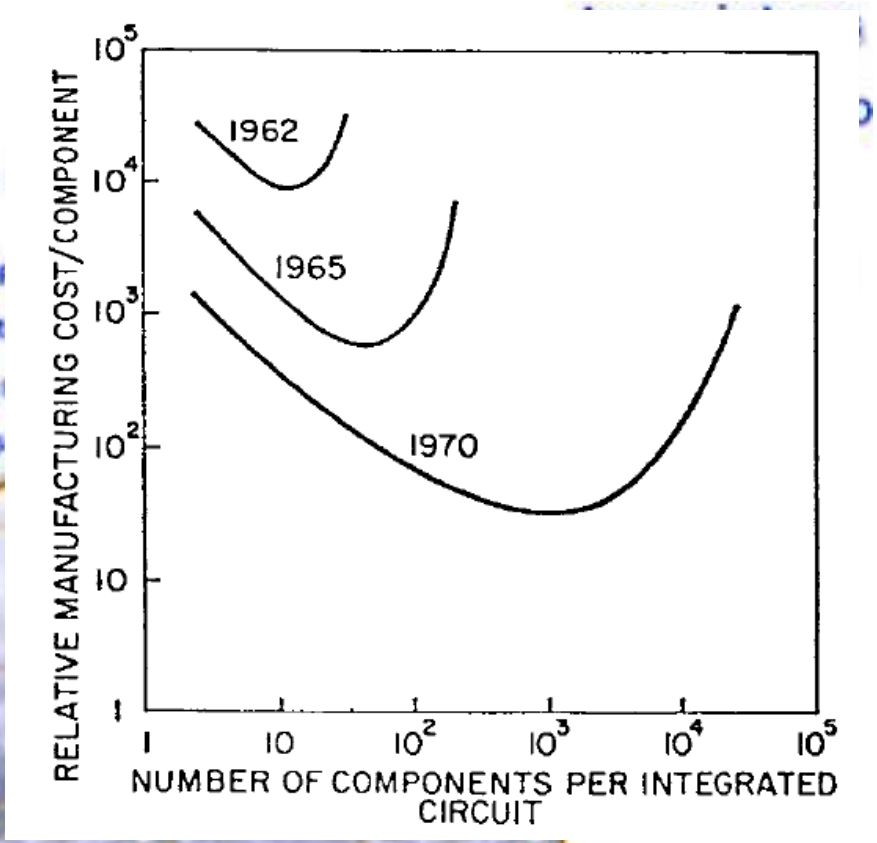
Figure 1.35 Three-input NAND gate schematic

Aside: Moore's Law: Enabler of Many Gates on a Chip

university of groningen

An Enabler: Moore's Law





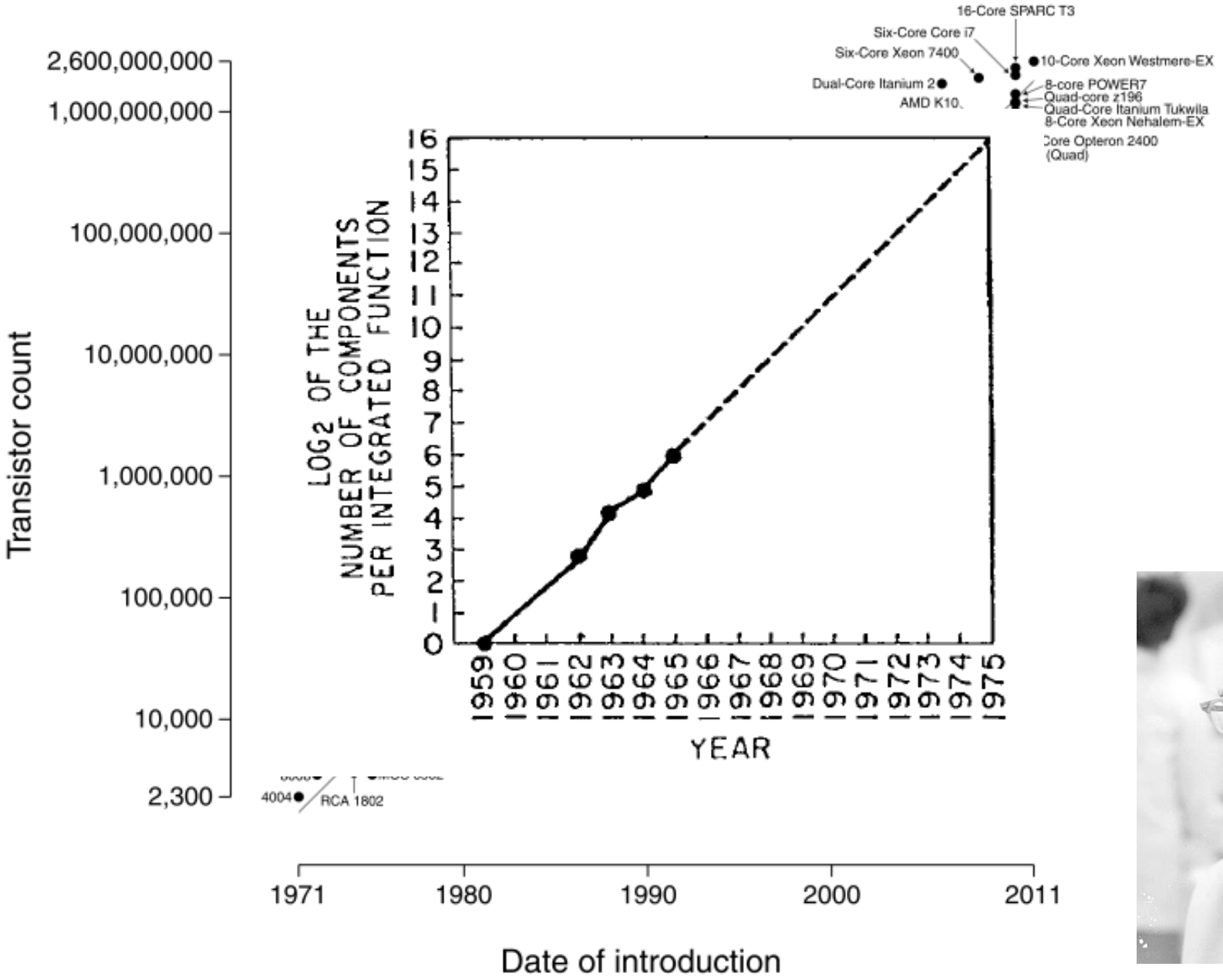


Moore, "Cramming more components onto integrated circuits," Electronics Magazine, 1965.

Component counts double every other year

Image source: Intel

Microprocessor Transistor Counts 1971-2011 & Moore's Law





Number of transistors on an integrated circuit doubles ~ every two years

Image source: Wikipedia

Recommended Reading

- Moore, "Cramming more components onto integrated circuits," Electronics Magazine, 1965.

- Only 3 pages

- A quote:

  "With unit cost falling as the number of components per circuit rises, by 1975 economics may dictate squeezing as many as 65 000 components on a single silicon chip."

- Another quote:

  "Will it be possible to remove the heat generated by tens of thousands of components in a single silicon chip?"

university of groningen

How Do We Keep Moore's Law: Innovation

- **Manufacturing smaller transistors/structures**
  - Some structures are already a few atoms in size

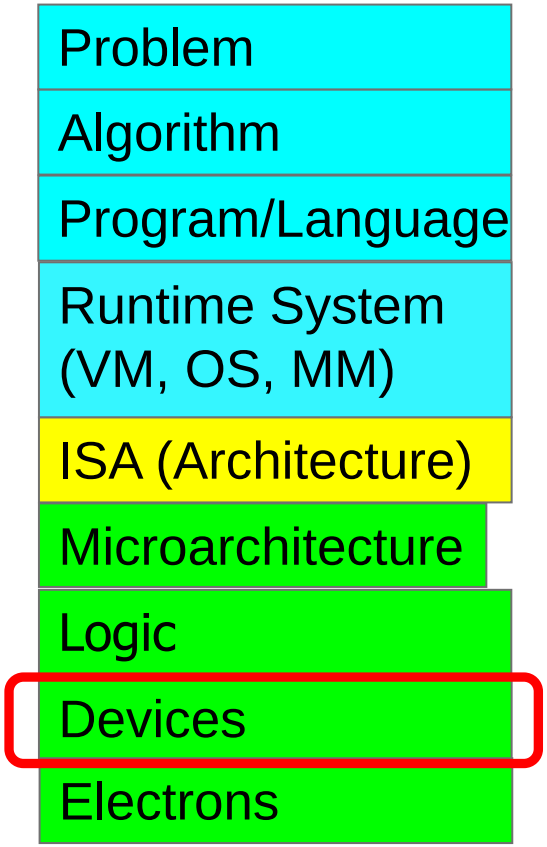
- **Using materials with better properties**
  - Copper instead of Aluminum (better conductor)
  - Hafnium Oxide, air for insulators
  - Making sure all materials are compatible is the challenge

- **Enabling precision manufacturing**
  - Extreme ultraviolet (EUV) light to pattern <10nm structures

- **Creating new device technologies**
  - FinFET, Gate All Around transistor, Single Electron Transistor…

university of groningen

Innovation At the Bottom Enables Computing



university of groningen

Combinational Logic Circuits

university of groningen

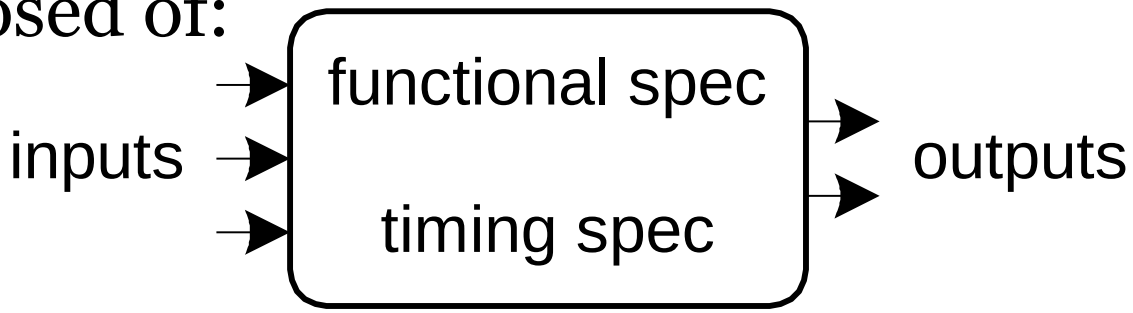
We Can Now Build Logic Circuits

Now, we understand the workings of the basic logic gates

What is our next step?

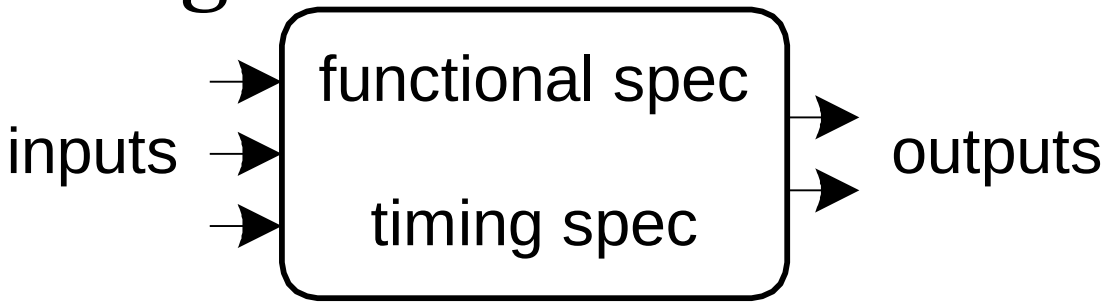
Build some of the logic structures that are important components of the microarchitecture of a computer

- A logic circuit is composed of:
  - Inputs
  - Outputs



- *Functional specification* (describes relationship between inputs and outputs)
- *Timing specification* (describes the delay between inputs changing and outputs responding)

Types of Logic Circuits



- **Combinational Logic**
  - **Memoryless**
  - Outputs are strictly dependent on the combination of input values that are being applied to circuit *right now*

- **Later we will learn: Sequential Logic**
  - **Has memory**
    - Structure stores history → Can ”store” data values
  - Outputs are determined by previous (historical) and current values of inputs

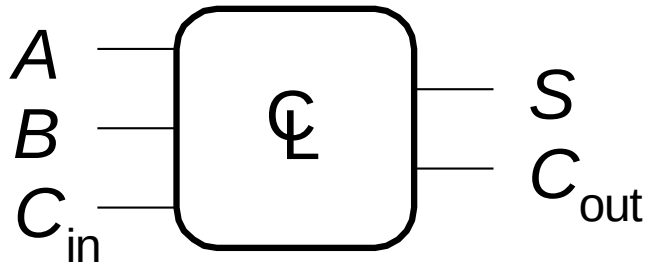
Boolean Logic Equations

Functional Specification

- Functional specification of outputs in terms of inputs
- What do we mean by "function"?
  - Unique mapping from input values to output values
  - The same input values produce the same output value every time
  - No memory (output does not depend on past input values)

- ***Example (full 1-bit adder – more later):***

$$S = \mathrm{F}(A, B, C_{\text{in}})$$
$$C_{\text{out}} = \mathrm{G}(A, B, C_{\text{in}})$$



$$S = A \oplus B \oplus C_{\text{in}}$$ 3-input XOR

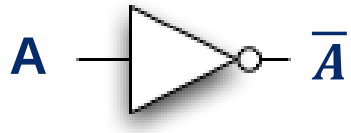
$$C_{\text{out}} = AB + AC_{\text{in}} + BC_{\text{in}}$$

3-input majority

university of groningen

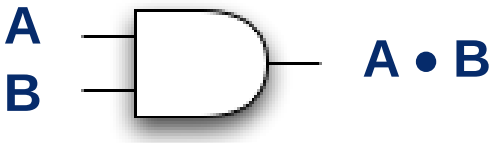
Simple Equations: NOT / AND / OR

$\overline{A}$ *(reads "not A")* is 1 iff A is 0



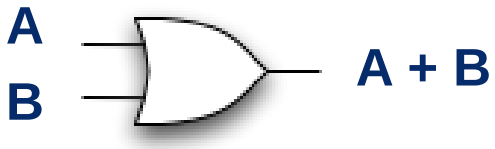
A	$\overline{A}$
0	1
1	0

A • B *(reads "A and B")* is 1 iff A and B are both 1



A	B	$A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

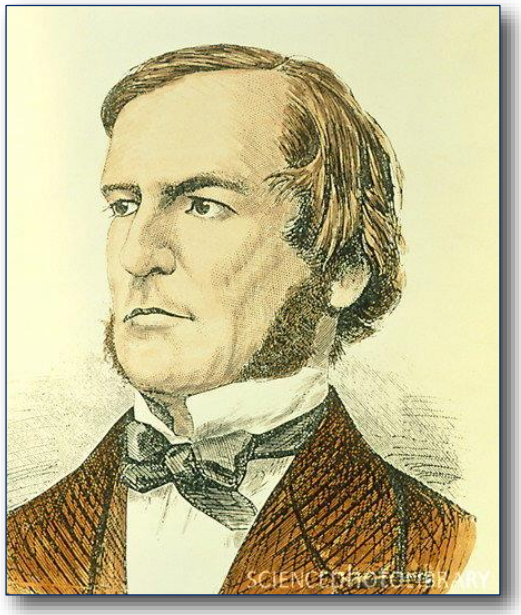
A + B *(reads "A or B")* is 1 iff either A or B is 1



A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1

university of groningen

Boolean Algebra: Big Picture



- An algebra on 1's and 0's
  - with AND, OR, NOT operations
- What you start with
  - Axioms: basic things about objects and operations you just assume to be true at the start
- What you derive first
  - Laws and theorems: allow you to manipulate Boolean expressions
  - …also allow us to do simplification on Boolean expressions
- What you derive later
  - More "sophisticated" properties useful for manipulating digital designs represented in the form of Boolean equations

George Boole, "The Mathematical Analysis of Logic," 1847.

university of groningen

Boolean Algebra: Axioms

Formal version	*English version*
1. *B* contains at least two elements, *0* and *1*, *such that* $0 \neq 1$	Math formality...
2. *Closure* $a,b \in B$, (i) $a + b \in B$ (ii) $a \bullet b \in B$	Result of AND, OR stays in set you start with
3. *Commutative Laws*: $a,b \in B$, (i) (ii)	For primitive AND, OR of 2 inputs, order doesn't matter
4. *Identities*: $0, 1 \in B$ (i) (ii)	There are identity elements for AND, OR, that give you back what you started with
5. *Distributive Laws*: (i) (ii)	• distributes over +, just like algebra + distributes over •, also
6. *Complement*: (i) (ii)	There is a complement element; AND/ORing with it gives the identity elm.

Boolean Algebra: Duality

- Observation
  - All the axioms come in "dual" form
  - Anything true for an expression also true for its dual
  - So any derivation you could make that is true, can be flipped into dual form, and it stays true
- Duality — More formally
  - A dual of a Boolean expression is derived by replacing
    - Every AND operation with… an OR operation
    - Every OR operation with… an AND
    - Every constant 1 with… a constant 0
    - Every constant 0 with… a constant 1
    - But don't change any of the literals or play with the complements!

Example

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

$$\rightarrow \; a + (b \cdot c) = (a + b) \cdot (a + c)$$

university of groningen

Boolean Algebra: Useful Laws

Dual

Operations with 0 and 1:

1. $X + 0 = X$ — 1D. $X \bullet 1 = X$
2. $X + 1 = 1$ — 2D. $X \bullet 0 = 0$

AND, OR with identities gives you back the original variable or the identity

Idempotent Law:

3. $X + X = X$ — 3D. $X \bullet X = X$

AND, OR with self = self

Involution Law:

4. $\overline{(\overline{X})} = X$

double complement = no complement

Laws of Complementarity:

5. $X + \overline{X} = 1$ — 5D. $X \bullet \overline{X} = 0$

AND, OR with complement gives you an identity

Commutative Law:

6. $X + Y = Y + X$ — 6D. $X \bullet Y = Y \bullet X$

Just an axiom…

university of groningen

Useful Laws (continued)

Associative Laws:

7. $(X + Y) + Z = X + (Y + Z)$
$= X + Y + Z$

7D. $(X \bullet Y) \bullet Z = X \bullet (Y \bullet Z)$
$= X \bullet Y \bullet Z$

Parenthesis order does not matter

Distributive Laws:

8. $X \bullet (Y + Z) = (X \bullet Y) + (X \bullet Z)$

8D. $X + (Y \bullet Z) = (X + Y) \bullet (X + Z)$

Axiom

Simplification Theorems:

9.

9D.

10.

10D.

11.

11D.

Useful for simplifying expressions

Actually worth remembering — they show up a lot in real designs…

university of groningen

Boolean Algebra: Proving Things

Proving theorems via axioms of Boolean Algebra:

EX: Prove the theorem: $X \cdot Y + X \cdot \overline{Y} = X$

Distributive (5)

Complement (6)

Identity (4)

EX2: Prove the theorem: $X + X \cdot Y = X$

Identity (4)

Distributive (5)

Identity (2)

Identity (4)

university of groningen

DeMorgan's Law: Enabling Transformations

DeMorgan's Law:

12. $\overline{(X + Y + Z + \cdots)} = \overline{X}.\overline{Y}.\overline{Z}.\ldots$

12D. $\overline{(X.Y.Z.\ldots)} = \overline{X} + \overline{Y} + \overline{Z} + \ldots$

- **Think of this as a transformation**
  - Let's say we have:

$$F = A + B + C$$

  - Applying DeMorgan's Law (12), gives us

$$F = \overline{\overline{(A + B + C)}} = \overline{(\overline{A}.\overline{B}.\overline{C})}$$

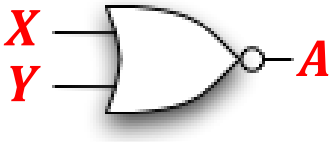
At least one of A, B, C is TRUE --> It is **not** the case that A, B, C are **all** false

university of groningen

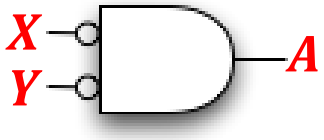
DeMorgan's Law (Continued)

These are conversions between different types of logic functions
They can prove useful if you do not have every type of gate…
Or, if some types of gates are more desirable to use than others…

$$A = \overline{(X + Y)} = \overline{X}\,\overline{Y}$$

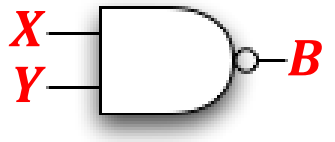


NOR is equivalent to AND with inputs complemented

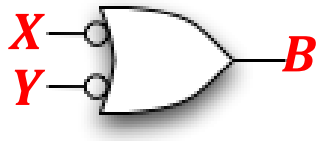


X	Y	$\overline{X+Y}$	$\overline{X}$	$\overline{Y}$	$\overline{X}\,\overline{Y}$
0	0	1	1	1	1
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	0

$$B = \overline{(XY)} = \overline{X} + \overline{Y}$$



NAND is equivalent to OR with inputs complemented



X	Y	$\overline{XY}$	$\overline{X}$	$\overline{Y}$	$\overline{X} + \overline{Y}$
0	0	1	1	1	1
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	0	0	0

Using Boolean Equations to Represent a Logic Circuit

Boolean Equations Enable Us To…

- Represent the function of a combinational logic block
  - Functional Specification

- Methodically transform the function into simpler functions
  - which lead to different hardware realizations
  - Logic Minimization or Logic Simplification
  - We can automate this process → Computer-Aided Design or Electronic Design Automation

- Different Boolean expressions lead to different logic gate implementations

  → Different hardware area, cost, latency, energy properties

Standardized Function Representations

- Enable a single, universally-agreed-on way of representing a Boolean function starting from its truth table
  - Also called “canonical representations”

- Sum of Products (SOP) form

- Product of Sums (POS) form

Sum of Products Form: Key Idea

- Assume **we have the truth table of Boolean Function F**

- How do we express the function in terms of the inputs in a **standard** manner?

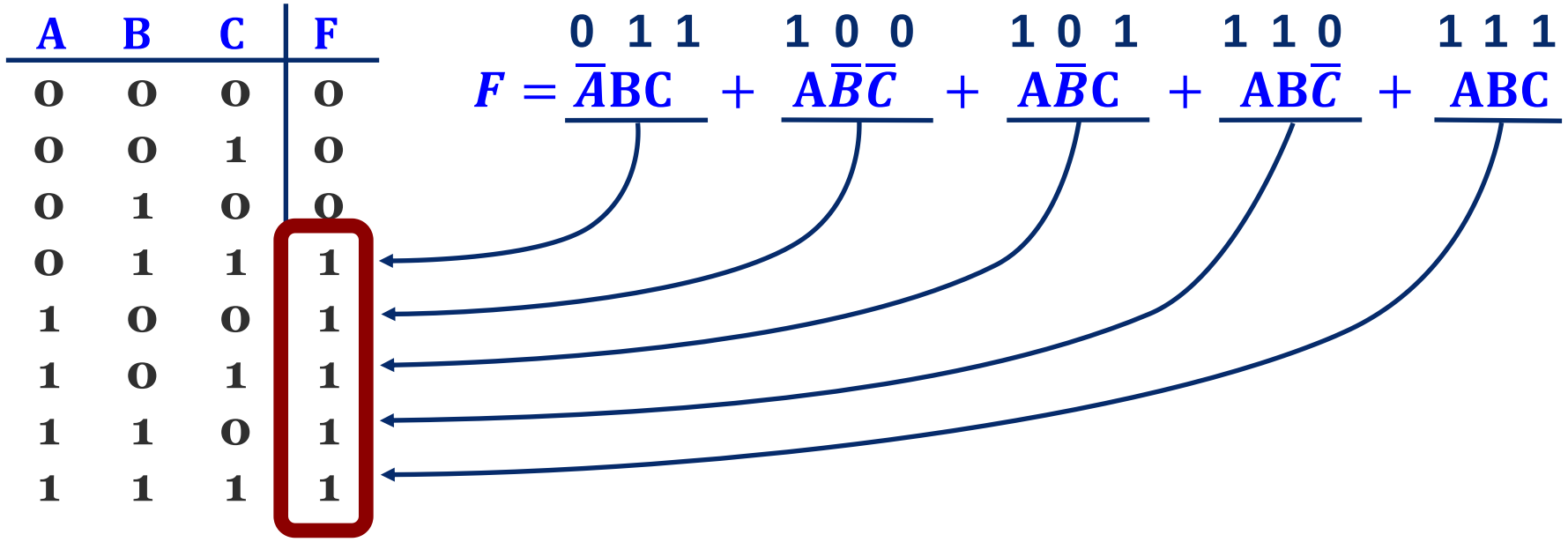
- Idea: **Sum of Products** form
- Express the truth table as a two-level Boolean expression
  - that contains **all** input variable combinations that result in a 1 output
  - If ANY of the combinations of input variables that results in a 1 is TRUE, then the output is 1
  - F = OR of all input variable combinations that result in a 1

university of groningen

Sum of Products Form: Example

Sum of Products Form (SOP)

Also known as **disjunctive normal form** or **minterm expansion**



F = OR of all input variable combinations that result in a 1

All Boolean equations can be written in SOP form

Find all the input combinations (minterms) for which the output of the function is TRUE.

Some Definitions (for a 3-Input Function)

- **Complement:** variable with a bar over it
  $\overline{A}, \overline{B}, \overline{C}$
- **Literal:** variable or its complement
  $A, \overline{A}, B, \overline{B}, C, \overline{C}$
- **Implicant:** product (AND) of literals
  $(A \cdot B \cdot \overline{C}), (\overline{A} \cdot C), (B \cdot \overline{C})$
- **Minterm:** product (AND) that includes **all** input variables
  $(A \cdot B \cdot \overline{C}), (\overline{A} \cdot \overline{B} \cdot C), (\overline{A} \cdot B \cdot \overline{C})$
- **Maxterm:** sum (OR) that includes **all** input variables
  $(A + \overline{B} + \overline{C}), (\overline{A} + B + \overline{C}), (A + B + \overline{C})$

Two-Level Canonical (Standard) Forms

- Truth table is the unique signature of a Boolean *function* …
  - But, it is an expensive representation

- A Boolean function can have many alternative Boolean expressions
  - i.e., many alternative Boolean expressions (and gate realizations) may have the same truth table (and function)
  - If they all specify the same thing, why do we care?
    - Different Boolean expressions lead to different logic gate implementations → Different cost, latency, energy properties

- Canonical form: standard form for a Boolean expression
  - Provides a unique algebraic signature

university of groningen

Two-Level Canonical Forms: SOP

Sum of Products Form (SOP)

Also known as **disjunctive normal form** or **minterm expansion**

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$$F = \underbrace{\bar{A}BC}_{0\,1\,1} + \underbrace{A\bar{B}\bar{C}}_{1\,0\,0} + \underbrace{A\bar{B}C}_{1\,0\,1} + \underbrace{AB\bar{C}}_{1\,1\,0} + \underbrace{ABC}_{1\,1\,1}$$

- Each row in a truth table has a minterm
- A minterm is a product (AND) of literals
- Each minterm is TRUE for that row (and only that row)

All Boolean equations can be written in SOP form

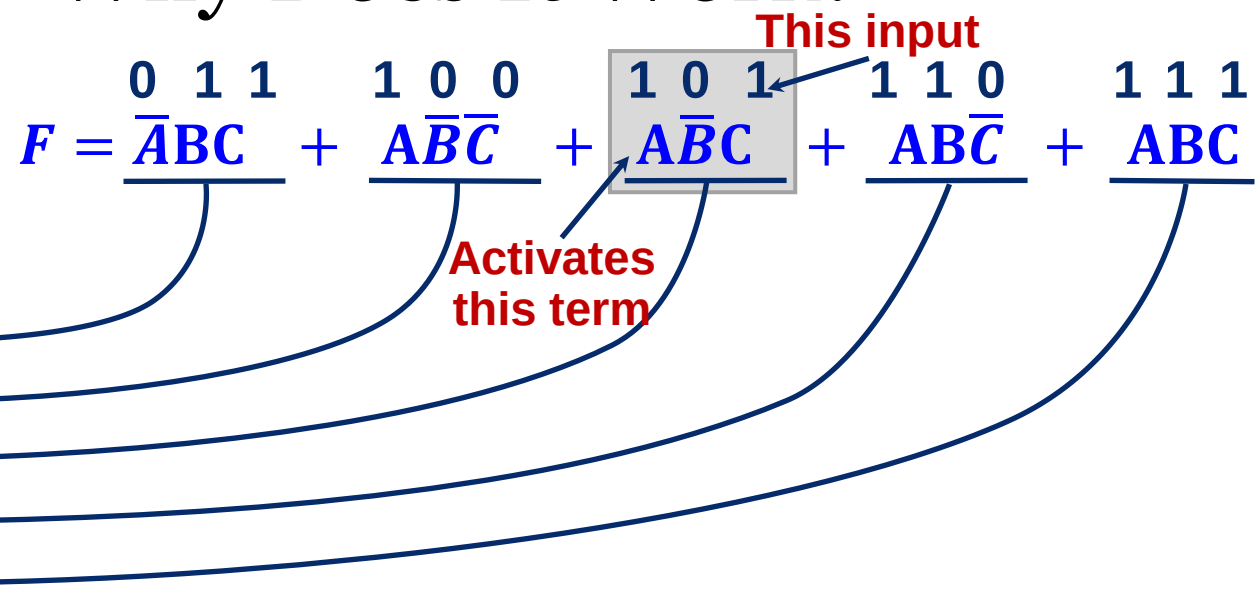
F = OR of all input variable combinations that result in a 1

Find all the input combinations (minterms) for which the output of the function is TRUE.

university of groningen

SOP Form — Why Does It Work?

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1



- Only the shaded product term — $A\bar{B}C = 1 \cdot \bar{0} \cdot 1$ — will be 1
- No other product terms will "turn on" — they will all be 0
- So if inputs A B C correspond to a product term in expression,
  - We get 0 + 0 + … + 1 + … + 0 + 0 = 1 for output
- If inputs A B C do not correspond to any product term in expression
  - We get 0 + 0 + … + 0 = 0 for output

The function evaluates to TRUE (i.e., output is 1)
if **any** of the **Products** (minterms) causes the output to be 1

university of groningen

Standard Notation for SOP Form

- Standard "shorthand" notation
  - If we agree on the order of the variables in the rows of truth table…
    - then we can enumerate each row with the decimal number that corresponds to the binary number created by the input pattern

A	B	C	F	
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	
1	0	0	1	**100 = decimal 4 so this is minterm #4, or m4**
1	0	1	1	
1	1	0	1	
1	1	1	1	**111 = decimal 7 so this is minterm #7, or m7**

f =

We can write this as a sum of products

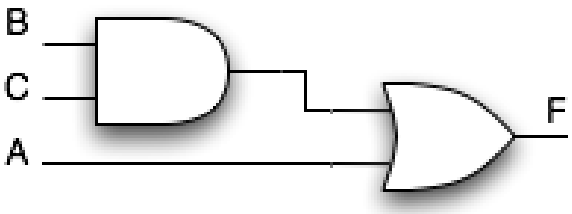
Or, we can use a summation notation

university of groningen

Canonical SOP Form

A	B	C	minterms	
0	0	0	$\bar{A}\bar{B}\bar{C}$	= m0
0	0	1	$\bar{A}\bar{B}C$	= m1
0	1	0	$\bar{A}B\bar{C}$	= m2
0	1	1	$\bar{A}BC$	= m3
1	0	0	$A\bar{B}\bar{C}$	= m4
1	0	1	$A\bar{B}C$	= m5
1	1	0	$AB\bar{C}$	= m6
1	1	1	ABC	= m7

Shorthand Notation for Minterms of 3 Variables



2-Level AND/OR Realization

F in canonical form:

$F(A,B,C) = \sum m(3,4,5,6,7)$

$= m3 + m4 + m5 + m6 + m7$

$F =$

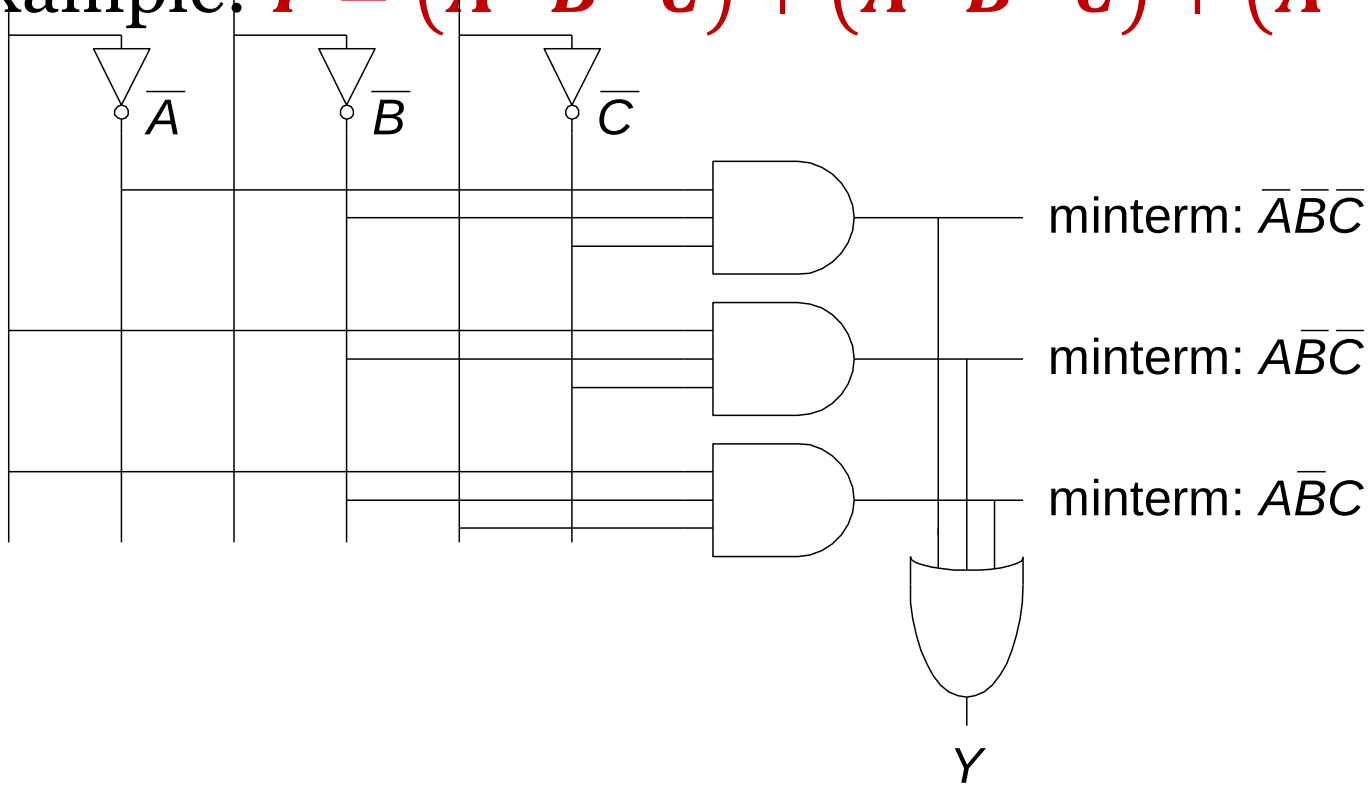
canonical form ≠ minimal form

F

university of groningen

From SOP to Gates

- **SOP (sum-of-products) leads to two-level logic**
- Example: $Y = (\overline{A} \cdot \overline{B} \cdot \overline{C}) + (A \cdot \overline{B} \cdot \overline{C}) + (A \cdot \overline{B} \cdot C)$



SOP form does NOT directly lead to *minimal* logic

university of groningen

Canonical Sum of Products Form: Summary

- Any 1-bit function can be represented as a Sum of Products
- A "Product" is the Boolean AND that includes ALL input variables of the function → minterm
- The 1-bit Output of the Function can be represented as
  - Sum (OR) of all minterms that lead to a 1 in the Output
- Logically
  - The function evaluates to TRUE (i.e., output is 1) if ANY of the Products (minterms) causes the Output to be 1
  - SOP form represents the function as the SUM (OR) of all Products (minterms) that cause the Output to be 1

university of groningen

Alternative Canonical Form: POS

DeMorgan of SOP of $\overline{F}$

Find all the input combinations (maxterms) for which the output of the function is FALSE.

F = AND of all input variable combinations that result in a 0

Product of Sums (POS)

product

$$F = (A + B + C)(A + B + \overline{C})(A + \overline{B} + C)$$

sums

This input

Each sum term represents one of the "**zeros**" of the function

$$F = \underset{0\ 0\ 0}{(A + B + C)}\ \underset{0\ 0\ 1}{(A + B + \overline{C})}\ \underset{0\ 1\ 0}{(A + \overline{B} + C)}$$

Activates this term

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

For the given input, only the shaded sum term will equal 0

$$A + \overline{B} + C = 0 + \overline{1} + 0$$

Anything ANDed with 0 is 0; Output F will be 0

The function evaluates to FALSE (i.e., output is 0)
if **any** of the Sums (maxterms) causes the output to be 0

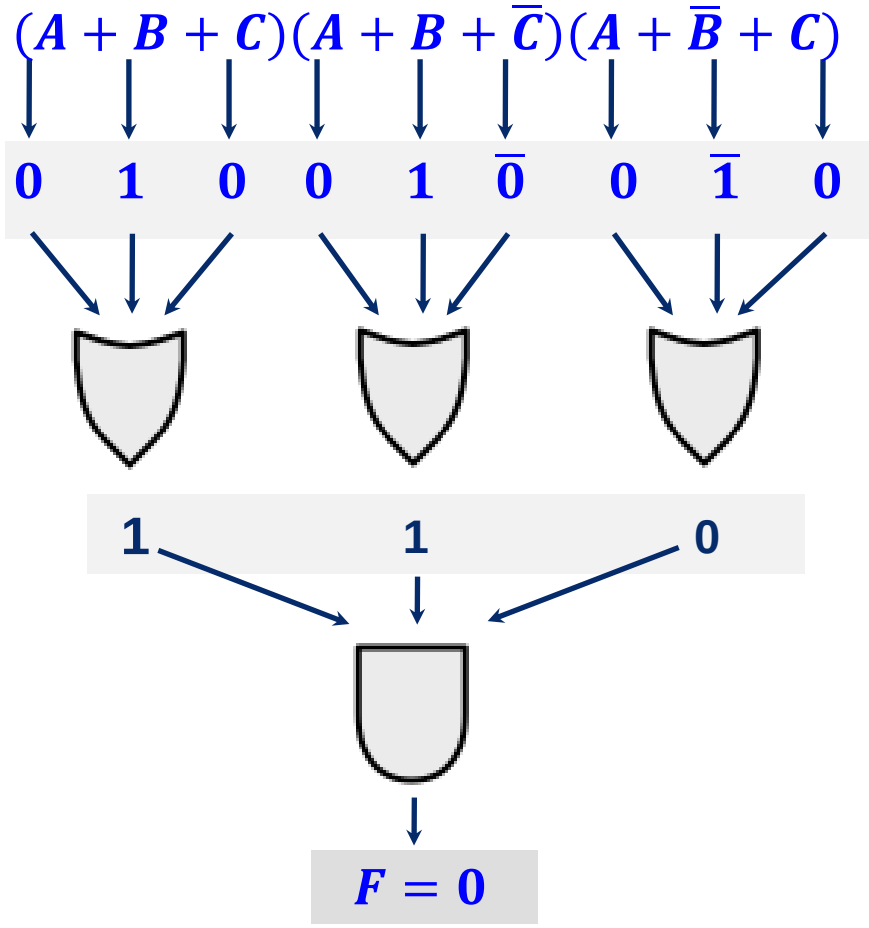
university of groningen

Consider A=0, B=1, C=0

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Input

0 1 0



Only one of the products will be 0, anything ANDed with 0 is 0

Therefore, the output is F = 0

university of groningen

POS: How to Write It

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$$F = (A + B + C)(A + B + \overline{C})(A + \overline{B} + C)$$

$A \quad \overline{B} \quad C$

$A + \overline{B} + C$

Maxterm form:

1. Find truth table rows where F is 0

2. 0 in input col → true literal
3. 1 in input col → complemented literal

4. OR the literals to get a Maxterm
5. AND together all the Maxterms

Or just remember" POS of F is the same as the DeMorgan of SOP of $\overline{F}$

university of groningen

Notation for the Canonical POS Form

Product of Sums / Conjunctive Normal Form / Maxterm Expansion

$$F = (A + B + C)(A + B + \overline{C})(A + \overline{B} + C)$$

$$\prod M(0, 1, 2)$$

A	B	C	Maxterms	
0	0	0	$A + B + C$	= M0
0	0	1	$A + B + \overline{C}$	= M1
0	1	0	$A + \overline{B} + C$	= M2
0	1	1	$A + \overline{B} + \overline{C}$	= M3
1	0	0	$\overline{A} + B + C$	= M4
1	0	1	$\overline{A} + B + \overline{C}$	= M5
1	1	0	$\overline{A} + \overline{B} + C$	= M6
1	1	1	$\overline{A} + \overline{B} + \overline{C}$	= M7

Maxterm shorthand notation for a function of three variables

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Note that you form the maxterms around the "zeros" of the function

This is not the complement of the function!

university of groningen

Useful Conversions

1. **Minterm to Maxterm conversion:**
   rewrite minterm shorthand using maxterm shorthand
   replace minterm indices with the indices not already used
   E.g., $\mathrm{F}(A,B,C) = \sum m(3,4,5,6,7) = \prod M(0,1,2)$

2. **Maxterm to Minterm conversion:**
   rewrite maxterm shorthand using minterm shorthand
   replace maxterm indices with the indices not already used
   E.g., $\mathrm{F}(A,B,C) = \prod M(0,1,2) = \sum m(3,4,5,6,7)$

3. **Expansion of F to expansion of $\overline{F}$:**

$$\text{E. g.}, \mathrm{F}(A,B,C) = \sum m(3,4,5,6,7) \longrightarrow \overline{F}(A,B,C) = \sum m(0,1,2)$$
$$= \prod M(0,1,2) \longrightarrow = \prod M(3,4,5,6,7)$$

4. **Minterm expansion of F to Maxterm expansion of $\overline{F}$:**
   rewrite in Maxterm form, using the same indices as F

$$\text{E. g.}, \mathrm{F}(A,B,C) = \sum m(3,4,5,6,7) \longrightarrow \overline{F}(A,B,C) = \prod M(3,4,5,6,7)$$
$$= \prod M(0,1,2) \longrightarrow = \sum m(0,1,2)$$

university of groningen

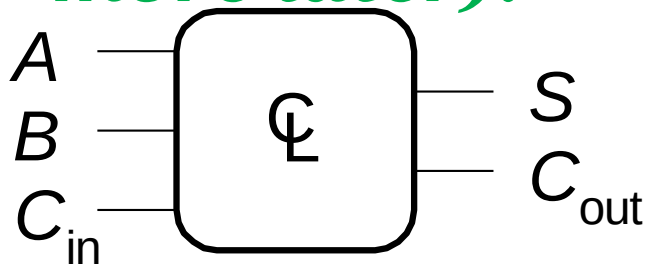
Logic Simplification (or Minimization)

- Using Boolean Algebra, we can simplify the SOP or POS form of any function in a methodical way

- Starting with the canonical SOP or POS form enables convenience and automation
  - Truth table → SOP/POS form → Boolean Simplification Rules

- ***Example (full 1-bit adder – more later):***

$$S = \mathrm{F}(A, B, C_{\text{in}})$$
$$C_{\text{out}} = \mathrm{G}(A, B, C_{\text{in}})$$

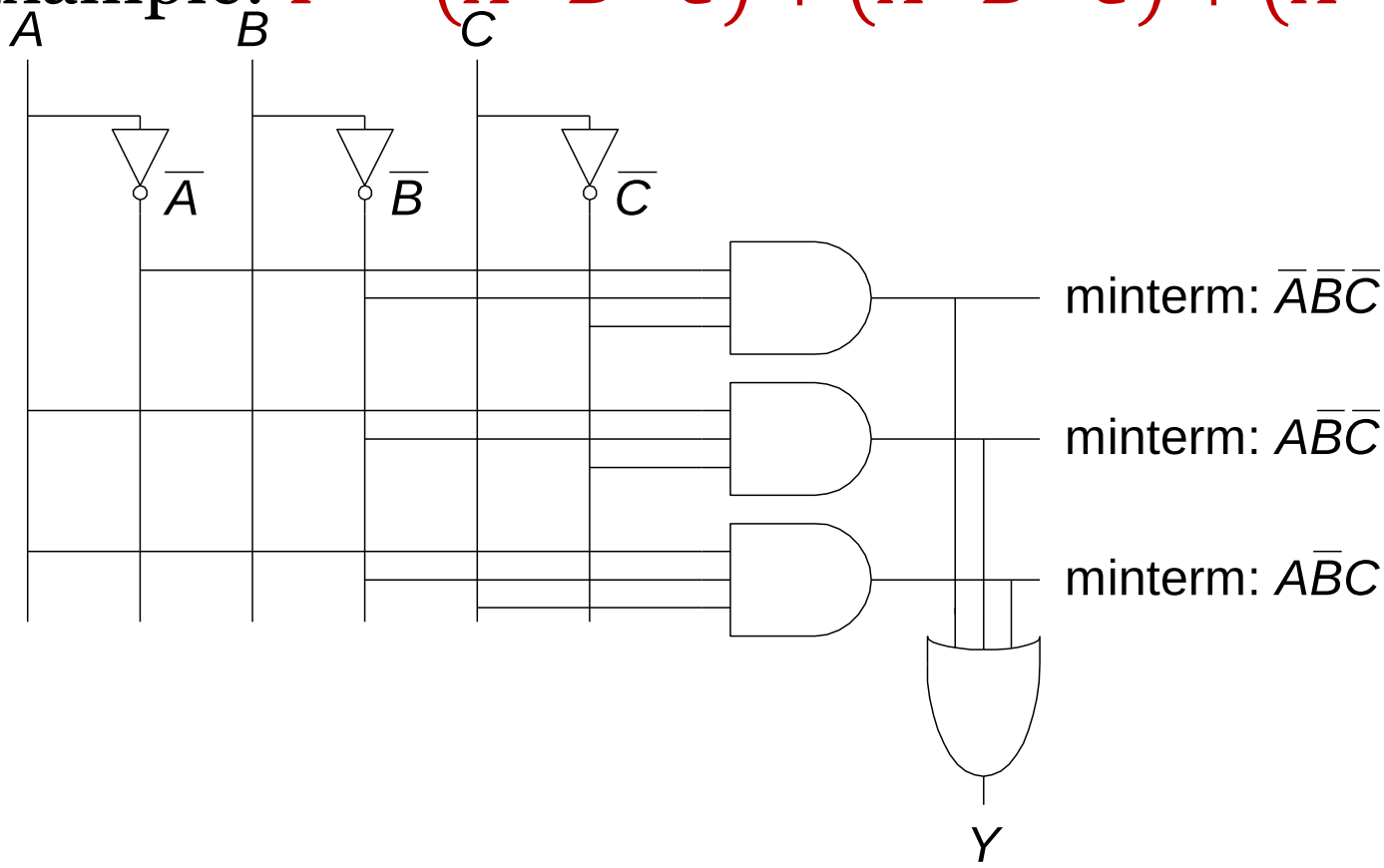


$$S = A \oplus B \oplus C_{\text{in}}$$ 3-input XOR
$$C_{\text{out}} = AB + AC_{\text{in}} + BC_{\text{in}}$$
3-input majority

university of groningen

Logic Simplification Example: SOP Form

- **SOP (sum-of-products) form of function Y**
- Example: $Y = (\overline{A} \cdot \overline{B} \cdot \overline{C}) + (A \cdot \overline{B} \cdot \overline{C}) + (A \cdot \overline{B} \cdot C)$

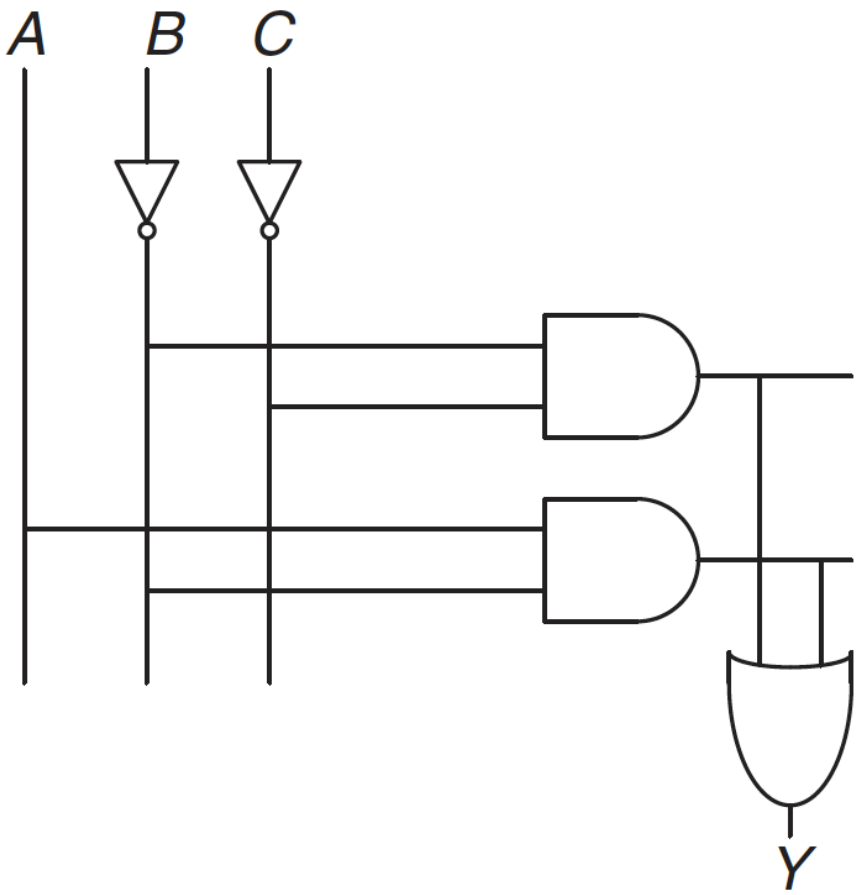


SOP form does NOT directly lead to *minimal* logic

university of groningen

Logic Simplification Example: Simplified

- **Simplified form of function Y**
- Example



Let's Cover Some Basic Combinational Blocks

university of groningen

Combinational Building Blocks used in Modern Computers

university of groningen

Recall: Common Logic Gates

Buffer



A	Z
0	0
1	1

AND



A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR



A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

XOR



A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

Inverter



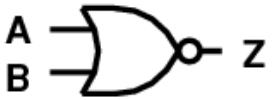
A	Z
0	1
1	0

NAND



A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

NOR



A	B	Z
0	0	1
0	1	0
1	0	0
1	1	0

XNOR



A	B	Z
0	0	1
0	1	0
1	0	0
1	1	1

Combinational Building Blocks

- Combinational logic is often grouped into larger building blocks to build more complex systems

- Hides the unnecessary gate-level details to emphasize the function of the building block

- We now examine:
  - Decoder
  - Multiplexer
  - Full adder
  - PLA (Programmable Logic Array)

We Will Cover Many Building Blocks

- Basic logic gates (AND, OR, NOT, NAND, NOR, XOR)
- Decoder
- Multiplexer
- Full Adder
- Programmable Logic Array (PLA)
- Comparator
- Arithmetic Logic Unit (ALU)
- Tri-State Buffer

- Standard form representations: SOP & POS
- Logic simplification via Boolean Algebra
- Logical completeness

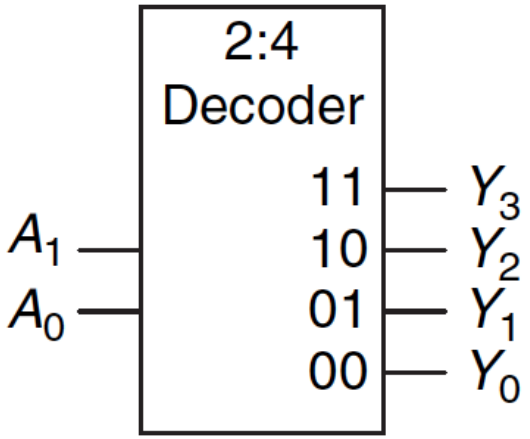
Decoder

university of groningen

Decoder

- "Input pattern detector"
- n inputs and 2^n outputs
- Exactly one of the outputs is 1 and all the rest are 0s
- The output that is logically 1 is the output corresponding to the input pattern that the logic circuit is expected to detect
- Example: 2-to-4 decoder

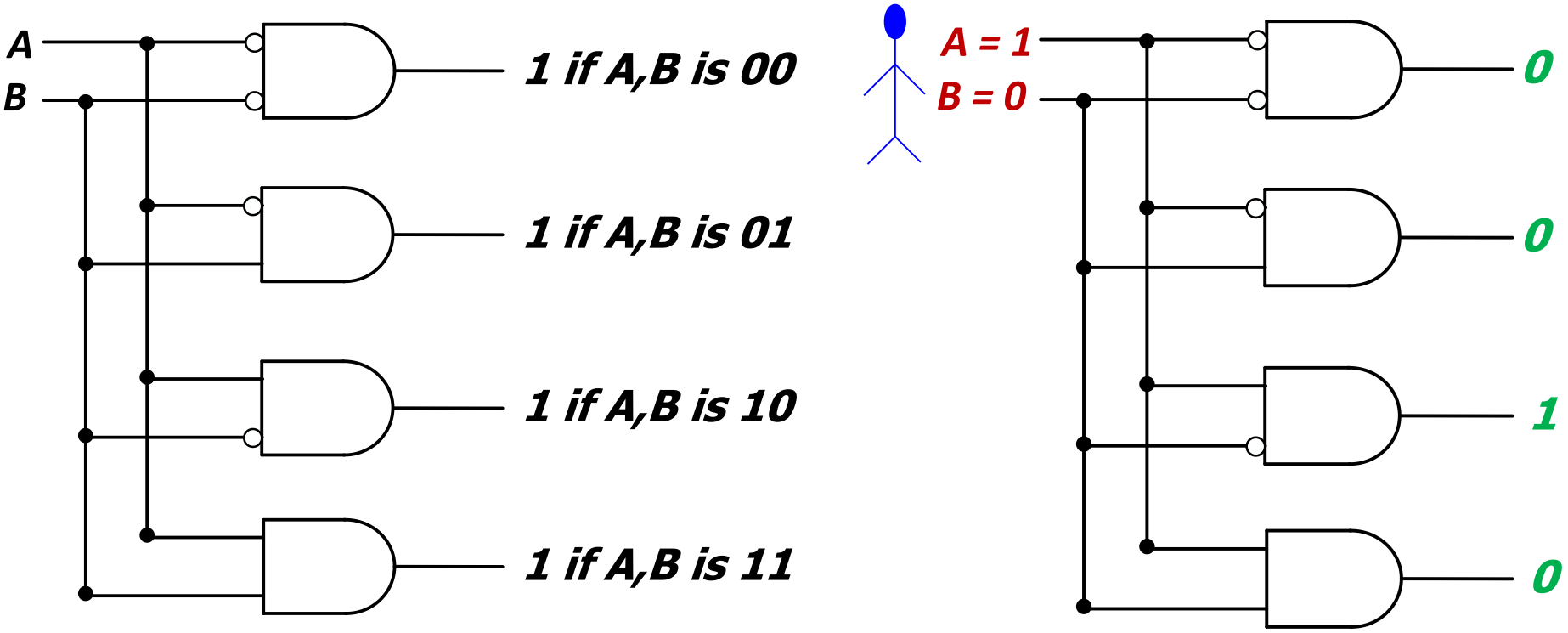
A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



university of groningen

Decoder (I)

- n inputs and 2^n outputs
- Exactly one of the outputs is 1 and all the rest are 0s
- The output that is logically 1 is the output corresponding to the input pattern that the logic circuit is expected to detect

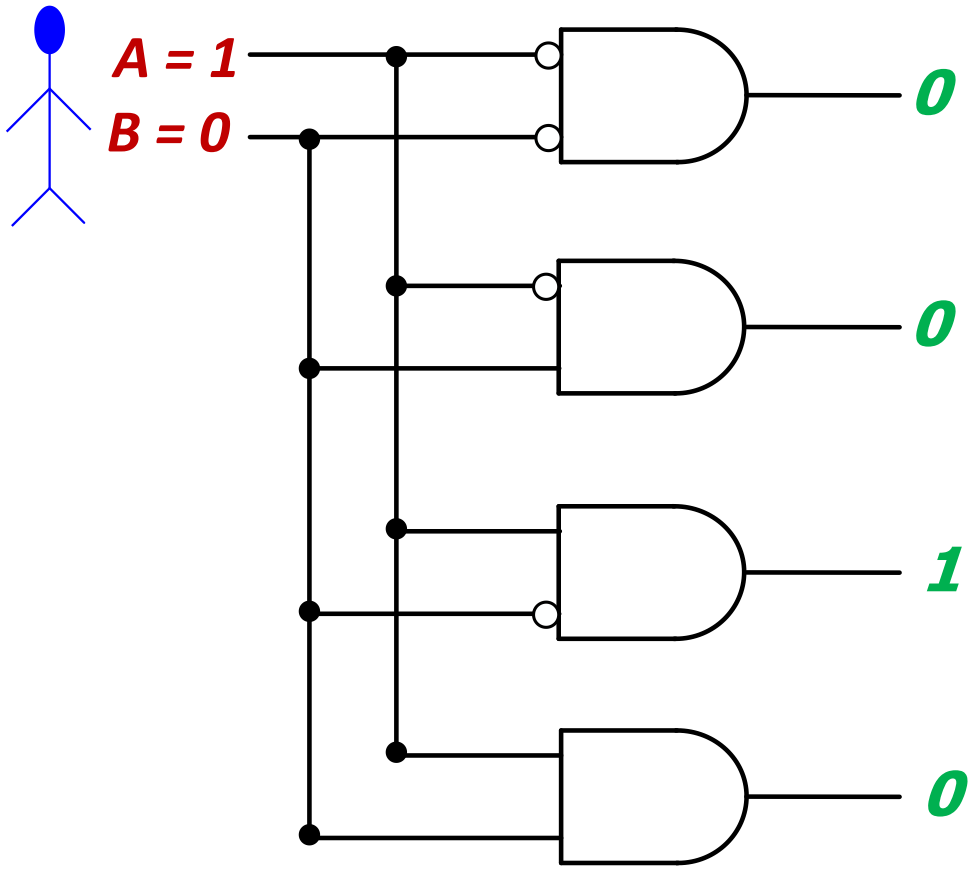


Decoder (II)

› The decoder is useful in determining how to interpret a bit pattern

- **It could be the address of a location in memory, that the processor intends to read from**

- **It could be an instruction in the program and the processor needs to decide what action to take (based on *instruction opcode*)**



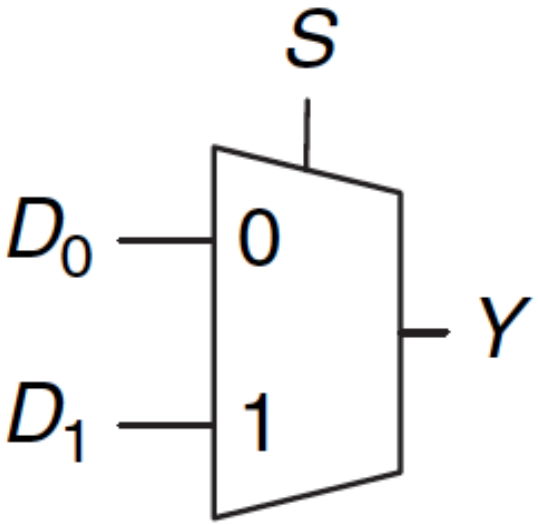
university of groningen

Multiplexer (MUX)

Multiplexer (MUX), or Selector

- Selects one of the N inputs to connect it to the output
  - based on the value of a $\log_2 N$-bit control input called select
- Example: 2-to-1 MUX

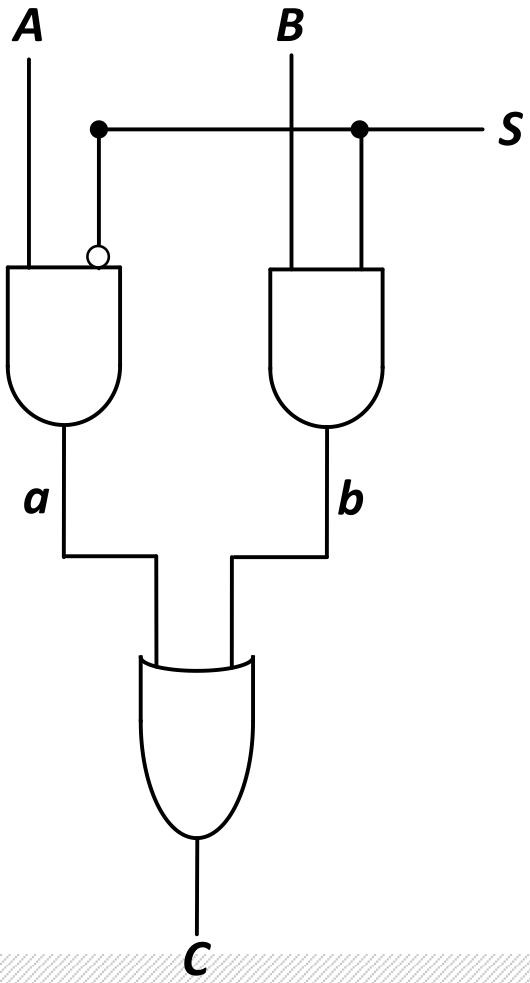
S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

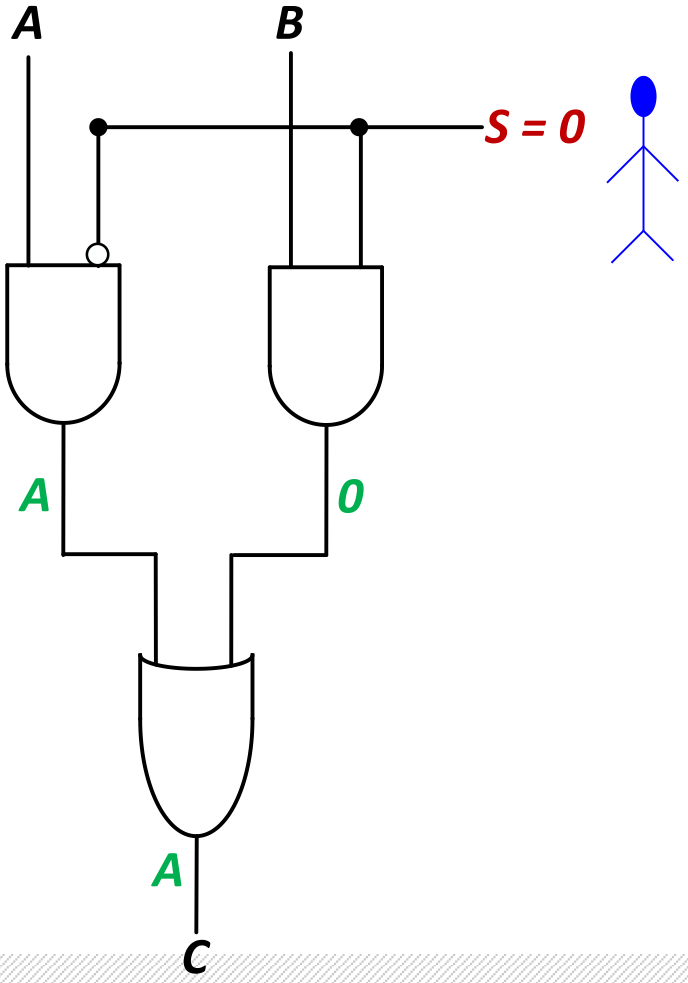


university of groningen

Multiplexer (MUX), or Selector (II)

- Selects one of the N inputs to connect it to the output
  - based on the value of a $\log_2 N$-bit control input called select
- Example: 2-to-1 MUX

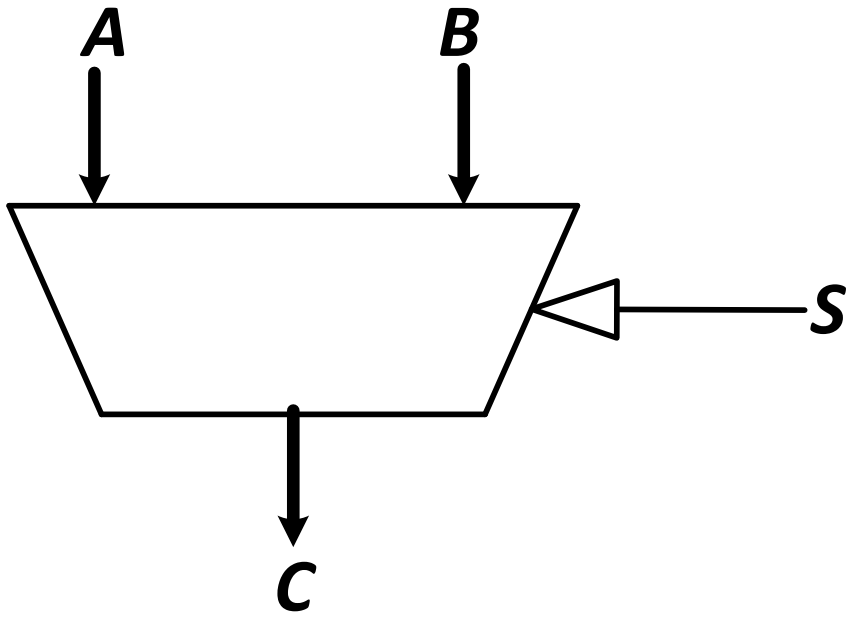




Multiplexer (MUX), or Selector (III)

- The output C is always connected to either the input A or the input B
  - Output value depends on the value of the select line S

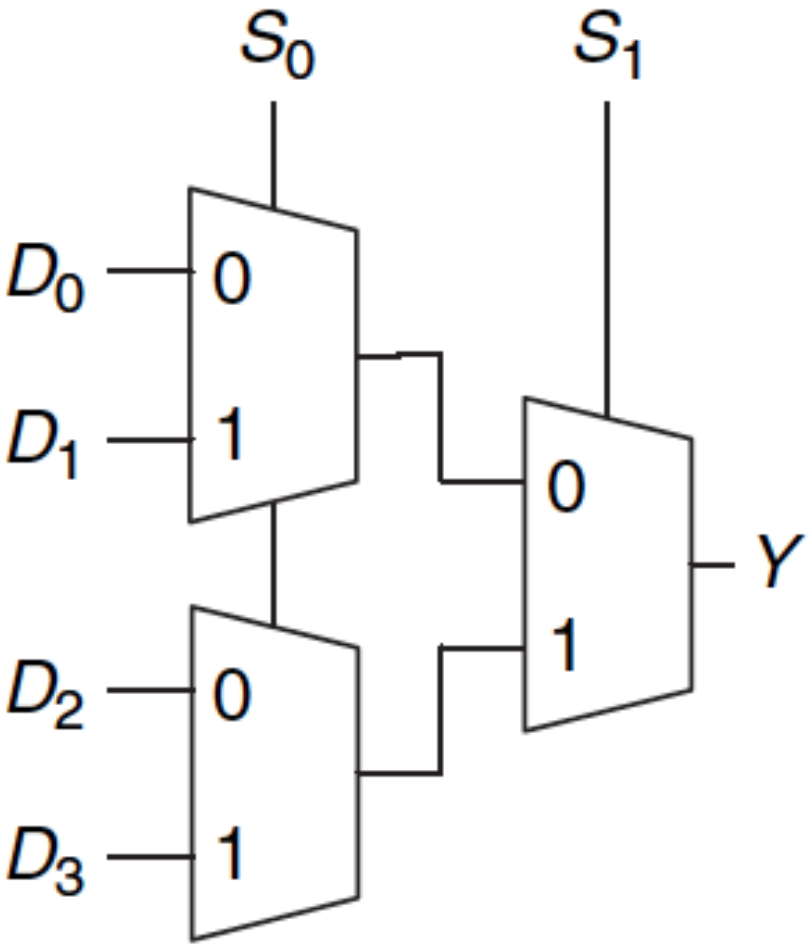
S	C
0	A
1	B

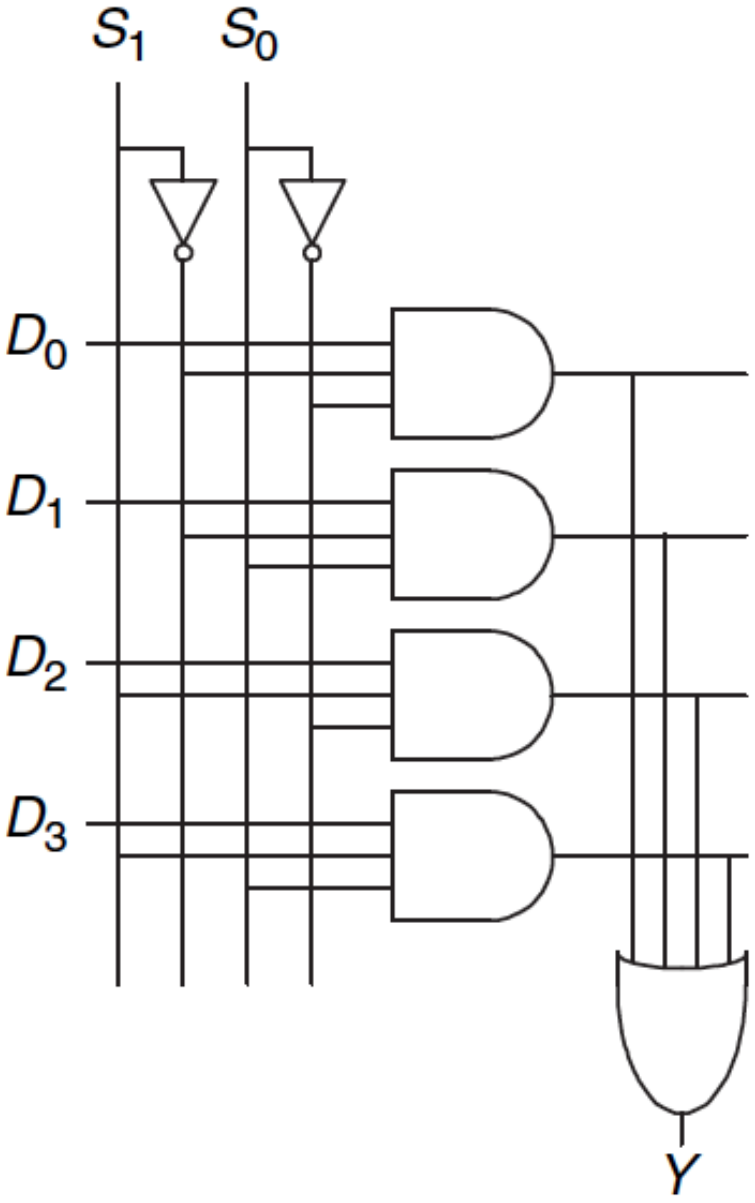


- Your task: Draw the schematic for an 4-input (4:1) MUX
  - Gate level: as a combination of basic AND, OR, NOT gates
  - Module level: As a combination of 2-input (2:1) MUXes

university of groningen

A 4-to-1 Multiplexer





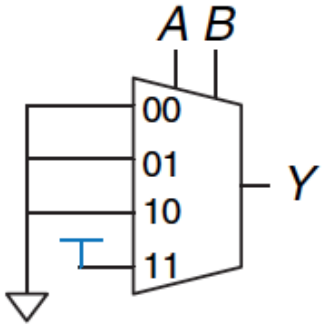
university of groningen

Aside: Logic Using Multiplexers

› Multiplexers can be used as “lookup tables” to perform logic functions

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

$Y = AB$



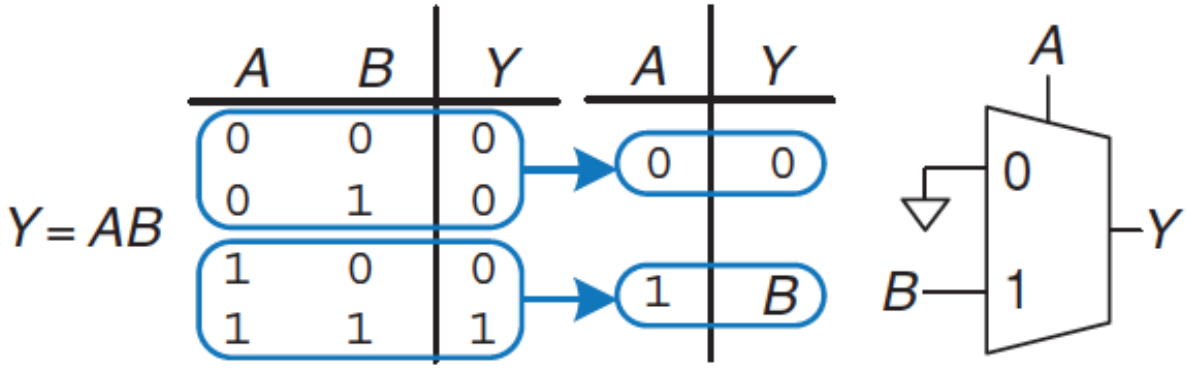


Figure 2.59 4:1 multiplexer implementation of two-input AND function

Idea: Formulate the truth table as a multiplexer

university of groningen

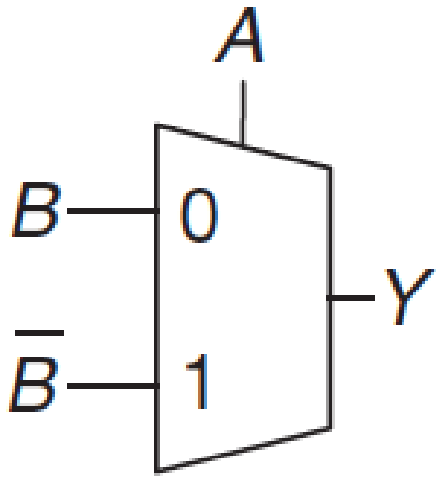
Aside: Logic Using Multiplexers (II)

› Multiplexers can be used as “lookup tables” to perform logic functions

$Y = A \oplus B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

A	Y
0	B
1	$\overline{B}$

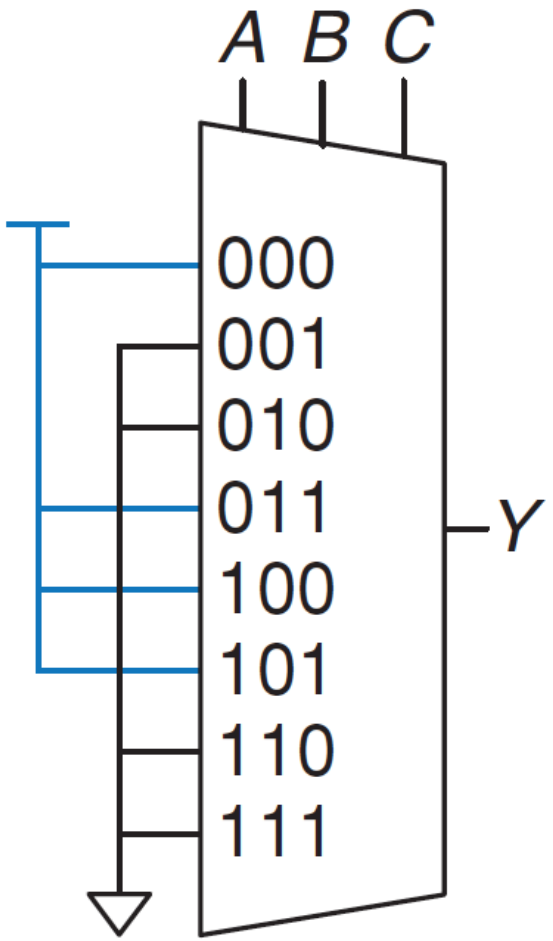


Aside: Logic Using Multiplexers (III)

› Multiplexers can be used as lookup tables to perform logic functions

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$Y = A\overline{B} + \overline{B}\,\overline{C} + \overline{A}BC$$

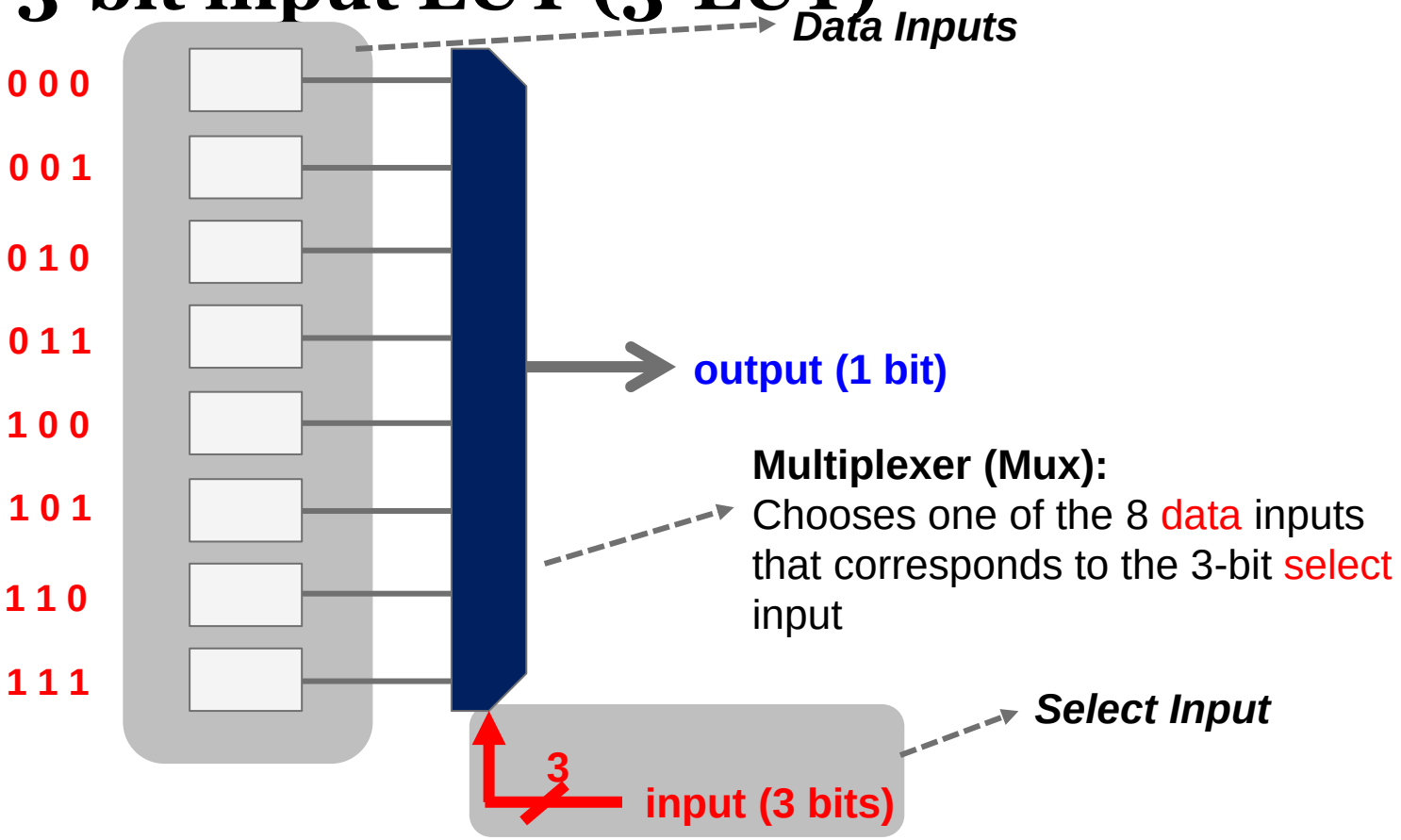


Read H&H Chapter 2.8

university of groningen

8-Input Lookup Table (LUT)

- **3-bit input LUT (3-LUT)**



3-LUT can implement
any 3-bit input function

university of groningen

An Example of Programming a LUT

› A function that outputs '1' when there are **at least two '1's in a 3-bit input**

In C:

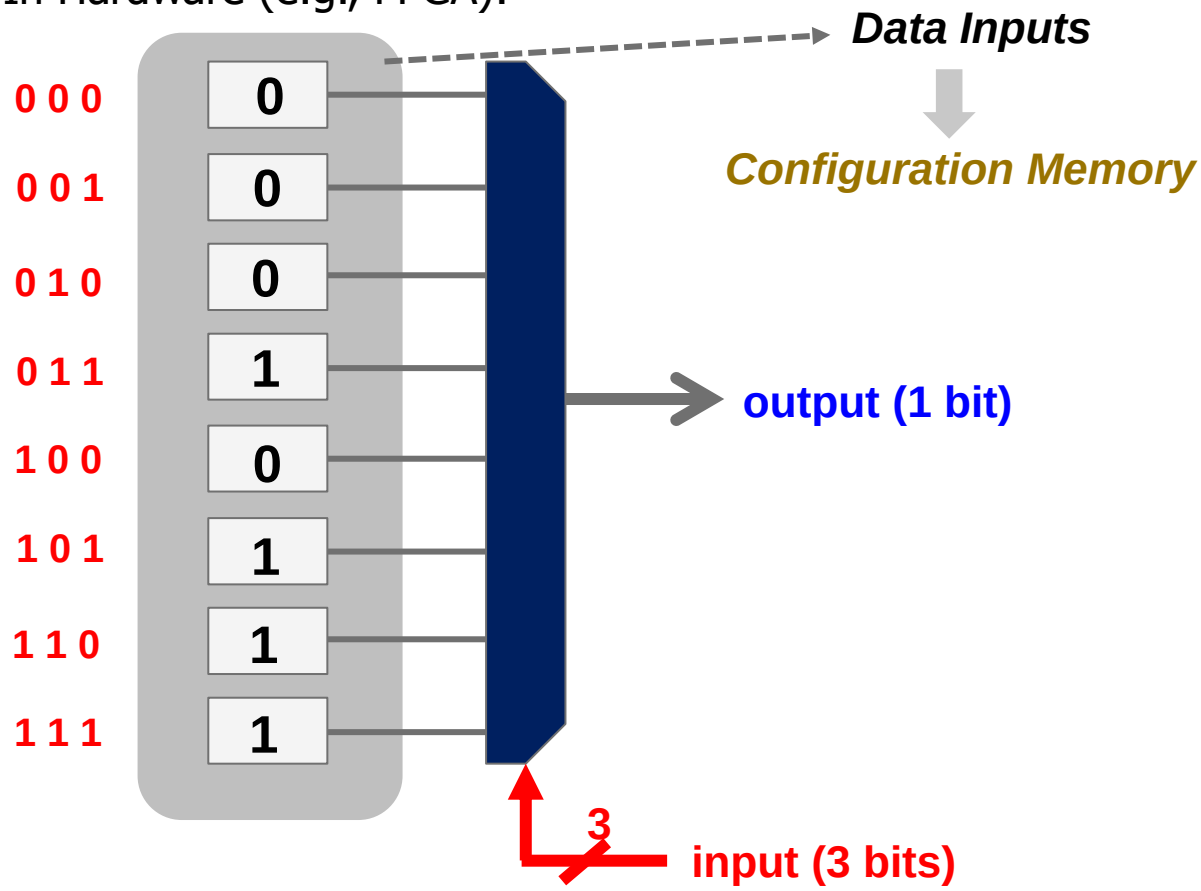
```
int count = 0;
for(int i = 0; i < 3; i++) {
    count += input & 1;
    input = input >> 1;
}

if(count > 1) return 1;

return 0;
```

```
switch(input){
   case 0:
   case 1:
   case 2:
   case 4:
      return 0;
   default:
      return 1;}
```

In Hardware (e.g., FPGA):



university of groningen

Aside: Logic Using Decoders (I)

› Decoders can be combined with OR gates to build logic functions.

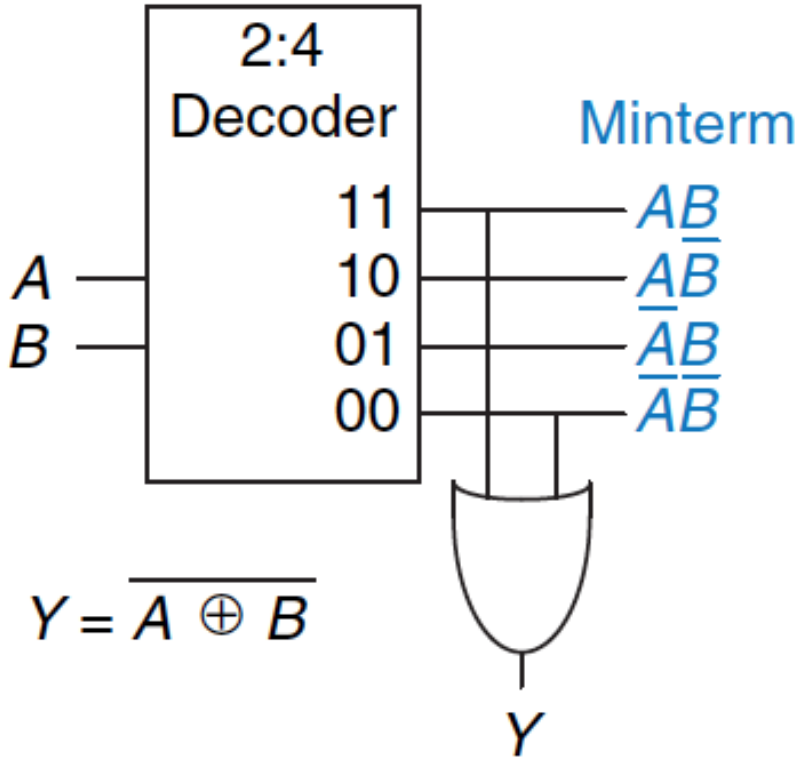


Figure 2.65 Logic function using decoder

Read H&H Chapter 2.8

Full Adder

university of groningen

Full Adder (I)

› **Binary addition**

- Similar to decimal addition
- From right to left
- One column at a time
- One sum and one carry bit

$$\begin{array}{r} a_{n-1}a_{n-2}\ldots a_1a_0 \\ b_{n-1}b_{n-2}\ldots b_1b_0 \\ C_n\,C_{n-1}\quad\ldots\quad C_1 \\ \hline S_{n-1}\quad\ldots\quad S_1S_0 \end{array}$$

› Truth table of binary addition on one column of bits within two n-bit operands

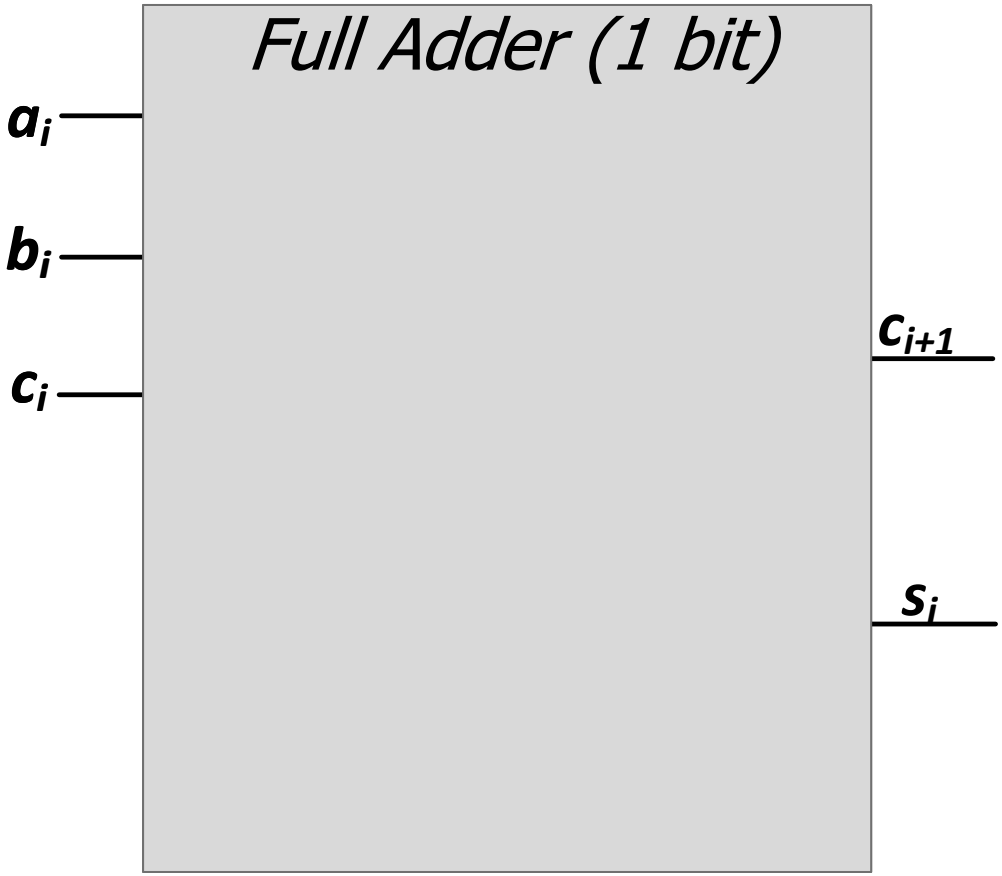
a_i	b_i	$carry_i$	$carry_{i+1}$	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

university of groningen

Full Adder (II)

› Binary addition

- N 1-bit additions
- **SOP of 1-bit addition**



$$\begin{array}{r} a_{n-1}a_{n-2}\ldots a_1a_0 \\ b_{n-1}b_{n-2}\ldots b_1b_0 \\ \hline C_n\,C_{n-1}\quad\ldots\quad C_1 \\ \hline S_{n-1}\quad\ldots\quad S_1S_0 \end{array}$$

a_i	b_i	$carry_i$	$carry_{i+1}$	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

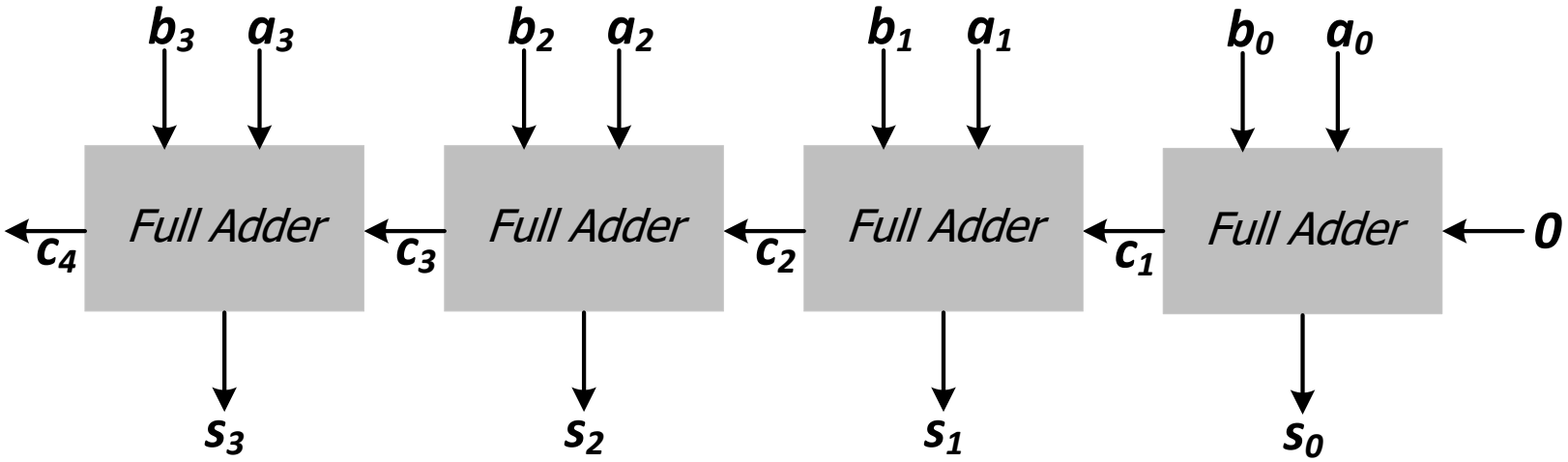
MAJ XOR

university of groningen

4-Bit Adder from Full Adders

› Creating a **4-bit adder** out of 1-bit full adders

- To add two 4-bit binary numbers A and B



$$\begin{array}{cccccc} & a_3 & a_2 & a_1 & a_0 \\ + & b_3 & b_2 & b_1 & b_0 \\ \hline c_4 & c_3 & c_2 & c_1 & \\ \hline & s_3 & s_2 & s_1 & s_0 \end{array} \qquad \begin{array}{cccccc} & 1 & 0 & 1 & 1 \\ + & 1 & 0 & 0 & 1 \\ \hline 1 & 0 & 1 & 1 & \\ \hline & 0 & 1 & 0 & 0 \end{array}$$

university of groningen

Adder Design: Ripple Carry Adder

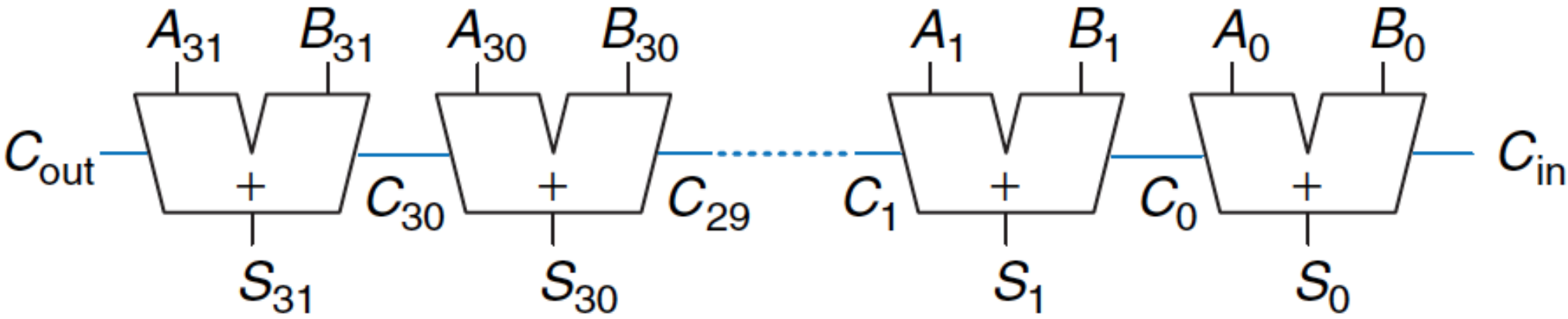
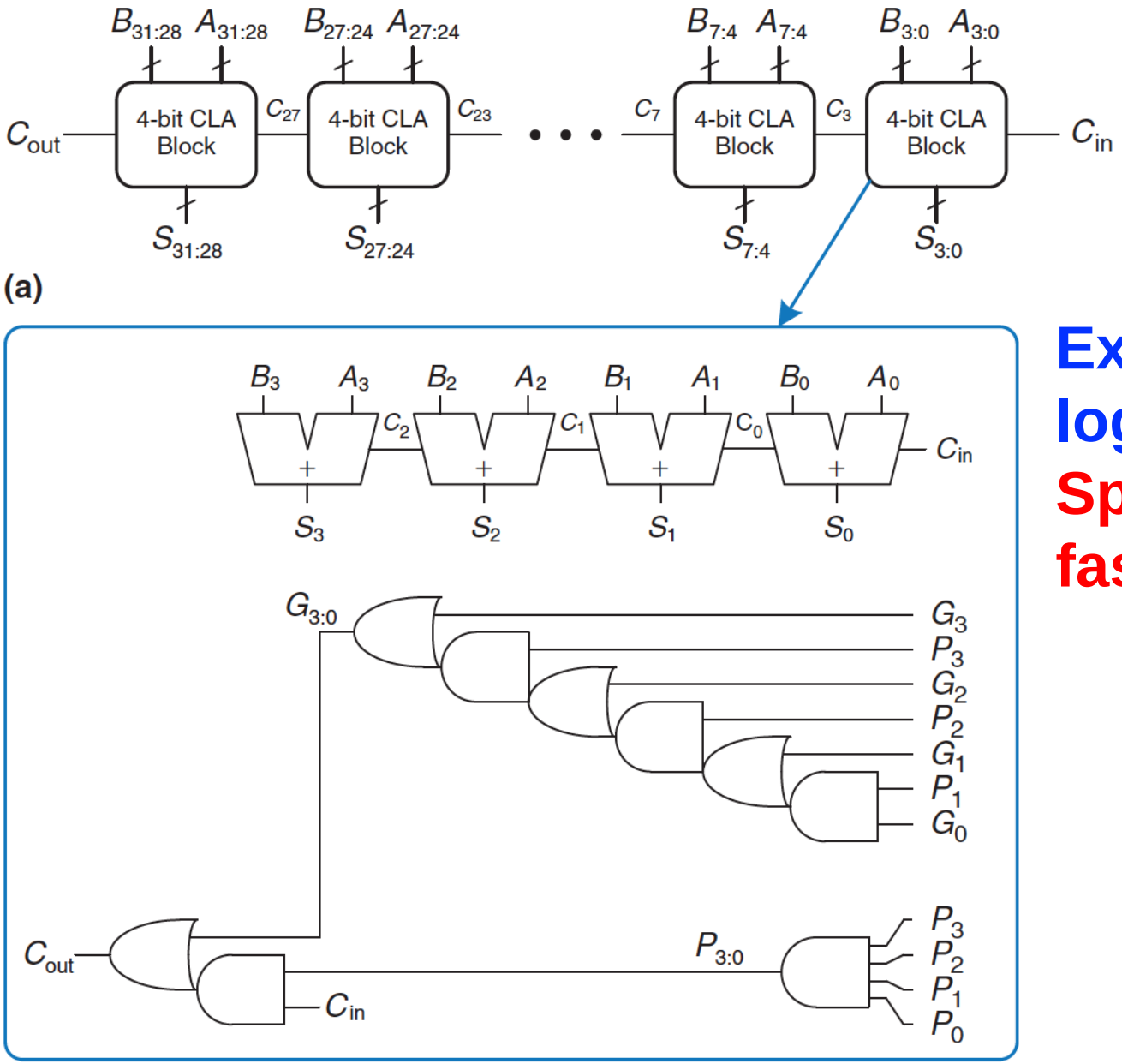


Figure 5.5 32-bit ripple-carry adder

H&H Section 5.2.1

university of groningen

Adder Design: Carry Lookahead Adder



Example of logic specialization: Specialized logic for fast carry calculation

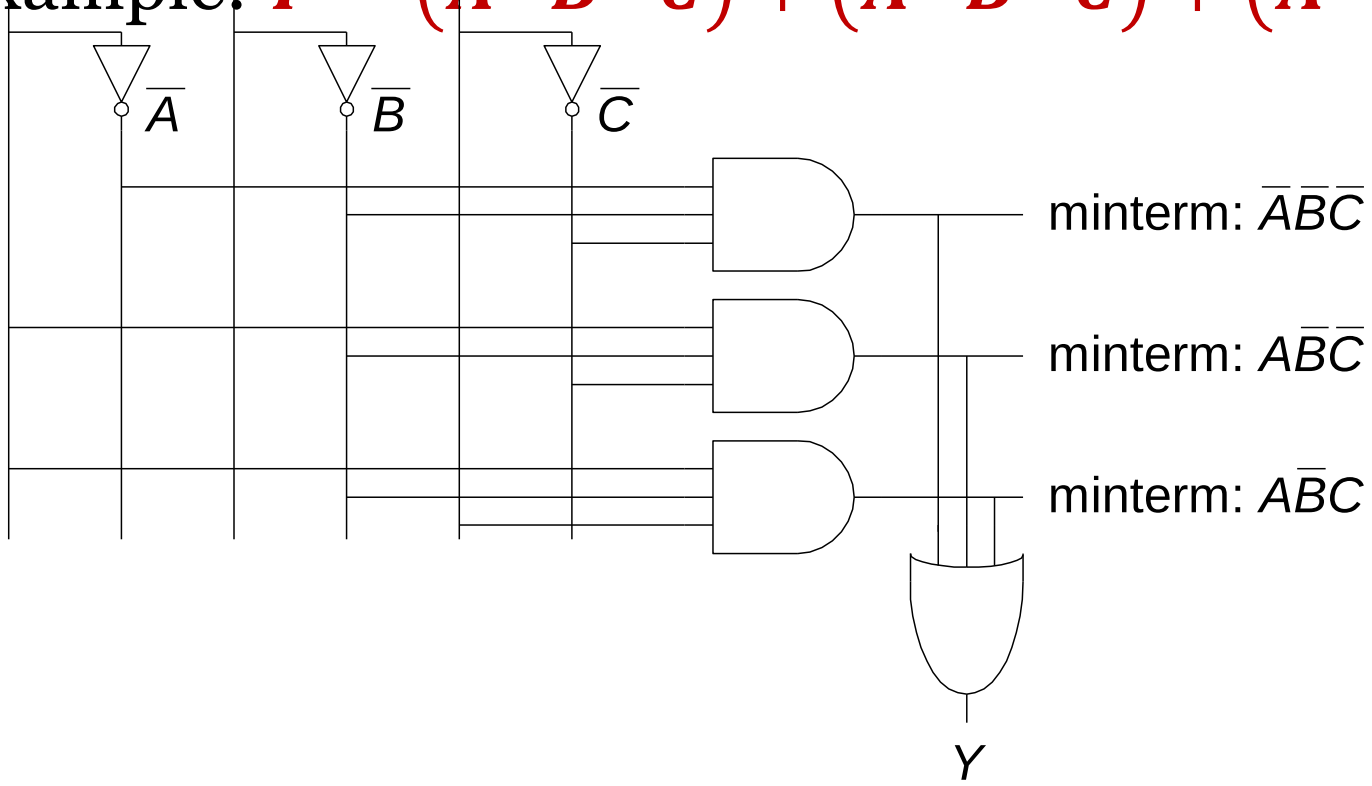
H&H Section 5.2.1

Programmable Logic Array (PLA)

university of groningen

PLA: Recall: SOP Form

- **SOP (sum-of-products) leads to two-level logic**
- Example: $Y = (\overline{A} \cdot \overline{B} \cdot \overline{C}) + (A \cdot \overline{B} \cdot \overline{C}) + (A \cdot \overline{B} \cdot C)$

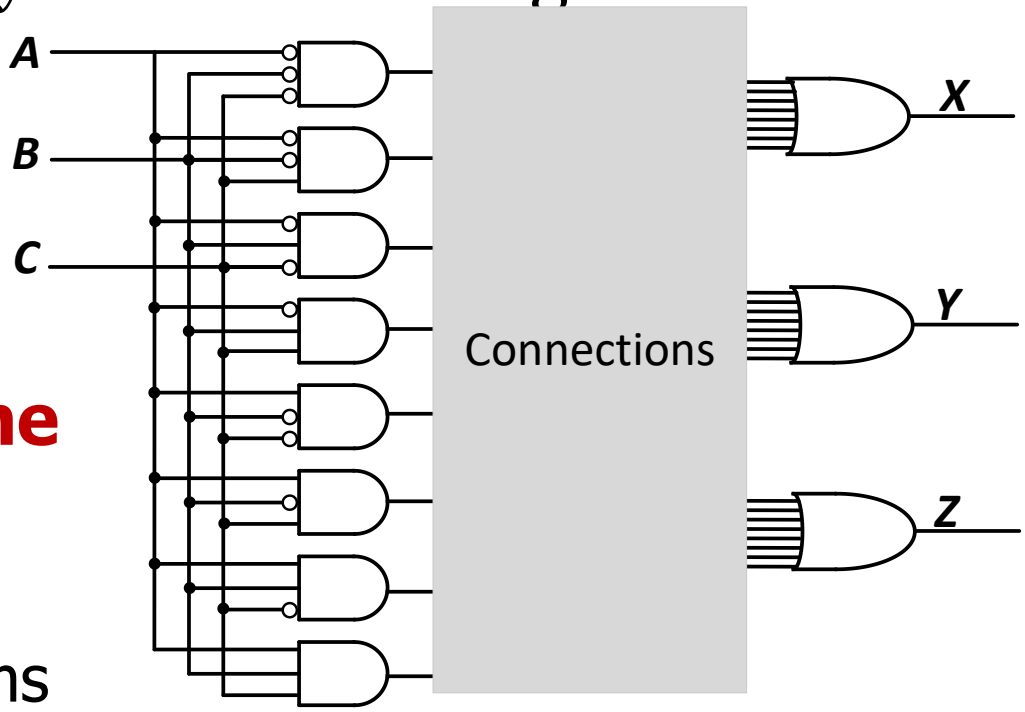


A PLA enables the two-level SOP implementation of **any** N-input M-output function

university of groningen

The Programmable Logic Array (PLA)

› The below logic structure is a very common building block for implementing any collection of logic functions one wishes to



- An array of AND gates followed by an array of OR gates
- **How do we determine the number of AND gates?**
  - **Remember SOP:** the number of possible minterms
  - For an n-input logic function, we need a PLA with 2^n n-input AND gates
- **How do we determine the number of OR gates?** The number of output columns in the truth table

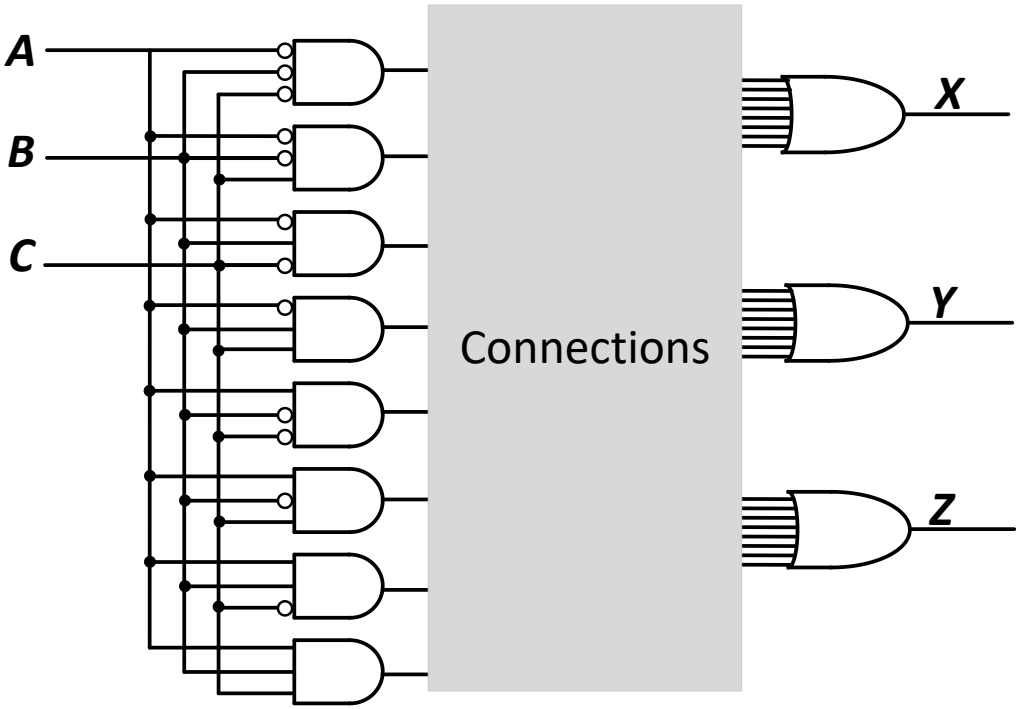
A PLA enables the two-level SOP implementation of **any** N-input M-output function

university of groningen

The Programmable Logic Array (PLA)

› How do we implement a logic function?

- Connect the output of an AND gate to the input of an OR gate if the corresponding minterm is included in the SOP
  - This is a simple programmable logic construct

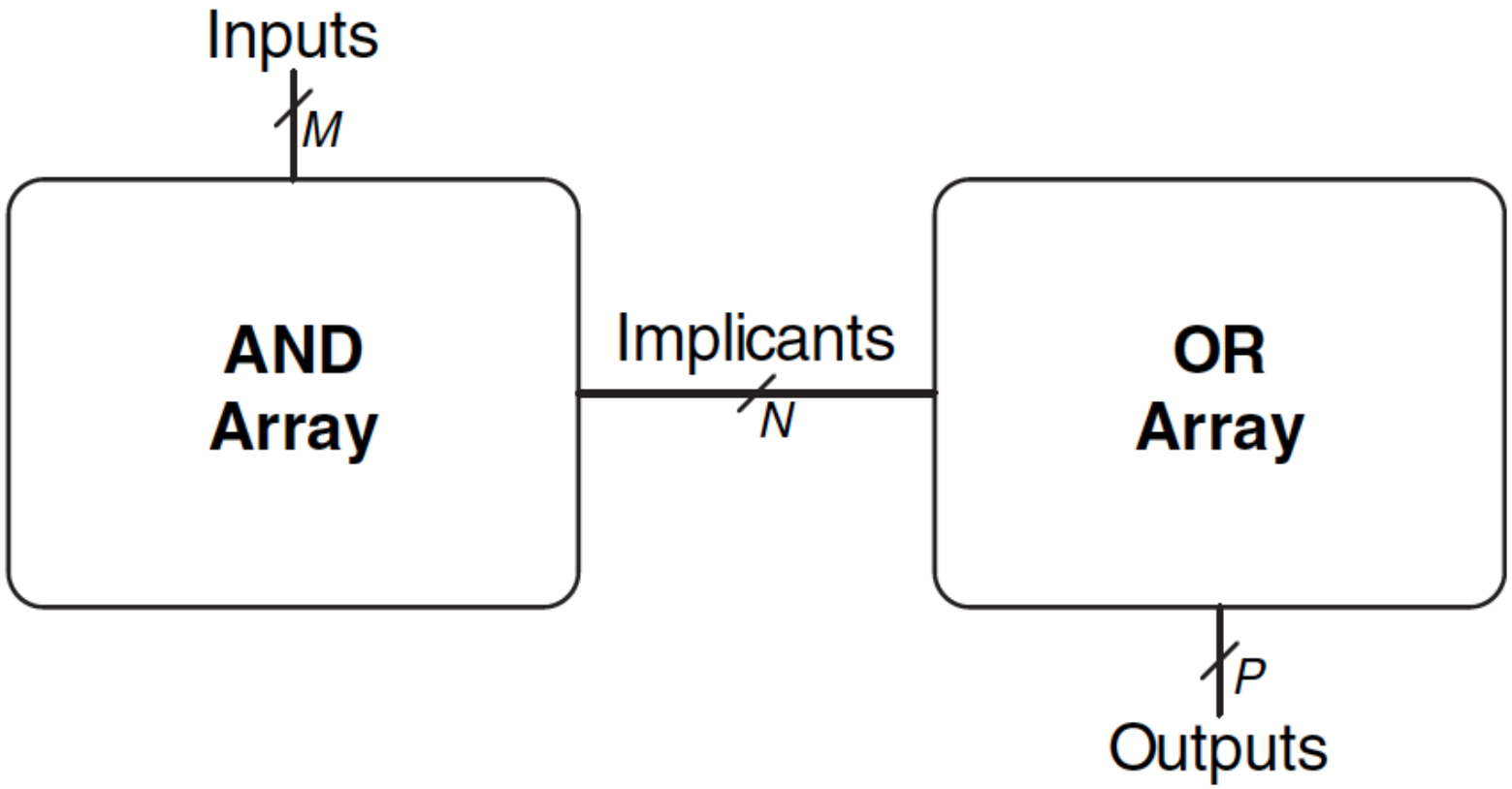


- **Programming a PLA**: we program the connections from AND gate outputs to OR gate inputs to implement a desired logic function
- Is there any other type of programmable logic?
  - Yes! An FPGA…
  - An FPGA uses more advanced structures, as we see in the labs

A PLA enables the two-level SOP implementation of **any** N-input M-output function

university of groningen

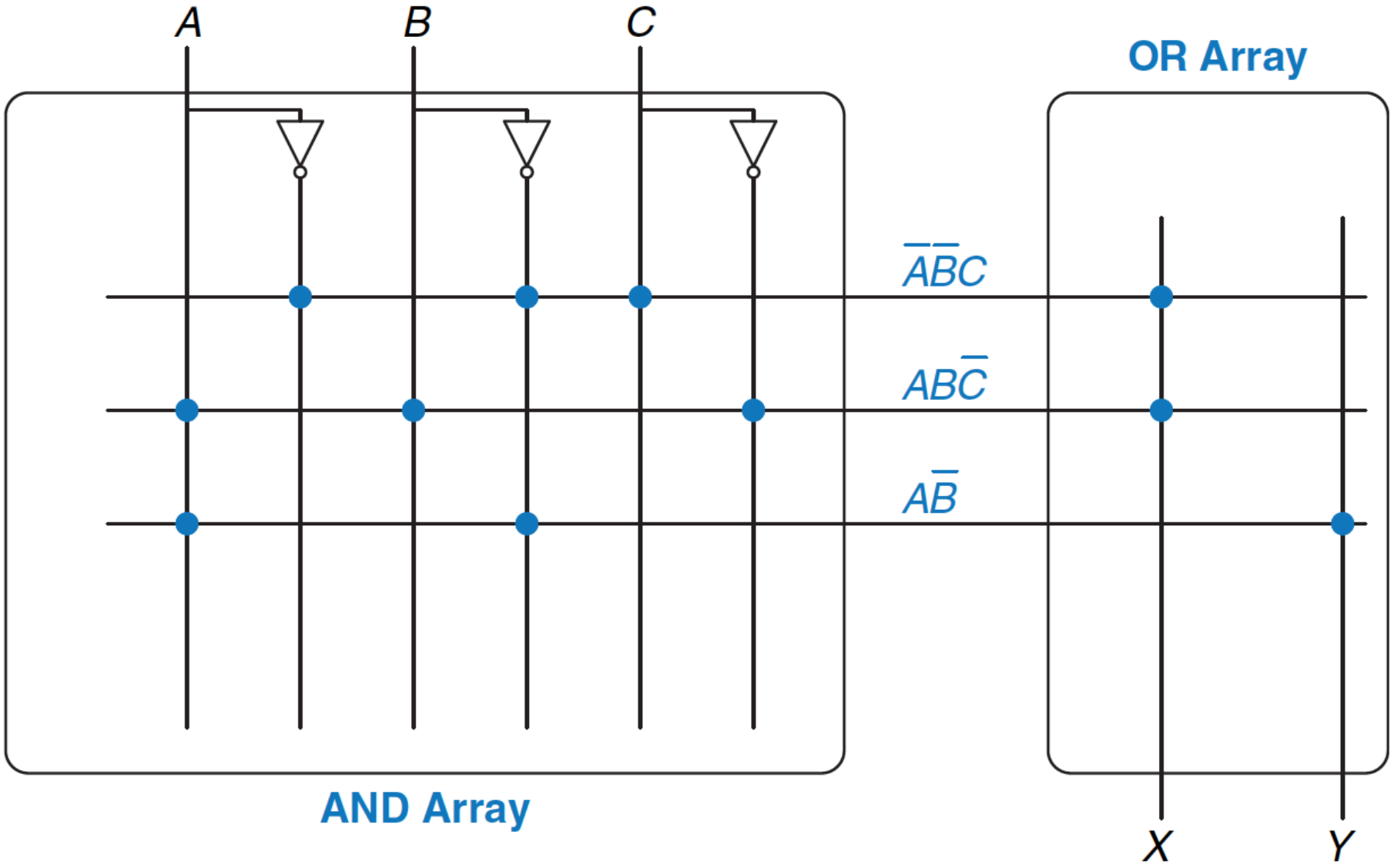
PLA Example (I)



Read H&H Chapter 5.6.1

university of groningen

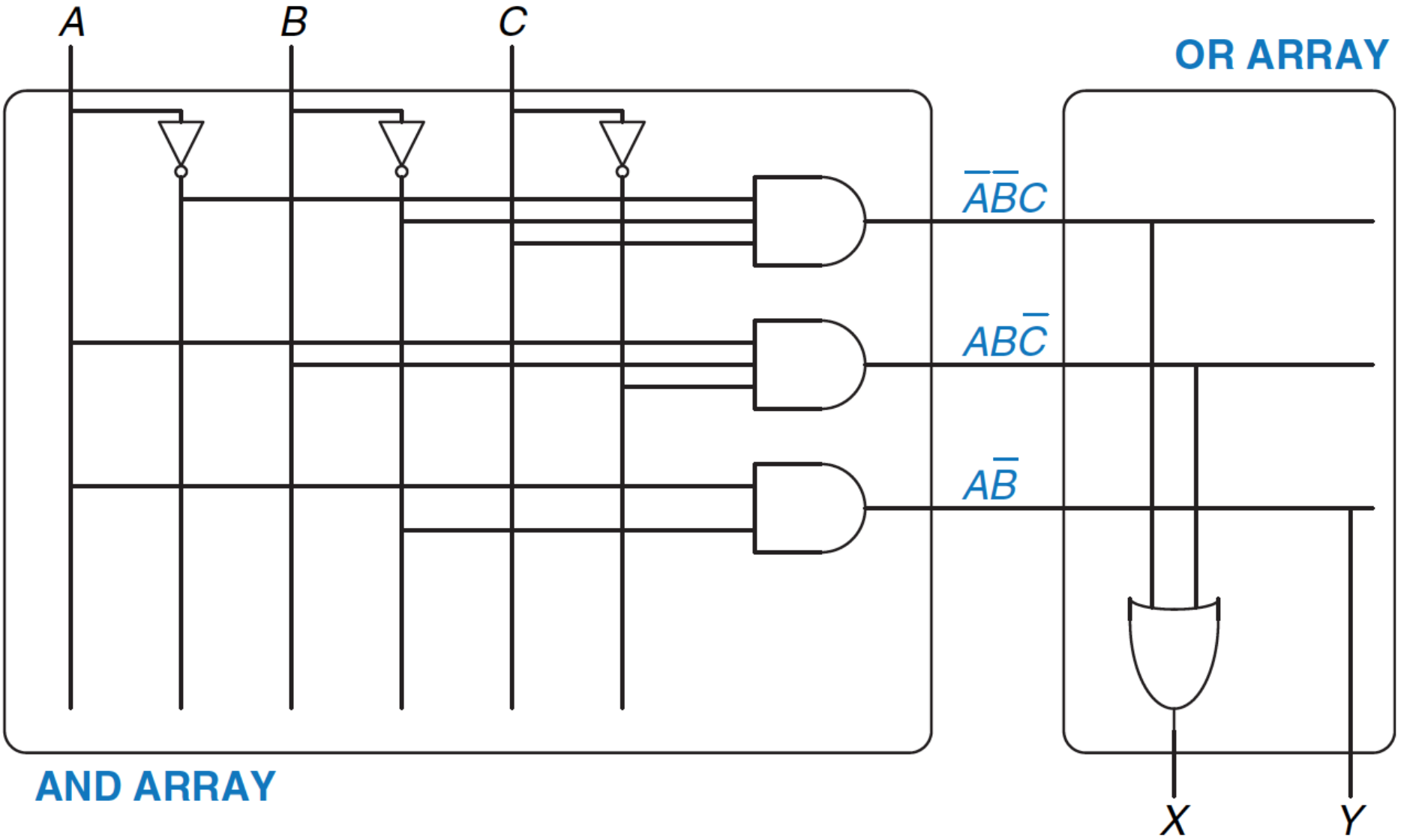
PLA Example Function (II)



Read H&H Chapter 5.6.1

university of groningen

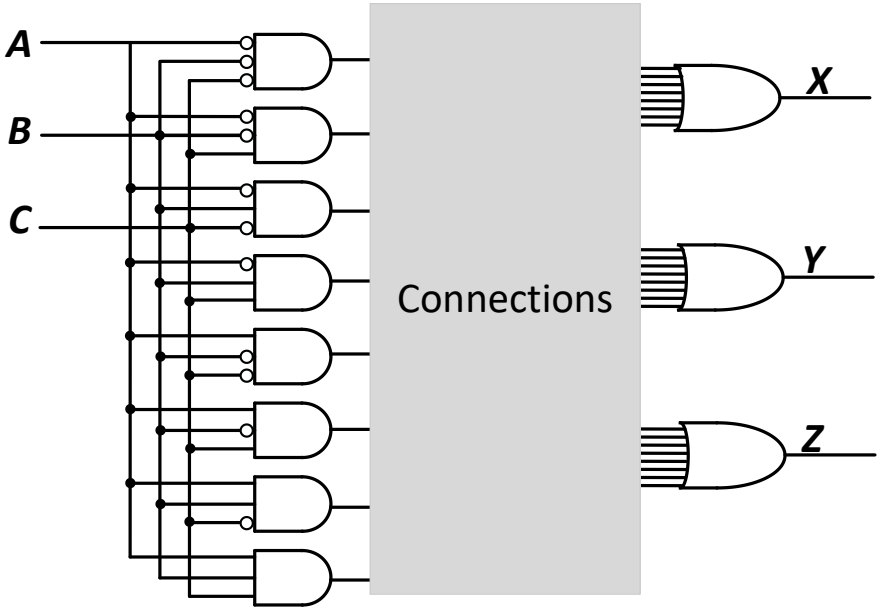
PLA Example Function (III)



Read H&H Chapter 5.6.1

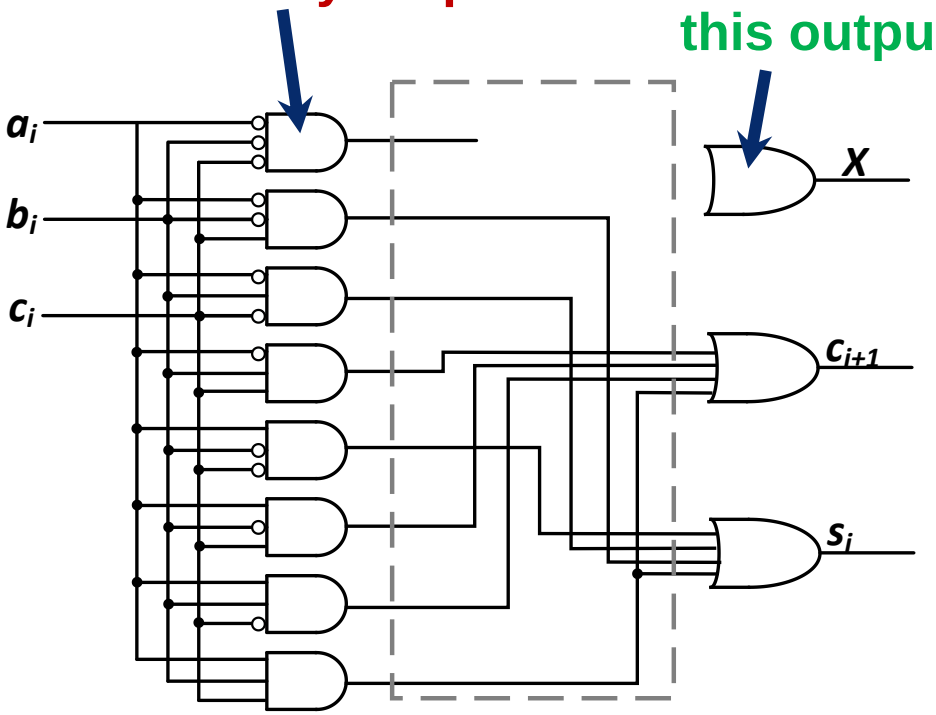
university of groningen

Implementing a Full Adder Using a PLA



This input should not be connected to any outputs

We do not need this output



Truth table of a full adder

a_i	b_i	$carry_i$	$carry_{i+1}$	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Logical Completeness

Logical (Functional) Completeness

- Any logic function we wish to implement could be accomplished with a PLA
  - PLA consists of only AND gates, OR gates, and inverters
  - We just have to program connections based on SOP of the intended logic function

- The set of gates {AND, OR, NOT} is logically complete because we can build a circuit to carry out the specification of any truth table we wish, without using any other kind of gate

- NAND is also logically complete. So is NOR.
  - Your task: Prove this.

More Combinational Blocks

More Combinational Building Blocks

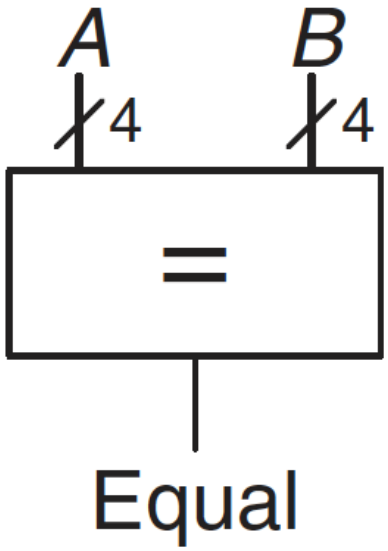
- H&H Chapter 2 in full
  - Required Reading
  - E.g., see Tri-state Buffer and Z values in Section 2.6
- H&H Chapter 5
  - Will be required reading soon
- You will benefit greatly by reading the "combinational" parts of Chapter 5 soon.
  - Sections 5.1 and 5.2
  - E.g., Adder, Subtractor, Comparator, Shifter/Rotator, Multiplier, Divider

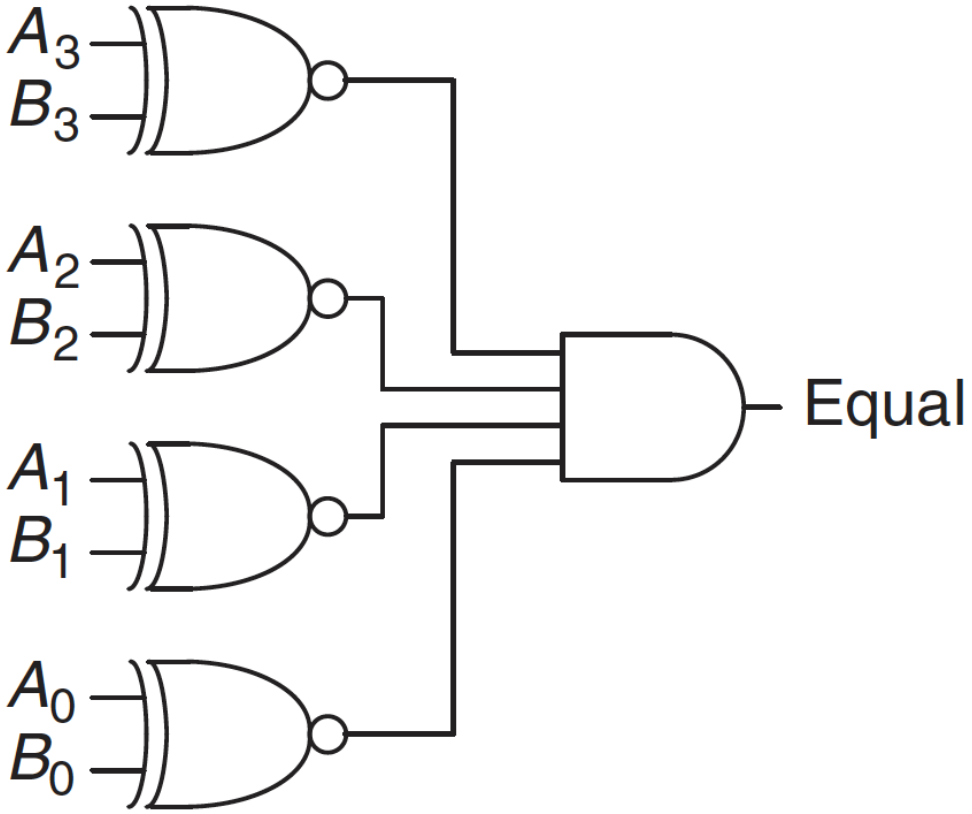
Comparator

university of groningen

Equality Checker (Compare if Equal)

- Checks if two N-input values are exactly the same
- Example: 4-bit Comparator





ALU (Arithmetic Logic Unit)

university of groningen

ALU (Arithmetic Logic Unit)

- Combines a variety of arithmetic and logical operations into a single unit (that performs only one function at a time)
- Usually denoted with this symbol:

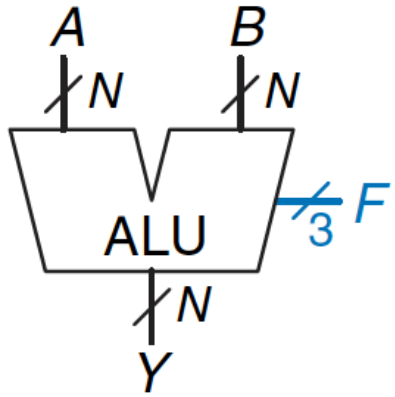


Figure 5.14 ALU symbol

Table 5.1 ALU operations

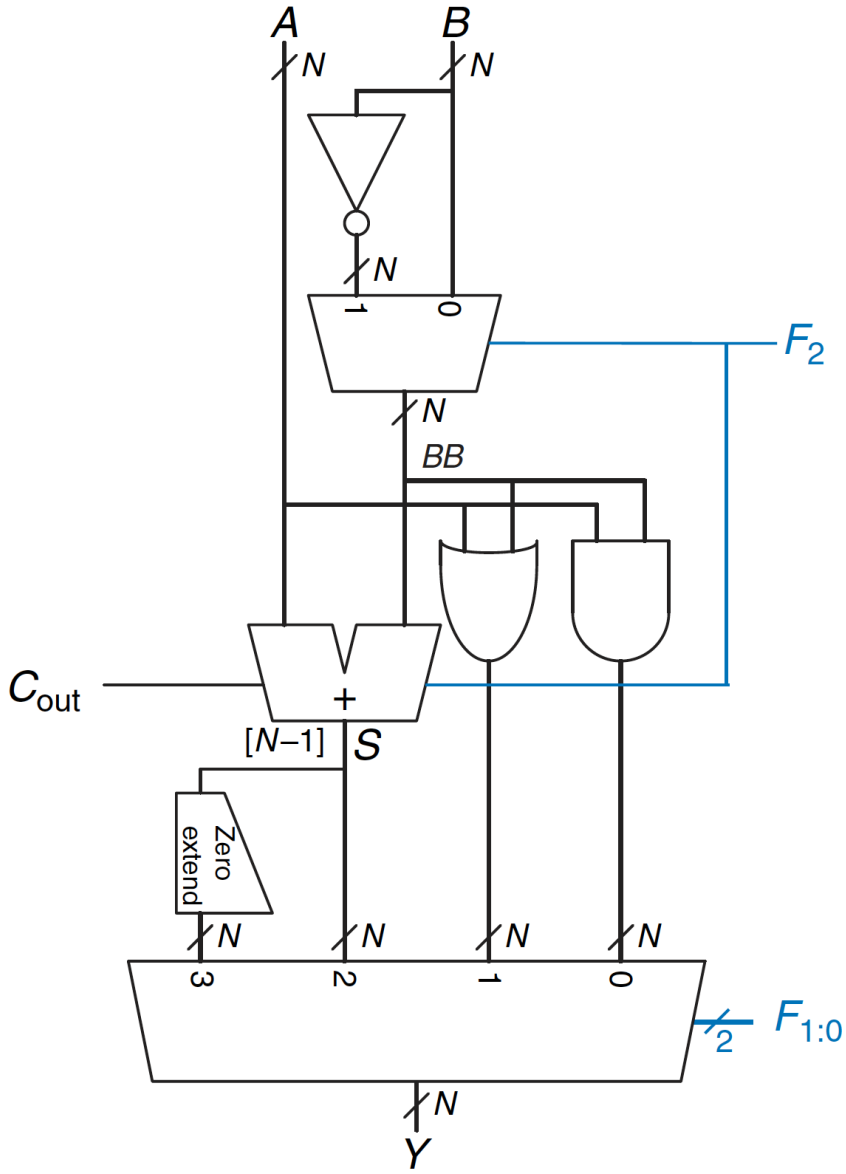
$F_{2:0}$	Function
000	A AND B
001	A OR B
010	A + B
011	not used
100	A AND $\overline{B}$
101	A OR $\overline{B}$
110	A – B
111	SLT

university of groningen

Example ALU (Arithmetic Logic Unit)

Table 5.1 ALU operations

$F_{2:0}$	Function
000	A AND B
001	A OR B
010	A + B
011	not used
100	A AND $\overline{B}$
101	A OR $\overline{B}$
110	A – B
111	SLT



More Combinational Building Blocks

- See H&H Chapter 5.2 for
  - Subtractor (using 2's Complement Representation)
  - Shifter and Rotator
  - Multiplier
  - Divider
  - …

More Combinational Building Blocks

- H&H Chapter 2 in full
  - Required Reading
  - E.g., see Tri-state Buffer and Z values in Section 2.6
- H&H Chapter 5
  - Will be required reading soon
- You will benefit greatly by reading the "combinational" parts of Chapter 5 soon.
  - Sections 5.1 and 5.2
  - E.g., Adder, Subtractor, Comparator, Shifter/Rotator, Multiplier, Divider

university of groningen

Tri-State Buffer

university of groningen

Tri-State Buffer

› A tri-state buffer enables gating of different signals onto a wire

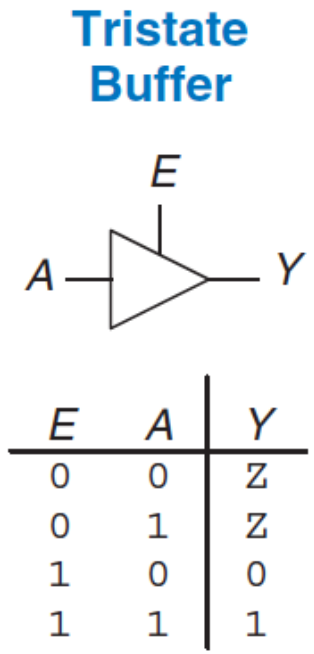


Figure 2.40 Tristate buffer

A tri-state buffer acts like a switch

› Floating signal (Z): Signal that is not driven by any circuit

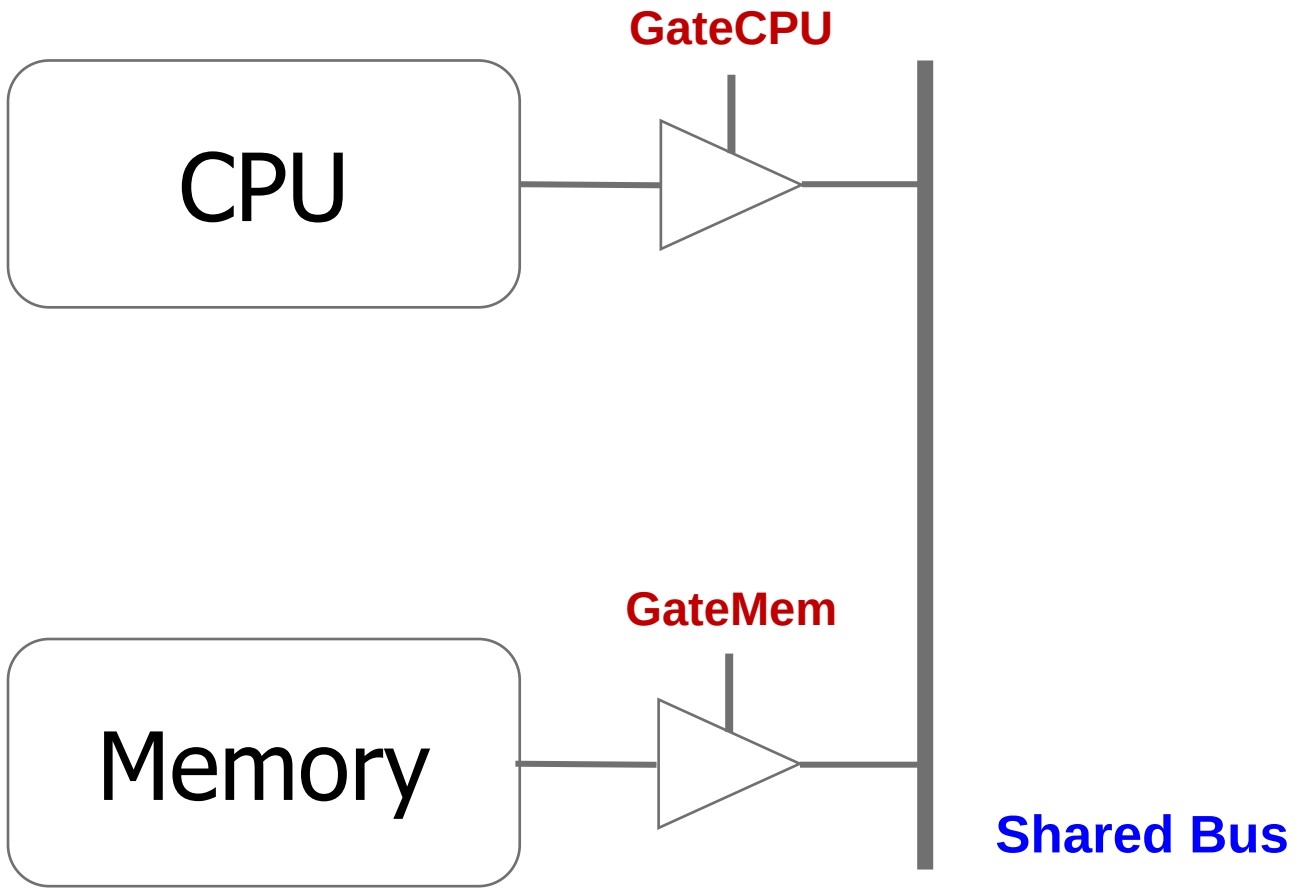
- Open circuit, floating wire

Example: Use of Tri-State Buffers

- Imagine a wire connecting the CPU and memory
  - At any time only the CPU or the memory can place a value on the wire, both not both
  - You can have two tri-state buffers: one driven by CPU, the other memory; and ensure at most one is enabled at any time

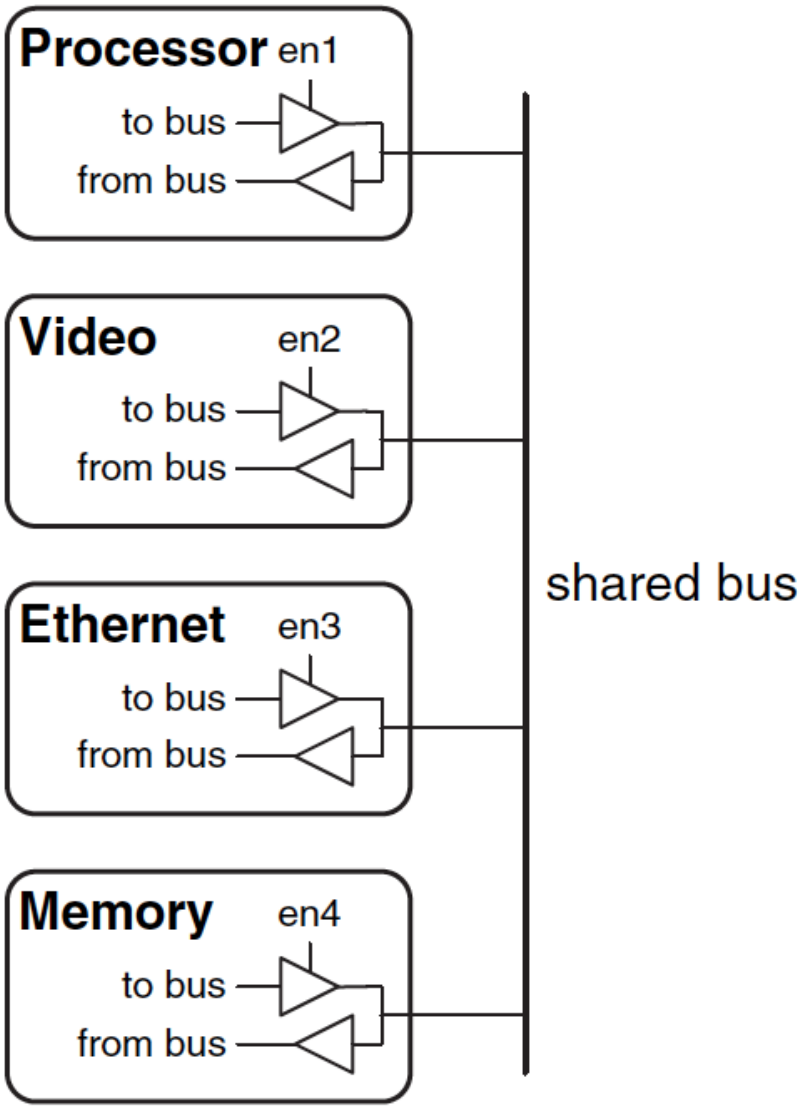
university of groningen

Example Design with Tri-State Buffers



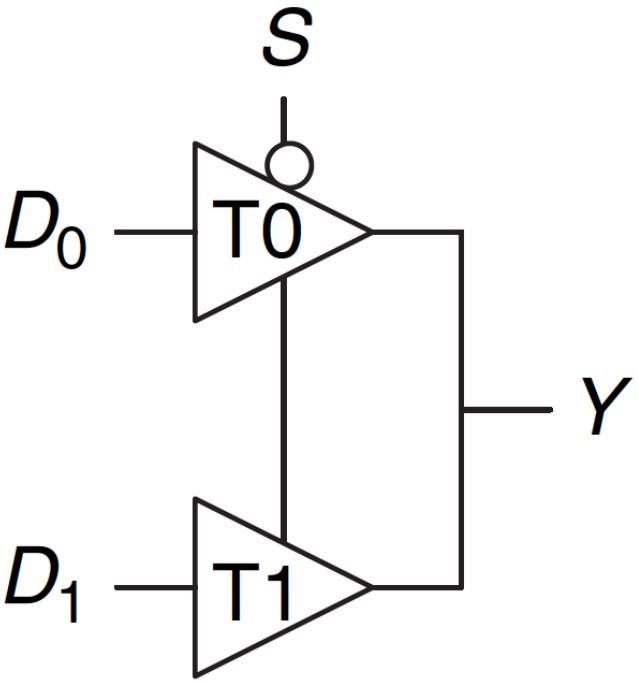
university of groningen

Another Example



university of groningen

Multiplexer Using Tri-State Buffers



$$Y = D_0\bar{S} + D_1 S$$

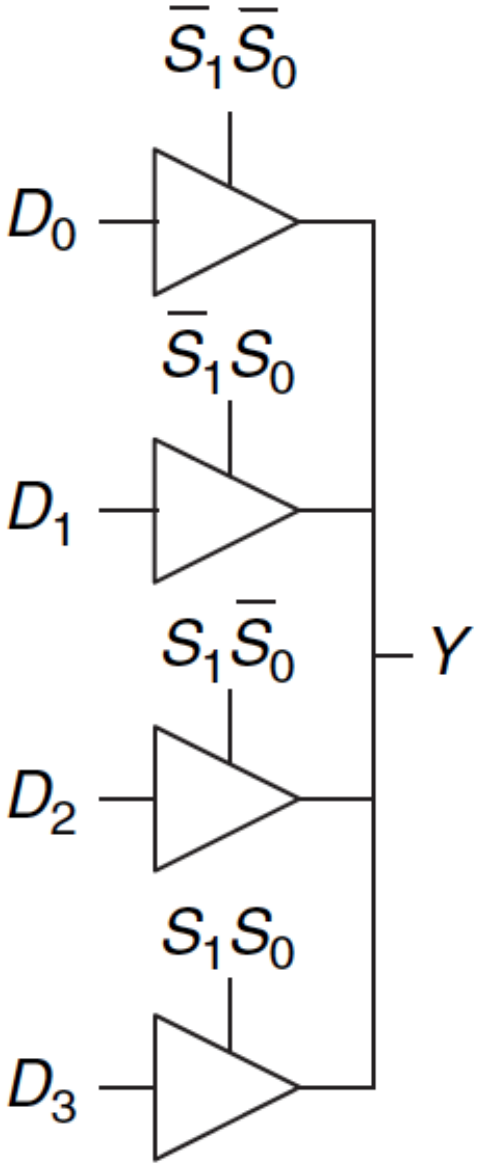
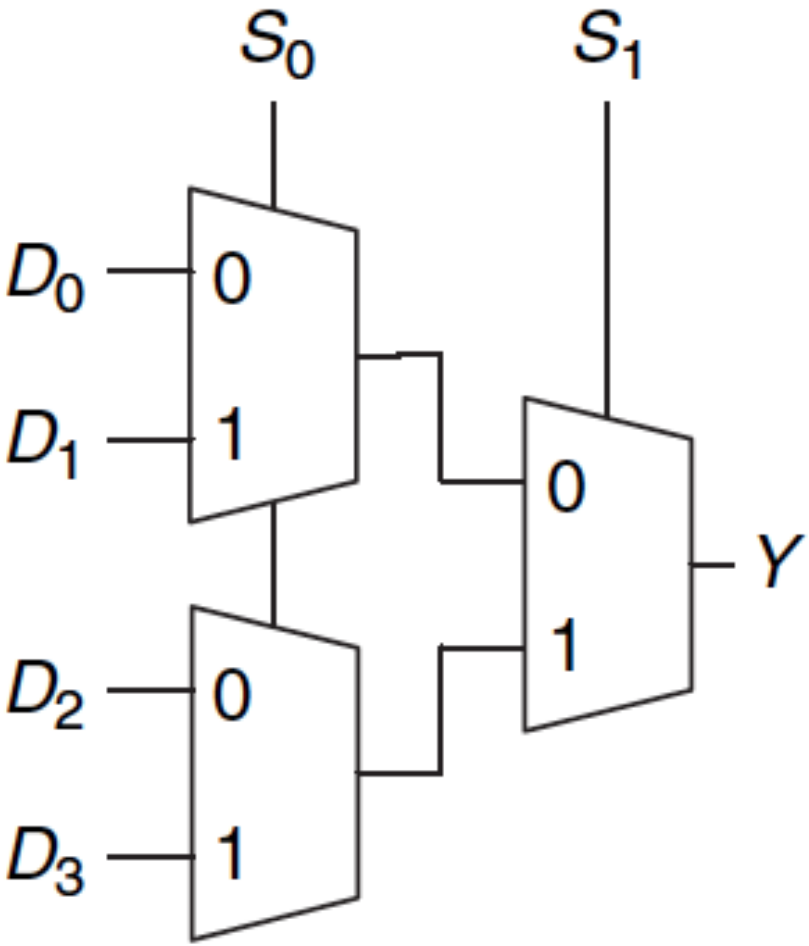
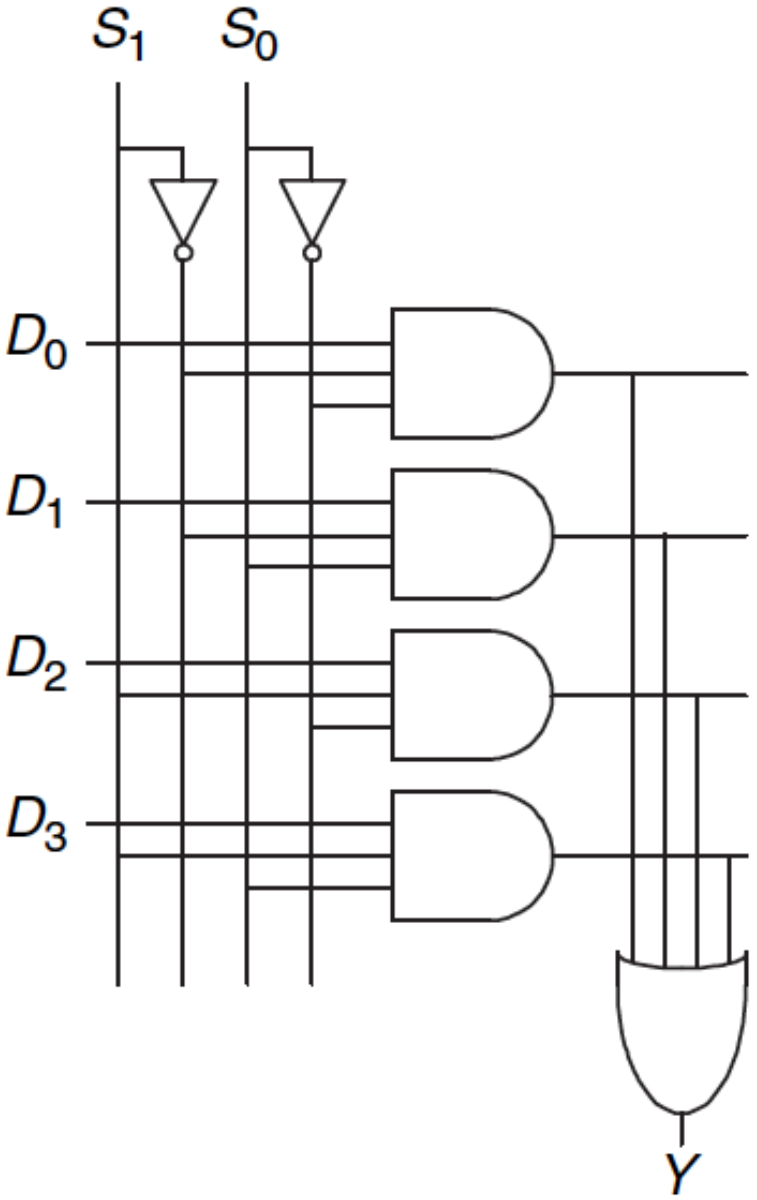


Figure 2.56 Multiplexer using tristate buffers

university of groningen

Recall: A 4-to-1 Multiplexer

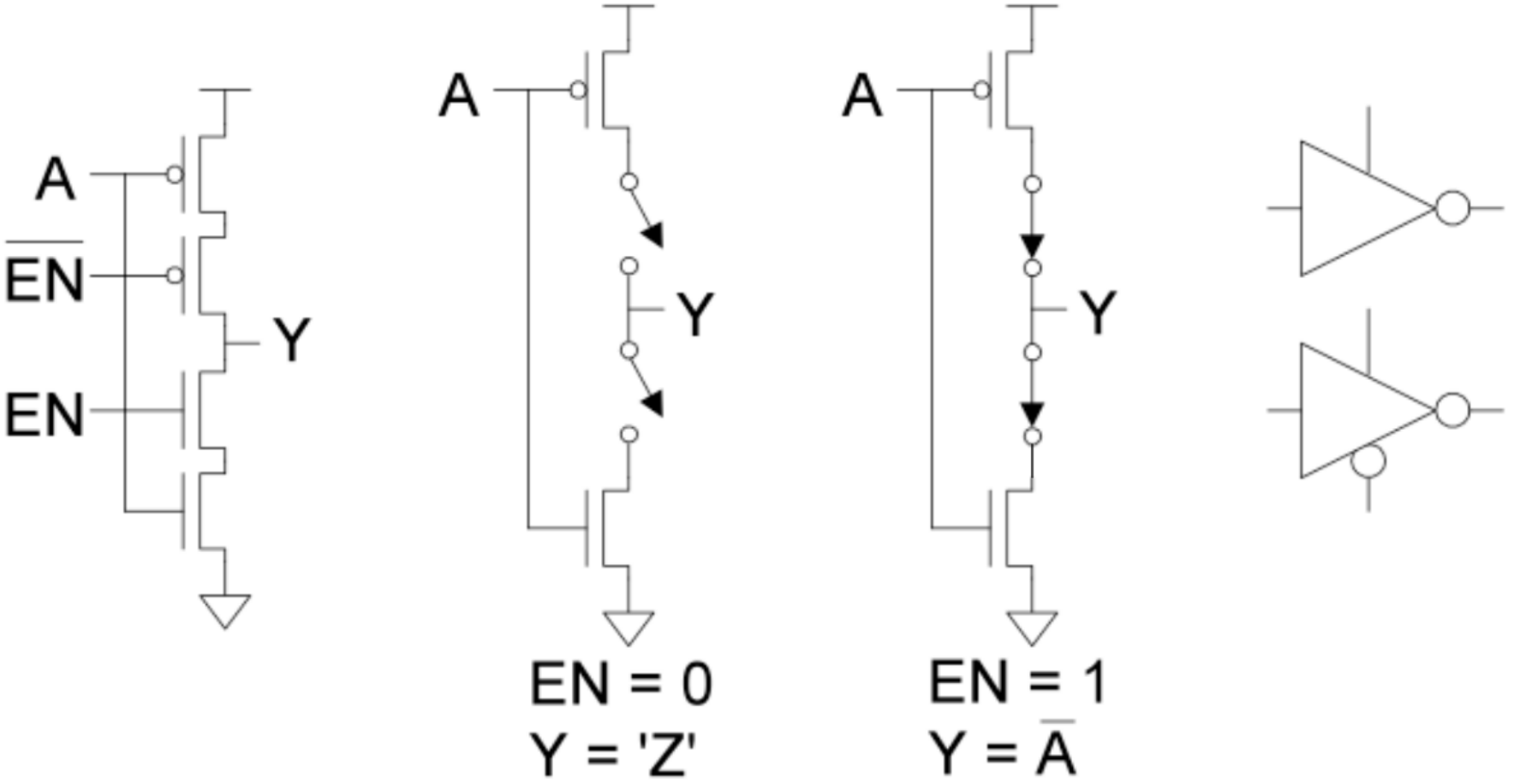




university of groningen

Digging Deeper: Tri-State Buffer in CMOS

› How do you implement a Tri-State Buffer using



http://people.ee.duke.edu/~krish/teaching/Lectures/CMOScircuits_2011.pdf

university of groningen

We Covered Combinational Logic Blocks

- Basic logic gates (AND, OR, NOT, NAND, NOR, XOR)
- Decoder
- Multiplexer
- Full Adder
- Programmable Logic Array (PLA)
- Comparator
- Arithmetic Logic Unit (ALU)
- Tri-State Buffer

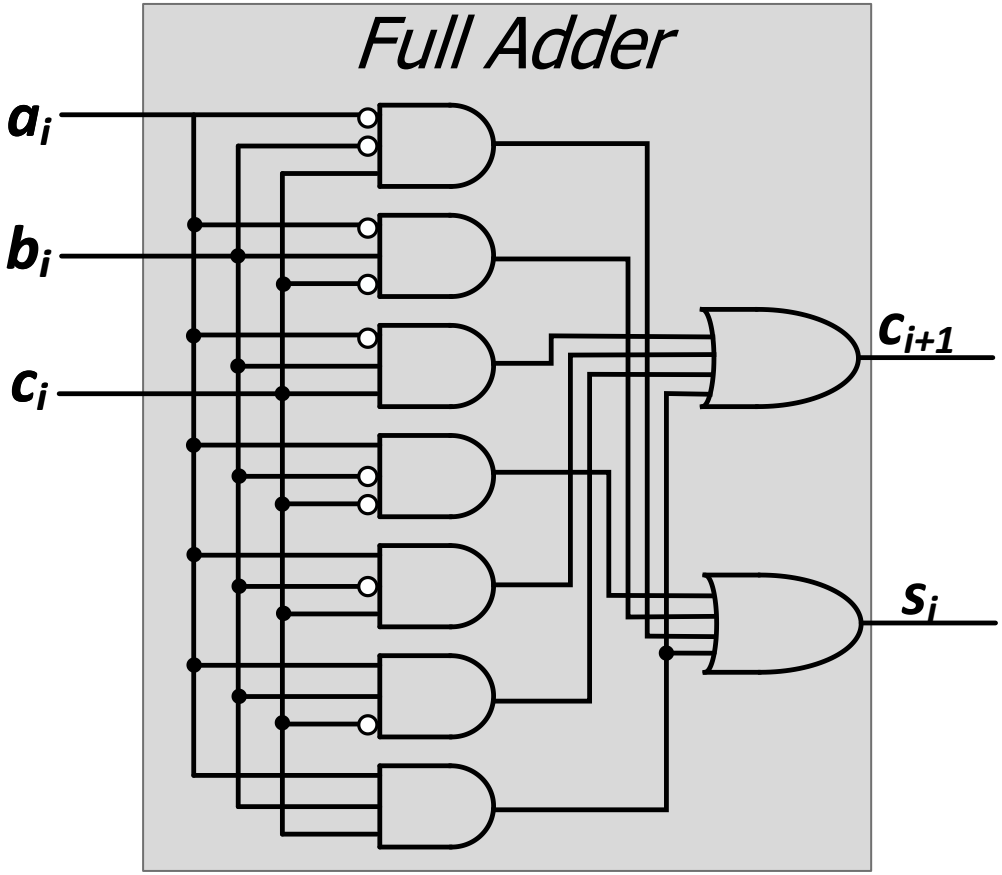
- Standard form representations: SOP & POS
- Logic simplification via Boolean Algebra
- Logical completeness

university of groningen

Logic Simplification using Boolean Algebra Rules

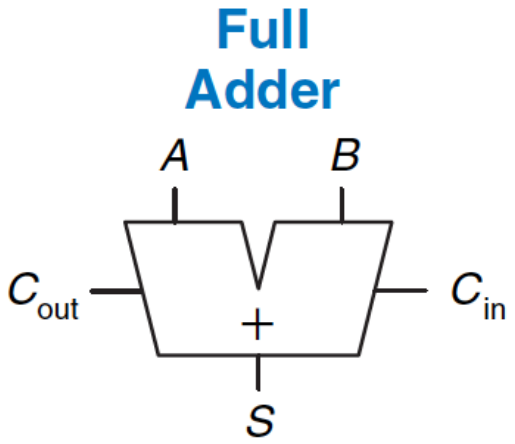
university of groningen

Recall: Full Adder in SOP Form Logic



a_i	b_i	$carry_i$	$carry_{i+1}$	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Goal: Simplified Full Adder



C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S = A \oplus B \oplus C_{in}$$ 3-input XOR

$$C_{out} = AB + AC_{in} + BC_{in}$$ 3-input majority

How do we simplify Boolean logic?

How do we automate simplification?

Quick Recap on Logic Simplification

- The original Boolean expression (i.e., logic circuit) may not be optimal

$$F = \sim A(A + B) + (B + AA)(A + \sim B)$$

- Can we reduce a given Boolean expression to an equivalent expression with fewer terms?

$$F = A + B$$

- The goal of logic simplification:
  - Reduce the number of gates/inputs
  - Reduce implementation cost (and potentially latency & power)

A basis for what the automated design tools are doing today

university of groningen

Logic Simplification

› Systematic techniques for simplifications

- amenable to automation

Key Tool: The Uniting Theorem — $F = A\overline{B} + AB$

A	B	F
0	0	0
0	1	0
1	0	1
1	1	1

$$F = A\overline{B} + AB = A(\overline{B} + B) = A(1) = A$$

B's value changes within the rows where F==1 ("ON set")

A's value does NOT change within the ON-set rows

If an input (B) can change without changing the output, that input value is not needed

→ B is eliminated, A remains

A	B	G
0	0	1
0	1	0
1	0	1
1	1	0

$$G = \overline{A}\overline{B} + A\overline{B} = (\overline{A} + A)\overline{B} = \overline{B}$$

B's value stays the same within the ON-set rows

A's value changes within the ON-set rows

→ A is eliminated, B remains

university of groningen

Logic Simplification

› Systematic techniques for simplifications

- amenable to automation

Key Tool: The Uniting Theorem — $F = A\overline{B} + AB$

A	B	F
0	0	0
0	1	0
1	0	1
1	1	1

$$F = A\overline{B} + AB = A(\overline{B} + B) = A(1) = A$$

Essence of Simplification:

Find two-element subsets of the ON-set where only one variable changes its value. This single varying variable *can be eliminated!*

Value is not needed

→ *B is eliminated, A remains*

A	B	G
0	0	1
0	1	0
1	0	1
1	1	0

$$G = \overline{A}\overline{B} + A\overline{B} = (\overline{A} + A)\overline{B} = \overline{B}$$

B's value stays the same within the ON-set rows

A's value changes within the ON-set rows

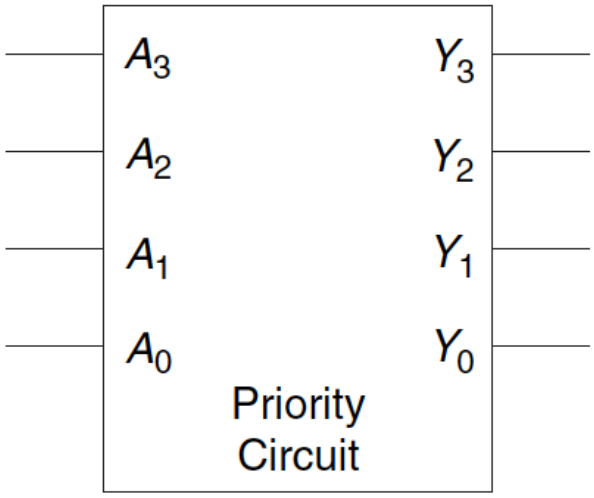
→ *A is eliminated, B remains*

university of groningen

Logic Simplification Example: Priority Circuit

› Priority Circuit

- Inputs: “Requestors” with priority levels
- Outputs: “Grant” signal for each requestor
- Example 4-bit priority circuit
- Real life example: Imagine a bus requested by 4 processors



A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0

university of groningen

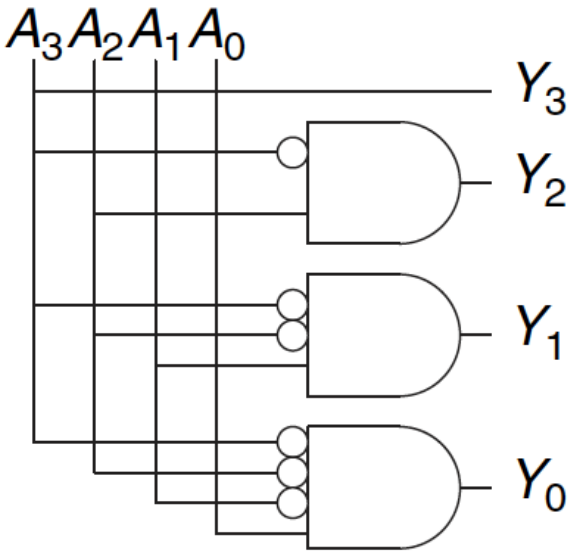
Simplified Priority Circuit

- Priority Circuit
  - Inputs: "Requestors" with priority levels
  - Outputs: "Grant" signal for each requestor
  - Example 4-bit priority circuit

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	X	0	0	1	0
0	1	X	X	0	1	0	0
1	X	X	X	1	0	0	0

Figure 2.29 **Priority circuit truth table with don't cares (X's)**



X (Don't Care) means *I don't care what the value of this input is*



university of groningen

We Are Done with Combinational Logic

- Building blocks of modern computers
  - Transistors
  - Logic gates
- Combinational circuits
- Boolean algebra
- Using Boolean algebra to represent combinational circuits
- Basic combinational logic blocks
- Simplifying combinational logic circuits